

DOSSIER DE CONCEPTION

Document de validation — à approuver avant tout développement



Alanya

Comptes business, certification,
compte officiel et listes de contacts

Six volets — approche fonctionnelle et approche technique

Périmètre Alanya(Flutter) ■ Alanya-Backend ■ Alanya Admin

Statut Conception — aucune ligne de code écrite

Échelle visée Centaines de milliers de comptes

Date 30 juillet 2026

Document confidentiel — diffusion restreinte

Résumé

Cinq fonctionnalités ont été demandées pour Alanya : les **comptes business** avec leur fiche étendue, la **certification des comptes**, un **compte officiel** au nom de l'application, les **listes de contacts préférés**, et l'**identité par QR code** — ajout de contact instantané et connexion multi-appareils.

La première décision de ce dossier est de constater que trois d'entre elles ne sont pas trois systèmes différents. Comptes business, certification et compte officiel répondent à la même question — *qui est ce compte ?* — et se ramènent à **deux axes indépendants** portés par un socle commun : ce que le compte *est* (personnel, business, officiel), et ce qu'Alanya *atteste* à son sujet (aucune démarche, en attente, vérifié, refusé, révoqué, expiré). Les concevoir séparément aurait produit trois systèmes de badges concurrents et une logique d'affichage impossible à tenir.

La seconde décision structurante est de **séparer l'identité de l'argent**. La vérification répond à « est-ce bien qui il prétend être ? » ; l'abonnement répond à « qu'est-ce qu'il a payé ? ». Les garder distincts permet de livrer la certification **gratuitement** en version 1, sans avoir arrêté l'offre commerciale — celle-ci devient une règle de configuration, pas une reprise de schéma.

Le dossier se lit en deux parties. La **partie I** décrit ce que chaque fonctionnalité change pour l'utilisateur et pour le client, sans vocabulaire technique, maquettes à l'appui. La **partie II** décrit comment on la construit : modèle de données, machines à états, impact fichier par fichier sur les trois dépôts.

Méthode. Toutes les affirmations sur l'existant ont été vérifiées en lisant le code des trois dépôts. Les chemins de fichiers cités sont réels. Là où une information manque, c'est écrit. Là où une décision de conception antérieure s'est révélée incompatible avec le code, l'écart est signalé et argumenté.

Comment lire ce dossier

Les six volets

Chaque fonctionnalité est traitée dans les *deux* parties. L'ordre suit les dépendances techniques, pas l'ordre dans lequel les fonctionnalités ont été demandées.

N°	Volet	Rôle
1	Socle de compte	La fondation : deux axes, un badge dérivé. Les volets 3, 4 et 5 en dépendent.
2	Listes de contacts	Indépendante des autres. Ranger ses favoris, en faire un groupe.
3	Compte officiel et diffusion	Messages et statuts adressés à une partie de la base.
4	Comptes business	Ce que contient une fiche : lieu, médias, horaires, catalogue.
5	Certification	Le dossier, son instruction, le sort des pièces.
6	Identité et connexion QR	Ajout de contact instantané par scan, et connexion d'un second appareil. Indépendante des cinq autres.

Les conventions du document

Encadré bleu. Une explication, une justification de choix, ou un point de méthode. Utile mais non bloquant.

Encadré vert. Un fait établi en lisant le code des trois dépôts, avec son chemin de fichier. C'est ce qui distingue une contrainte réelle d'une supposition.

Encadré rouge. Un point de vigilance : un piège, une contrainte technique dure, ou une décision dont l'oubli coûterait cher. À lire avant de commencer le développement concerné.

Ce qui reste à trancher

Chaque volet se termine par ses questions ouvertes. Elles ne bloquent pas la validation de l'architecture, mais elles doivent être arbitrées avant l'implémentation du volet concerné.

Table des matières

Résumé	1
Comment lire ce dossier	2
I Approche naïve	11
VOLET 1 — Socle de compte	
1 Ce que l'on ajoute, et pourquoi	13
1.1 La situation aujourd'hui	13
1.2 Deux questions, pas une	13
1.3 Ce que couvre l'étape 1 — et ce qu'elle ne couvre pas	13
2 Ce que l'utilisateur voit	14
2.1 Le principe du badge	14
2.2 Les cinq rendus	14
2.3 Où le badge apparaît	15
2.4 Pourquoi la distinction déclaré / vérifié est critique	15
2.5 Protéger le badge des contournements	16
3 Les règles de vie d'un compte	17
3.1 Devenir un compte business, et revenir en arrière	17
3.2 La vie d'une vérification	17
3.3 L'expiration et sa relance	17
3.4 Le compte officiel	18
3.5 Un compte business ne peut pas être administrateur	18
VOLET 2 — Listes de contacts	
4 Le besoin	20
4.1 Ce qui existe déjà	20
4.2 Ce qu'on ajoute	20
4.3 Les règles produit	20
5 Les écrans	21
5.1 Vue d'ensemble	21

5.2	Écran 1 — Contacts préférés, filtrés	21
5.3	Écran 2 — Gestion des listes	22
5.4	Écran 3 — Détail d'une liste	22
5.5	Points d'entrée	22

VOLET 3 — Compte officiel et diffusion

6	Ce que l'on ajoute	24
6.1	Le besoin	24
6.2	Ce qui a déjà été décidé	24
7	Cibler une diffusion	25
7.1	Un critère, pas un choix parmi trois	25
7.2	Savoir qui recevra, avant d'envoyer	25
7.3	L'historique	26
8	Ce que voit l'utilisateur	27
8.1	Un message diffusé	27
8.2	Un statut diffusé	27

VOLET 4 — Comptes business

9	Ce qu'un commerçant peut présenter	29
9.1	La fiche	29
9.2	Le lieu et la géolocalisation	29
9.3	Médias de présentation	30
9.4	Liens et présence en ligne	30
9.5	Les quatre écrans	31
9.6	Le badge fait son travail	31
10	La catégorie et les mots-clés	32
10.1	Une liste courte, pas une arborescence	32
10.2	Une taxonomie est difficile à changer	32
11	Les horaires, l'écran le plus difficile	33
12	Le tableau de bord du commerçant	34
12.1	Ce qu'il montre	34
12.2	Ce que cela implique	34

VOLET 5 — Certification

13	Ce que la certification dit, et ne dit pas	36
13.1	Une seule question	36

13.2 Qui peut être vérifié	36
14 Le parcours du demandeur	37
14.1 Trois états	37
14.2 Les pièces demandées	37
14.3 Ce qui est promis, et ce qui ne l'est pas	38
15 Le parcours de l'administrateur	39
15.1 L'écran d'instruction	39
15.2 Trois issues, pas deux	39
15.3 Qui décide	39
VOLET 6 — Identité et connexion par QR code	
16 Le besoin	41
16.1 Ce qui existe déjà	41
16.2 Ce qu'on ajoute	41
16.3 Les règles produit	42
17 Mon code personnel	43
17.1 Vue d'ensemble	43
17.2 Écran — Mon code	44
17.3 Être prévenu, ajouter en retour	45
17.4 Reconnaître un contact ajouté par QR	45
17.5 Partager son code	46
17.6 Écran — Scanner	46
17.7 Points d'entrée	46
18 Connexion d'un second appareil	47
18.1 Le principe	47
18.2 Écran — Se connecter par QR	47
18.3 Écran — Confirmer la connexion	49
18.4 Écran — Mes appareils connectés	49
18.5 Points d'entrée	51
II Approche technique	52
VOLET 1 — Socle de compte	
19 Modèle de données	54
19.1 Pourquoi étendre <code>users</code> plutôt que créer une table <code>accounts</code>	54
19.2 La migration	54
19.3 Les énumérations, côté Node	55

19.4 Raccordement au schéma existant	55
20 Dérivation du badge	56
20.1 Une fonction pure, trois implémentations identiques	56
20.2 Le piège à corriger avant tout	57
20.3 Composant existant à ne pas confondre	57
21 Règles appliquées par le serveur	58
21.1 Exclusion business / administrateur	58
21.2 Validation des noms d’affichage	58
21.3 Interdire la réponse au compte officiel	59
21.4 Expiration : réutiliser le mécanisme existant	59
22 Impact fichier par fichier	61
22.1 Backend — Alanya-Backend	61
22.2 Mobile — Talky (Flutter)	62
22.3 Administration — Alanya Admin	62
23 Points de vigilance et questions ouvertes	63
23.1 À vérifier avant d’appliquer la migration	63
23.2 Collision de numérotation	63
23.3 Écart assumé avec la conception initiale	63
23.4 Ce qui reste à décider	63
VOLET 2 — Listes de contacts	
24 Modèle de données	65
24.1 Deux tables	65
24.2 L’appartenance n’est pas contrainte par la base	66
25 API	67
25.1 Contrôleur — <code>src/controllers/contactListController.js</code>	67
25.2 Routes — <code>src/routes/contactLists.js</code>	67
26 Client Flutter	68
26.1 Couche données	68
26.2 Interface	69
27 Arbitrage : liste de contacts <i>n’est pas</i> audience de diffusion	70
28 Vérification	71
28.1 Backend	71
28.2 Client	71
28.3 Points ouverts	71

VOLET 3 — Compte officiel et diffusion

29 La stratégie de livraison	73
29.1 Le désaccord à trancher	73
29.2 Les trois options, chiffrées	73
29.3 Le moment de la matérialisation	74
29.4 Le garde-fou contre la dérive	74
30 Modèle de données	75
30.1 Migration 025_broadcast.sql	75
31 Le résolveur de critères	77
31.1 Format stocké	77
31.2 Deux évaluateurs pour un même critère	77
31.3 Sécurité : compiler n'est pas concaténer	78
31.4 Ce qu'on n'accepte pas comme critère	78
32 Les statuts officiels	79
32.1 Le correctif de visibilité	79
32.2 Les statuts n'ont pas besoin de matérialisation	79
33 La file de jobs	80
33.1 Pourquoi elle devient obligatoire ici	80
33.2 Table plutôt que Redis	80
33.3 Consommation concurrente	80
33.4 Découpage du fan-out	81
34 Impact fichier par fichier	82
34.1 Backend	82
34.2 Administration	82
34.3 Mobile	83
35 Points de vigilance et questions ouvertes	84
35.1 Chantier d'infrastructure nommé	84
35.2 Coût de la matérialisation sur le chemin chaud	84
35.3 Questions ouvertes	84

VOLET 4 — Comptes business

36 Modèle de données	86
36.1 Vue d'ensemble	86
36.2 Le profil	86
36.3 Liens, médias, mots-clés	87

36.4 Horaires	88
36.5 Catalogue	89
37 Un lieu, ou plusieurs	90
37.1 Ce qui est retenu	90
37.2 La variante multi-lieux	90
38 Annuaire et recherche de proximité	91
38.1 L'état actuel	91
38.2 La requête	91
38.3 « Ouvert maintenant »	91
39 Tableau de bord d'audience	92
39.1 Journal d'événements	92
39.2 Ce que le commerçant voit	92
40 Médias : stockage et transcodage	93
40.1 Ce que l'existant sait déjà faire	93
40.2 Trois problèmes que la fiche business introduit	93
40.3 La chaîne de traitement	93
40.4 Limites à fixer	93
41 Impact fichier par fichier	95
41.1 Backend	95
41.2 Mobile	95
42 Points de vigilance et questions ouvertes	96
42.1 Chantiers d'infrastructure nommés	96
42.2 Questions ouvertes	96
VOLET 5 — Certification	
43 Modèle de données	98
43.1 Ce que le socle a déjà posé	98
43.2 Migration 027_verification.sql	98
43.3 La machine à états du dossier	99
44 Sécurité des pièces	101
44.1 Chiffrement au repos	101
44.2 La purge	101
45 Abonnement et paiement — conçus, dormants	102
45.1 Pourquoi les concevoir maintenant	102
45.2 Renouvellement manuel, pas prélèvement	103

45.3 La règle à ne jamais enfreindre	103
45.4 Une contrainte de plateforme à anticiper	103
46 Impact fichier par fichier	104
46.1 Backend	104
46.2 Administration et mobile	104
47 Points de vigilance et questions ouvertes	105
47.1 Chantiers nommés	105
47.2 Questions ouvertes	105
VOLET 6 — Identité et connexion par QR code	
48 Enveloppe de payload et résolution	107
49 Modèle de données	108
49.1 Le jeton d'identité permanent, et son verrou	108
49.2 La nature de <code>device_id</code>	108
49.3 Le jeton et la table <code>appareils</code>	109
49.4 L'origine d'un contact	110
50 Rendre la révocation réelle	112
50.1 Ce que fait vraiment la révocation	113
51 Ajout de contact instantané	114
51.1 Le jeton éphémère, en mémoire	114
51.2 Résolution et consommation	115
51.3 Prévenir le propriétaire	116
51.4 L'ajout en retour	116
52 La session de connexion éphémère	117
52.1 Une structure en mémoire, pas une table	117
52.2 Deux secrets par session, jamais interchangeables	117
52.3 Cycle de vie	118
52.4 L'exception d'authentification socket	119
53 API	120
53.1 Limitation de débit	121
54 Client Flutter	122
54.1 Couche données	122
54.2 Interface	124
54.3 Mon code : décompte, consommation, régénération	125
54.4 La pastille, rendue à un seul endroit	125

54.5 Le dialogue global d'ajout en retour	125
54.6 Liens profonds	126
55 Impact fichier par fichier	127
55.1 Backend — Alanya-Backend	128
55.2 Mobile — Talky (Flutter)	130
56 Vérification	132
56.1 Backend	132
56.2 Client	132
57 Points de vigilance et questions ouvertes	134
Ce qui reste à construire	136

PARTIE I

Approche naïve

VOLET 1

Socle de compte

Deux axes indépendants, un badge dérivé, et les règles de vie d'un compte

CHAPITRE 1

Ce que l'on ajoute, et pourquoi

1.1 La situation aujourd'hui

Dans Alanya, tous les comptes sont identiques. Un commerçant, une personnalité publique et un utilisateur ordinaire ont exactement le même profil et la même apparence dans une liste de conversations. Rien ne permet de distinguer l'entreprise officielle d'un imitateur qui aurait pris le même nom et la même photo.

C'est ce manque que les trois fonctionnalités demandées viennent combler — et c'est pourquoi elles partagent la même fondation.

1.2 Deux questions, pas une

L'erreur naturelle serait de créer une liste de « genres de comptes » : personnel, business, vérifié, officiel, business vérifié... Cette liste devient ingérable dès qu'on ajoute un cas.

On sépare donc deux questions **indépendantes** :

Quel genre de compte ?	Personnel, business, ou officiel. C'est ce que le compte <i>est</i> .
A-t-il été vérifié ?	Aucune démarche, en attente, vérifié, refusé, révoqué, ou expiré. C'est ce qu'Alanya <i>atteste</i> à son sujet.

Ces deux questions ne dépendent pas l'une de l'autre. Un commerce peut exister sans être vérifié. Une personne privée peut être vérifiée sans être un commerce — c'est le cas d'une personnalité publique. Le compte officiel Alanya, lui, est la combinaison « officiel + vérifié d'office ».

Le bénéfice concret. Poser deux questions séparées plutôt qu'une seule liste, c'est ce qui permet d'ajouter demain un nouveau genre de compte, ou un nouvel état de vérification, sans toucher à l'autre axe ni refaire les écrans.

1.3 Ce que couvre l'étape 1 — et ce qu'elle ne couvre pas

Dans cette étape	Savoir <i>qui</i> est un compte : son genre, son état de vérification, et le badge qui en découle partout dans l'application.
Étapes suivantes	Ce que ce compte <i>contient</i> : la fiche business (horaires, adresse, catalogue, liens), le dossier de certification (pièces justificatives, approbation, abonnement), et la diffusion du compte officiel.

La raison de cette coupure est simple : le genre et l'état d'un compte suffisent déjà à afficher le bon badge partout. Le contenu détaillé d'une fiche business dépend de règles qu'on n'a pas encore écrites — le figer maintenant obligerait à tout refaire à l'étape suivante.

CHAPITRE 2

Ce que l'utilisateur voit

2.1 Le principe du badge

Un badge qui combinerait « c'est un commerce » et « c'est vérifié » en un seul dessin devient illisible à la taille où il apparaît réellement — environ 12 pixels, à côté d'un nom dans une liste de conversations. Et deux badges côte à côte mangent la largeur du nom, qui est déjà tronqué sur un petit écran.

La solution retenue sépare les deux informations sur deux canaux visuels distincts :

Le glyphe dit <i>quoi</i>	Une coche pour une personne notable, un panier pour un commerce, la marque Alanya pour le compte officiel.
Le contenant dit <i>vérifié</i> ou <i>pas</i>	Un glyphe nu = déclaré par le compte lui-même. Un glyphe posé dans un sceau festonné  = attesté par Alanya.

Pourquoi un sceau festonné, et pas un rond

Un disque plein ne se distingue d'aucune autre icône circulaire de l'interface : avatars, puces de comptage, indicateurs de présence. À 12 pixels, un badge rond devient un point coloré parmi d'autres points colorés.

Le feston à douze lobes, lui, a une silhouette reconnaissable même quand le glyphe qu'il contient n'est plus lisible. C'est la propriété qui compte : l'utilisateur qui distingue la *forme* sait qu'Alanya s'est prononcé sur ce compte, sans avoir à lire le détail. Et cette forme est bien plus difficile à imiter avec un caractère Unicode dans un nom d'affichage qu'un simple rond ou une simple coche.

2.2 Les cinq rendus

Genre	État	À taille réelle	Agrandi
Personnel	non vérifié	—	—
Personnel	vérifié		
Business	déclaré		
Business	vérifié		
Officiel	—		

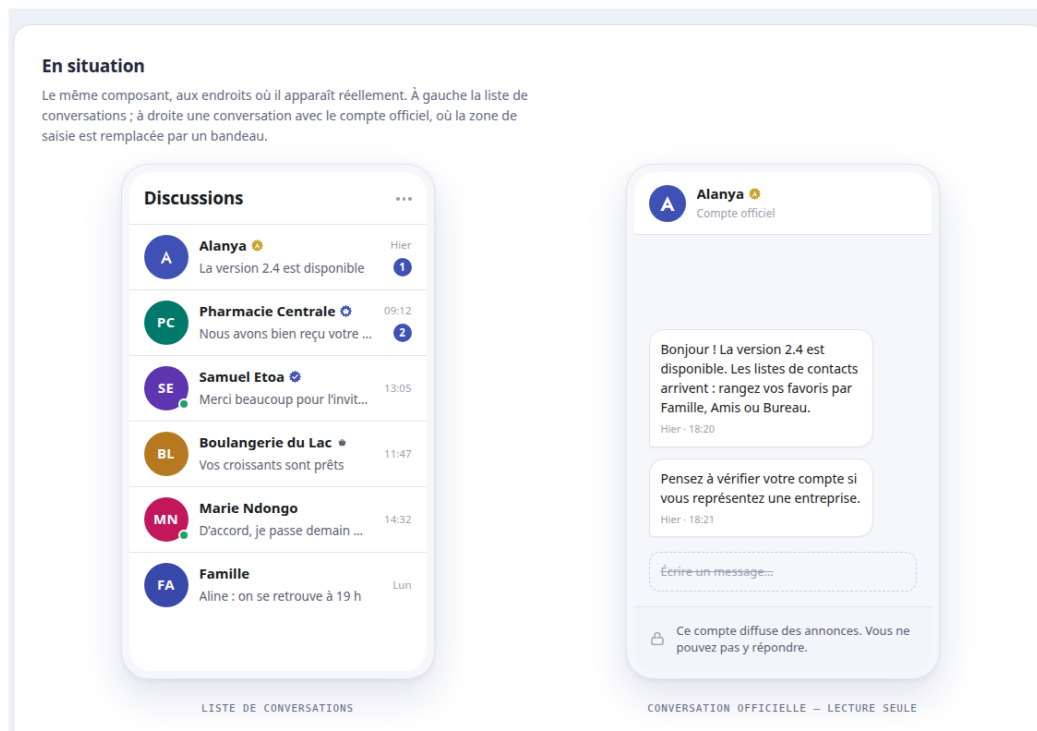
La colonne « à taille réelle » est là pour être jugée telle quelle : c'est exactement la taille à laquelle le badge apparaîtra à côté d'un nom. Si un rendu n'est pas distinguable à cette échelle, il doit être redessiné avant l'implémentation.

Pourquoi une couleur différente pour l'officiel. Le sceau du compte officiel est doré, pas indigo. Il n'existe qu'un seul compte officiel dans toute l'application : lui donner une couleur à part évite qu'il soit confondu avec un compte simplement vérifié, et rend l'usurpation visuellement plus difficile.

2.3 Où le badge apparaît

Le même badge, dessiné par le même composant, dans quatre contextes :

- la **liste de conversations**, juste après le nom ;
- l'**en-tête d'une conversation** ouverte ;
- la **page de profil** du compte ;
- les **résultats de recherche** et les listes de contacts.



Maquette interactive. La version vivante de cette maquette permet de faire varier la taille du badge de 10 à 28 pixels pour juger la lisibilité, et montre la conversation officielle avec son bandeau de lecture seule :

claude.ai/code/artifact/029627c6

Source versionnée : docs/architecture/maquettes/liste-conversations-badges.html

2.4 Pourquoi la distinction déclaré / vérifié est critique

Se déclarer « commerce » prend trente secondes et n'engage personne : le compte l'affirme lui-même, personne ne l'a contrôlé. Être vérifié suppose au contraire un dossier examiné par Alanya.

Si les deux badges avaient le même poids visuel, n'importe qui créerait un compte business et hériterait d'une caution qui ne lui a jamais été accordée. C'est exactement le scénario qu'un arnaqueur cherche.

La règle qui en découle. Le sceau festonné est la signature d'Alanya. Rien d'autre dans l'interface ne doit pouvoir lui ressembler — ni une icône décorative, ni un emoji dans un nom d'utilisateur.

2.5 Protéger le badge des contournements

Un système de badge peut être contourné sans jamais être attaqué techniquement : il suffit qu'un utilisateur s'appelle « Alanya ✓ » pour que le nom affiché imite la certification.

Deux protections sont donc posées dès cette étape :

1. les caractères de type coche, étoile et sceau sont **refusés** dans le nom et le pseudo ;
2. les variantes du nom « Alanya » sont **réservées** et ne peuvent pas être prises par un compte ordinaire.

CHAPITRE 3

Les règles de vie d'un compte

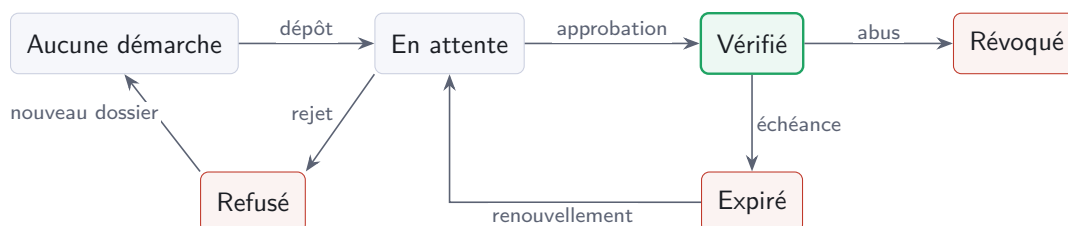
3.1 Devenir un compte business, et revenir en arrière

Le passage d'un compte personnel à un compte business est **réversible**. Les règles proposées :

Personnel → business	Libre, à l'initiative de l'utilisateur. Devenir commerce est déclaratif : cela n'atteste de rien et ne demande donc aucune validation.
Business → personnel	Libre également. La fiche business est conservée mais masquée , pas supprimée : un retour en arrière ne doit pas détruire un catalogue de produits saisi pendant des semaines.
Une vérification en cours	La demande est annulée : elle portait sur une identité d'entreprise qui n'est plus revendiquée.
Une vérification obtenue	Elle est révoquée , pas suspendue. Le badge atteste d'une entreprise ; si le compte n'en est plus une, l'attestation n'a plus d'objet. Un nouveau dossier sera nécessaire pour la retrouver.

À confirmer avec le client. La bascule business → personnel pourrait aussi être soumise à validation d'un administrateur, pour éviter qu'un commerce échappe à un signalement en repassant en personnel. La règle proposée ci-dessus est la plus simple ; l'alternative est plus sûre.

3.2 La vie d'une vérification



3.3 L'expiration et sa relance

Une vérification n'est pas éternelle : elle a une date de fin. Le comportement attendu :

À J – 30	Le titulaire est prévenu qu'il doit renouveler, par trois canaux à la fois : une notification sur son téléphone, un e-mail, et un avertissement visible dans l'application. L'avertissement n'est envoyé qu'une fois.
À l'échéance	La vérification est suspendue . Le badge disparaît. Le compte et son contenu restent intacts — seule l'attestation tombe.
Après	Le titulaire peut déposer un nouveau dossier pour retrouver son badge.

Suspendu, pas supprimé. Une vérification expirée est un état distinct d'un refus. Le compte a bien été vérifié par le passé ; il a simplement laissé passer la date. Cette nuance compte pour le service client, et pour ne pas retraiter un dossier déjà instruit.

3.4 Le compte officiel

Le compte officiel est le compte au nom de l'application. Ses règles :

- il est **créé par un administrateur**, jamais par une inscription ordinaire ;
- sa photo de profil est le **logo Alanya** ;
- il porte le badge officiel, distinct du badge de vérification ;
- les utilisateurs **ne peuvent pas lui répondre**. Ses messages sont des diffusions, pas des conversations.

Concrètement, dans une conversation avec le compte officiel, la zone de saisie est remplacée par un bandeau explicatif, et les actions « répondre » et « réagir » n'apparaissent pas sur ses messages.

Ce n'est pas qu'une question d'affichage. Masquer la zone de saisie ne suffit pas : un client modifié pourrait envoyer le message quand même. Le refus doit être appliqué par le serveur. La partie II décrit où.

3.5 Un compte business ne peut pas être administrateur

Décision prise : les rôles d'administration de la plateforme et les genres de compte sont mutuellement exclusifs. Un compte business ou officiel ne peut pas recevoir de droits d'administration, et réciproquement.

La raison est de sécurité : un compte visible du public, susceptible d'être usurpé ou compromis, ne doit pas ouvrir la porte au back-office.

VOLET 2

Listes de contacts

Ranger ses favoris dans des listes nommées, et en faire une conversation de groupe

CHAPITRE 4

Le besoin

4.1 Ce qui existe déjà

Alanya a aujourd'hui une seule forme d'organisation des contacts : la liste plate des **contacts préférés**, les favoris. Elle est implémentée de bout en bout — base de données, API, cache local hors ligne, écran dédié.

Ce qui n'existe pas : aucun moyen de *regrouper* ces favoris. Un utilisateur qui a quarante contacts préférés les voit dans une seule liste alphabétique, sans distinction entre sa famille, ses amis et ses collègues.

Un mot à ne pas confondre. Dans le code actuel, « groupe » désigne uniquement les **conversations de groupe** (`conversation.isGroup`). C'est autre chose. Une liste de contacts n'est pas une conversation : c'est un rangement personnel, invisible des autres.

4.2 Ce qu'on ajoute

Permettre à l'utilisateur de ranger ses contacts préférés dans des **listes nommées** — Famille, Amis, Bureau — pour deux usages :

1. **filtrer** l'écran des contacts préférés d'un seul geste ;
2. **créer une conversation de groupe** avec toute une liste, sans resélectionner les membres un par un.

4.3 Les règles produit

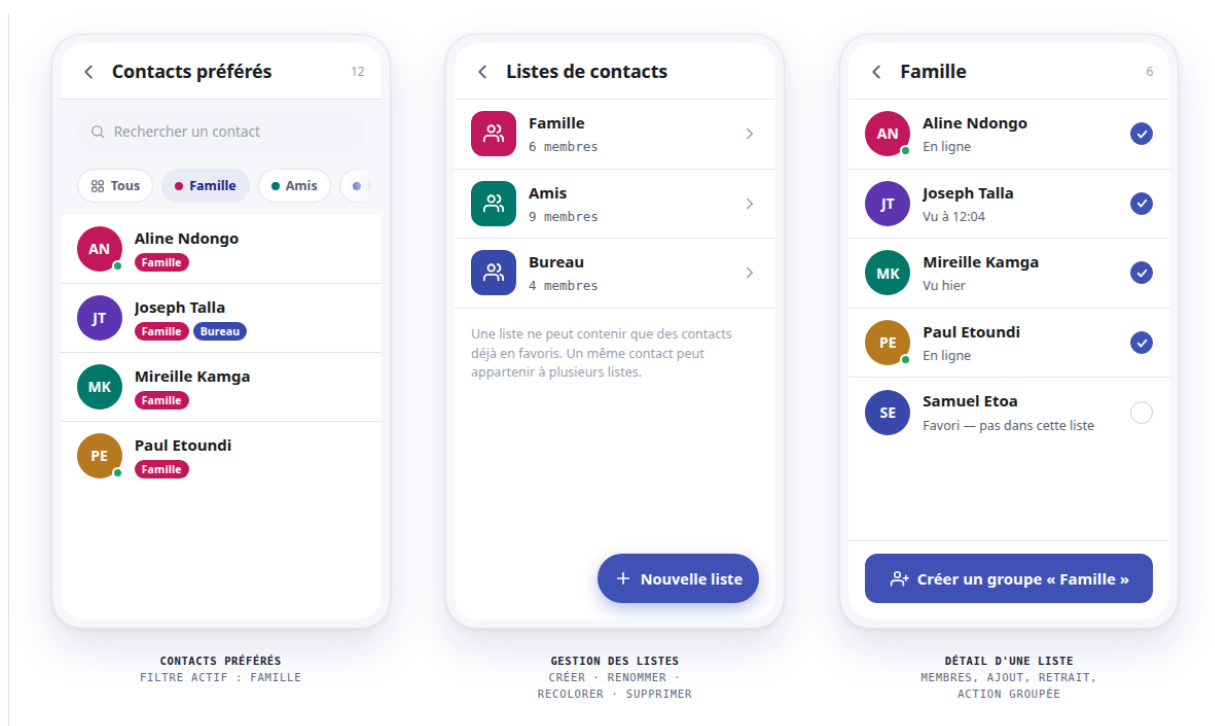
Qui peut être membre	Uniquement des contacts déjà en favoris . Une liste range ce qui est déjà rangé ; elle n'est pas un second carnet d'adresses.
Appartenance	Plusieurs à plusieurs . Un contact peut être à la fois en Famille et en Bureau.
Action groupée	Chat de groupe uniquement. Pas d'appel de groupe ni de diffusion de statut à ce stade — les tables les permettront plus tard sans modification.
Visibilité	Strictement privée . Personne ne sait dans quelle liste il est rangé, ni même qu'il l'est.
Emplacement	Puces de filtre dans l'écran des contacts préférés, <i>et</i> un écran dédié de gestion.
Navigation	Aucun sixième onglet . La barre est codée en dur à cinq onglets (<code>lib/screens/home/glass_nav_bar.dart</code>).

CHAPITRE 5

Les écrans

5.1 Vue d'ensemble

Trois écrans, un parcours : on filtre, on gère, on agit.



Maquette interactive. claude.ai/code/artifact/d15fb039

Source versionnée : docs/architecture/maquettes/listes-contacts.html

5.2 Écran 1 — Contacts préférés, filtrés

L'écran existant gagne une rangée de puces au-dessus de la liste :

- une puce « **Tous** », active par défaut ;
- une puce **par liste**, portant sa couleur ;
- une puce « **+ Gérer** » qui ouvre l'écran de gestion.

La rangée défile horizontalement — le nombre de listes n'est pas borné. Le filtre se combine avec la recherche existante : chercher « Ali » dans le filtre Famille cherche dans la famille seulement.

Chaque contact affiche les puces des listes auxquelles il appartient. C'est le seul endroit où l'appartenance multiple se voit d'un coup d'œil.

5.3 Écran 2 — Gestion des listes

Une liste par ligne : pastille de couleur, nom, nombre de membres. Créer, renommer, recolorer, supprimer. Un bouton flottant « Nouvelle liste ».

Supprimer une liste ne supprime aucun contact — seulement le rangement.

5.4 Écran 3 — Détail d'une liste

Les membres de la liste, plus les autres favoris présentés en grisé avec une case à cocher : ajouter et retirer se font au même endroit, sans écran intermédiaire.

En bas, un bouton pleine largeur : « **Créer un groupe « Famille »** ». Le nom de la liste est pré-rempli et reste modifiable. Un appui ouvre la nouvelle conversation de groupe.

Pourquoi la case à cocher plutôt qu'une feuille de sélection. Montrer les non-membres dans la même liste, en grisé, évite un aller-retour vers un écran de sélection. L'utilisateur voit d'un seul coup qui est dedans et qui pourrait y être.

5.5 Points d'entrée

1. Le profil, entrée « Listes de contacts » à côté de « Contacts préférés » (`lib/screens/profile/profile_screen.dart` méthode `_openPreferredContacts`).
2. La puce « + Gérer » de l'écran 1.

VOLET 3

Compte officiel et diffusion

Adresser un message ou un statut à tout ou partie des comptes

CHAPITRE 6

Ce que l'on ajoute

6.1 Le besoin

Alanya n'a aujourd'hui aucun moyen de s'adresser à ses utilisateurs. Annoncer une maintenance, une nouvelle version ou prévenir un commerçant que sa vérification expire suppose de passer par un canal extérieur — e-mail, réseaux sociaux — que la plupart des utilisateurs ne consultent pas.

On ajoute donc un **compte officiel** capable d'envoyer deux choses :

- des **messages**, qui apparaissent comme une conversation ordinaire ;
- des **statuts**, qui apparaissent dans le fil des statuts.

Dans les deux cas, à **tous** les utilisateurs ou à une **partie** d'entre eux.

6.2 Ce qui a déjà été décidé

Le socle de compte (étape 1) a posé les règles qui encadrent ce compte :

Création	Par un administrateur, jamais par une inscription ordinaire.
Apparence	Avatar = logo Alanya, badge officiel doré, distinct du badge de vérification.
Réponse	Impossible. Le compte diffuse, il ne converse pas.
Nombre	Le modèle en supporte plusieurs (support, actualités...) sans modification.

CHAPITRE 7

Cibler une diffusion

7.1 Un critère, pas un choix parmi trois

La spécification d'origine prévoyait trois cibles : tous, par pays, ou une liste de comptes choisis. C'est insuffisant — on veut aussi pouvoir s'adresser aux commerçants, aux comptes inscrits depuis plus de trois mois, à ceux vus récemment, et à d'autres critères qu'on ne connaît pas encore.

Le ciblage devient donc une **combinaison de conditions**, que l'administrateur assemble. Ajouter un critère demain ne demandera aucune modification de la base.

Pays	Un ou plusieurs pays.
Genre de compte	Personnel, business, officiel.
État de vérification	Vérifié, en attente, expiré. . .
Ancienneté	Inscrit avant ou après une date, ou depuis une durée.
Activité	Vu dans les N derniers jours.
Comptes choisis	Une liste explicite, pour un message de service.

7.2 Savoir qui recevra, avant d'envoyer

À chaque modification du ciblage, l'administrateur voit l'estimation se recalculer. C'est le garde-fou principal : on ne diffuse pas à trois cent mille personnes sans l'avoir vu écrit.

Nouvelle diffusion

Le contenu à gauche, le ciblage à droite. L'estimation se recalcule à chaque changement.

NATURE

Message

Statut (24 h)

CONTENU

Bonjour ! La version 2.4 arrive la semaine prochaine. Les commerçants vérifiés pourront enfin publier leur catalogue directement depuis leur fiche. ...

APERÇU CÔTÉ UTILISATEUR

Le message apparaît dans une conversation avec le compte Alanya, sans zone de saisie. Voir la maquette du socle de compte.

Diffuser maintenant

CIBLAGE

☒ Pays est Cameroun
users.idPays = 541 %

ET

☒ Genre de compte est business
users.account_type = 14 %

ET

☐ Inscrit depuis plus de 3 mois
users.created_at62 %

ET

☐ Vu dans les 30 derniers jours
users.last_seen47 %

+ Ajouter une condition

DESTINATAIRES ESTIMÉS

4 880

sur 312 480 comptes non bannis

Diffusion large : le fan-out des notifications sera étalé par lots par la file de jobs. La conversation, elle, n'est créée qu'à la première ouverture de chaque destinataire.

Maquette interactive. Les conditions s'activent et se désactivent, l'estimation et la requête produite suivent :

claude.ai/code/artifact/b609a75d

Source versionnée : [docs/architecture/maquettes/diffusion-admin.html](#)

7.3 L'historique

Chaque diffusion conserve son critère, et non sa liste de destinataires. Six mois plus tard, on peut relire à qui elle s'adressait — ce qu'une liste d'identifiants ne dirait pas.

L'historique affiche aussi le taux d'ouverture, dérivé du nombre de destinataires ayant réellement rouvert l'application depuis l'envoi.

La relance J-30 passe par ce canal. Les avertissements d'expiration de vérification conçus à l'étape 1 ne sont rien d'autre qu'une diffusion dont le critère est « business, vérifié, échéance dans moins de 30 jours ». Aucun mécanisme spécifique n'est construit pour eux.

CHAPITRE 8

Ce que voit l'utilisateur

8.1 Un message diffusé

Une conversation ordinaire avec le compte Alanya, à ceci près que la zone de saisie est remplacée par un bandeau : « Ce compte diffuse des annonces. Vous ne pouvez pas y répondre. » Les actions « répondre » et « réagir » n'apparaissent pas sur ses messages.

Tout le reste fonctionne normalement : notification, compteur de non-lus, conservation hors ligne, recherche. C'est voulu — voir chapitre 29.

8.2 Un statut diffusé

Il apparaît dans le fil des statuts, avec l'avatar Alanya et le badge officiel, exactement comme le statut d'un contact. Il expire au bout de 24 heures comme les autres.

Un obstacle que la spécification d'origine n'avait pas vu. Aujourd'hui, un statut n'est visible que si l'auteur et le lecteur sont **mutuellement** en contacts préférés. En l'état, un statut officiel serait invisible pour absolument tout le monde — le compte Alanya ne peut pas ajouter trois cent mille personnes en favoris, ni espérer qu'elles le fassent en retour. Le correctif tient en trois lignes et se trouve au chapitre 32.

VOLET 4

Comptes business

Une vitrine : lieu, médias, horaires, catalogue et annuaire de proximité

CHAPITRE 9

Ce qu'un commerçant peut présenter

9.1 La fiche

Un compte business dispose d'une fiche étendue, absente d'un compte personnel :

Identité	Raison sociale, description, logo, image de couverture.
Catégorie	Une catégorie principale, choisie dans une liste courte.
Mots-clés	Saisis librement par le commerçant, pour être trouvé.
Lieu	Adresse, position sur la carte, distance affichée au visiteur.
Horaires	Jours et créneaux d'ouverture, exceptions.
Liens	Site web, réseaux sociaux, dans l'ordre choisi.
Médias	Images et vidéos de présentation.
Moyens de paiement	Ce que l'établissement accepte sur place.
Catalogue	Produits ou services, avec prix et disponibilité.

Trois de ces rubriques méritent d'être détaillées, parce qu'elles ne sont pas de simples champs de formulaire.

9.2 Le lieu et la géolocalisation

C'est la rubrique dont dépend tout l'annuaire, et la seule qui parte réellement de zéro — Alanya ne stocke aujourd'hui aucune donnée géographique.

Adresse écrite	Ce que le visiteur lit et recopie. « Avenue Kennedy, Yaoundé ». Texte libre : il n'existe pas de référentiel d'adresses fiable sur le marché visé.
Position	Un point sur la carte, posé par le commerçant lui-même. C'est <i>elle</i> qui sert à la recherche par proximité, pas l'adresse écrite.
Distance affichée	Calculée à la volée entre le visiteur et le commerce. « à 1,2 km de vous ».

Comment le commerçant pose son point. Deux gestes seulement : « utiliser ma position actuelle » quand il est sur place, ou déplacer un repère sur une carte. Pas de géocodage d'adresse — il donnerait des résultats médiocres et masquerait les erreurs derrière une fausse précision. Le commerçant sait où est sa boutique ; on lui demande de le montrer.

Aucune dépendance à ajouter. `geolocator` et `flutter_map` sont **déjà** dans `pubspec.yaml` — ils servent au partage de position dans les conversations (`lib/screens/chats/location_picker_screen.dart`). Le sélecteur de position du commerçant peut réutiliser cet écran.

9.3 Médias de présentation

Images *et* vidéos. C'est ce qui fait la différence entre une fiche remplie et une fiche convaincante — et c'est aussi la rubrique la plus coûteuse à exploiter.

Couverture	Une image en tête de fiche, format paysage.
Galerie	Photos de l'établissement, des produits, de l'équipe. Ordre choisi par le commerçant.
Vidéos	Présentation de l'activité. Avec vignette, durée, et lecture au clic.
Logo	Réutilise l'avatar du compte — pas de champ supplémentaire.

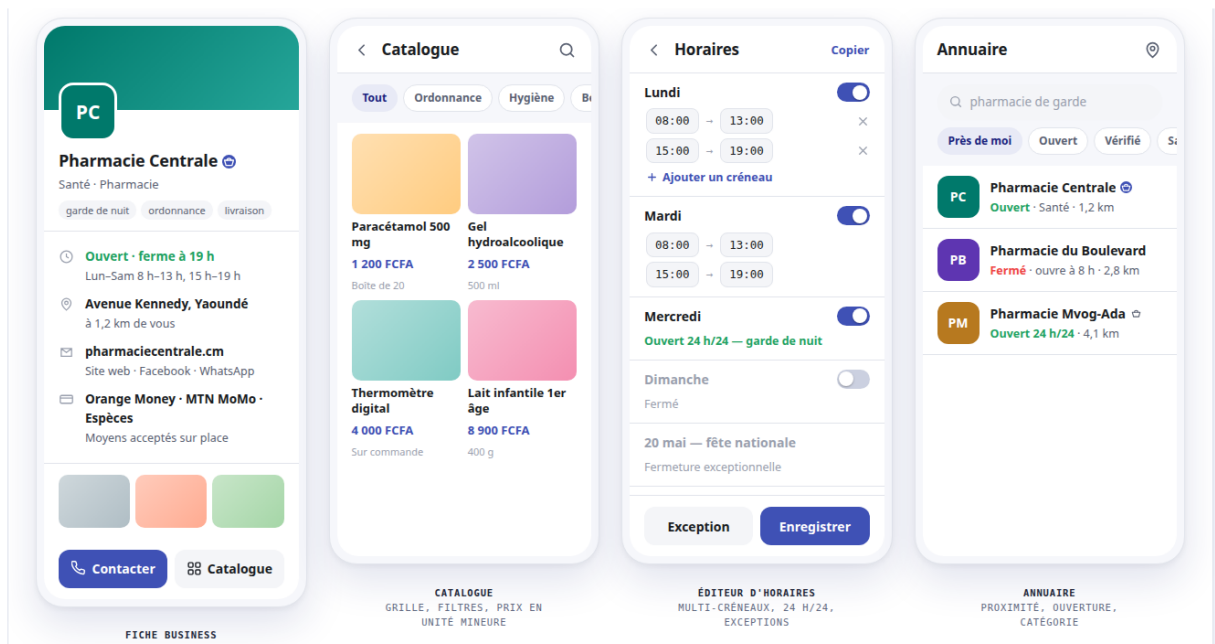
La vidéo est un chantier, pas un champ. Une vidéo téléversée depuis un téléphone pèse couramment plusieurs dizaines de mégaoctets. La servir telle quelle à chaque ouverture de fiche est intenable. Il faut une vignette, une limite de durée, et un transcodage — lequel suppose la file de jobs conçue à l'étape 3. Voir le chapitre 40.

9.4 Liens et présence en ligne

Site web, Facebook, Instagram, WhatsApp, TikTok... Le commerçant en ajoute autant qu'il veut et choisit leur ordre d'affichage.

Pas une colonne par réseau. Le point que le document d'amorce soulignait à juste titre. Une table `business_link` (`type`, `url`, `ordre`) permet d'ajouter le réseau à la mode de l'an prochain sans migration. Une colonne `facebook_url` ne le permettrait pas.

9.5 Les quatre écrans



Maquette interactive. claude.ai/code/artifact/37b8e6c7

Source versionnée : docs/architecture/maquettes/comptes-business.html

9.6 Le badge fait son travail

Dans l'annuaire, trois commerces se distinguent d'un coup d'œil : deux portent la pastille pleine  — vérifiés par Alanya — et un porte le panier nu , c'est-à-dire un commerce qui s'est déclaré tel sans qu'on l'ait contrôlé.

C'est exactement la distinction que le socle devait rendre lisible à 13 pixels, et c'est dans l'annuaire qu'elle compte le plus : un visiteur qui cherche une pharmacie de garde prend une décision sur cette base.

CHAPITRE 10

La catégorie et les mots-clés

10.1 Une liste courte, pas une arborescence

Le choix retenu : **une catégorie principale** dans une liste courte, plus des **mots-clés libres** saisis par le commerçant.

Pourquoi pas deux niveaux	Un sélecteur à deux étages est pénible sur un téléphone, et il faudrait arrêter dès aujourd'hui une liste longue qu'on ne pourra plus changer.
Pourquoi des mots-clés	Ils absorbent tout ce que la liste ne prévoit pas — « garde de nuit », « livraison », « sur mesure » — sans figer quoi que ce soit.

Le revers, à assumer. La qualité de la recherche dépendra de ce que les commerçants écrivent. Deux garde-fous tempèrent : les mots-clés sont *normalisés* (minuscules, accents repliés) et le champ propose les mots déjà utilisés par d'autres commerces de la même catégorie. Un mot-clé partagé vaut mieux que dix orthographes du même mot.

10.2 Une taxonomie est difficile à changer

C'est le point que le document d'amorce soulignait à juste titre : une fois des milliers de fiches rangées, renommer ou fusionner une catégorie devient une opération de reprise de données. D'où la liste courte — vingt à trente entrées — et la table de référence éditable plutôt que des valeurs figées dans le code.

CHAPITRE 11

Les horaires, l'écran le plus difficile

Ce n'est pas un champ de formulaire. Six cas doivent tenir sur un écran de téléphone, et cinq d'entre eux restent invisibles tant qu'on n'a pas essayé.

Multi-créneaux	Ouvert le matin, fermé à midi, rouvert l'après-midi. Deux plages pour un seul jour — le cas le plus courant, et celui qu'un champ « de... à... » ne sait pas exprimer.
24 h/24	N'est pas « 00 :00 → 00 :00 ». C'est un état distinct, sans quoi le calcul « ouvert maintenant » se trompe.
Passage de minuit	Un bar ouvert de 20 h à 2 h : l'heure de fin est <i>avant</i> l'heure de début.
Copie d'un jour	Saisir sept fois les mêmes horaires fait abandonner. « Copier vers » n'est pas un confort, c'est ce qui décide si la fiche sera remplie.
Exceptions	Jours fériés, congés, fermeture ponctuelle. Une liste à part, pas un champ de plus.
Fuseau horaire	« Ouvert maintenant » se calcule dans le fuseau du commerce, pas dans celui du visiteur.

Le fuseau existe déjà. La table `pays` porte `timeZone` et `decalageHoraire` (`migrations/001_initial_schema.sql`), et `users.idPays` les relie. Rien à créer — mais tout à ne pas oublier.

À budgéter comme un écran. Cet éditeur mérite sa propre estimation, au même titre qu'un écran de conversation. Le traiter comme un champ du formulaire de profil garantit qu'il sera bâclé, puis refait.

CHAPITRE 12

Le tableau de bord du commerçant

12.1 Ce qu'il montre

Puisque les transactions se font hors application, le tableau de bord ne peut pas parler d'argent. Il parle de ce qu'Alanya **observe réellement** : l'audience de la fiche.

- nombre de vues du profil, et son évolution ;
- provenance des visites — annuaire, recherche, conversation, lien partagé ;
- répartition géographique des visiteurs ;
- conversations entamées depuis la fiche.

Pourquoi pas de commandes déclarées. On aurait pu laisser le commerçant saisir ses ventes pour obtenir un « suivi des transactions ». L'expérience de ce genre de saisie manuelle est constante : elle est abandonnée après quelques semaines, et le tableau de bord devient faux — pire que vide. Écarté volontairement.

12.2 Ce que cela implique

Compter des vues suppose de les enregistrer. C'est un journal d'événements, avec le volume que cela suppose — voir le chapitre technique 8.

VOLET 5

Certification

Attester une identité, sans vendre de badge

CHAPITRE 13

Ce que la certification dit, et ne dit pas

13.1 Une seule question

Le badge répond à *une* question : « ce compte est-il bien qui il prétend être ? » Rien d'autre.

Il ne dit pas que le commerce est sérieux, que ses prix sont justes, ni qu'il livre à l'heure. C'est une attestation d'identité, pas une recommandation — et la communication doit le refléter, sous peine d'engager Alanya sur ce qu'elle ne vérifie pas.

Ce qu'on vérifie	Que l'entreprise existe, et que la personne qui tient le compte est habilitée à la représenter.
Ce qu'on ne vérifie pas	La qualité des produits, l'honnêteté commerciale, le respect des délais.

13.2 Qui peut être vérifié

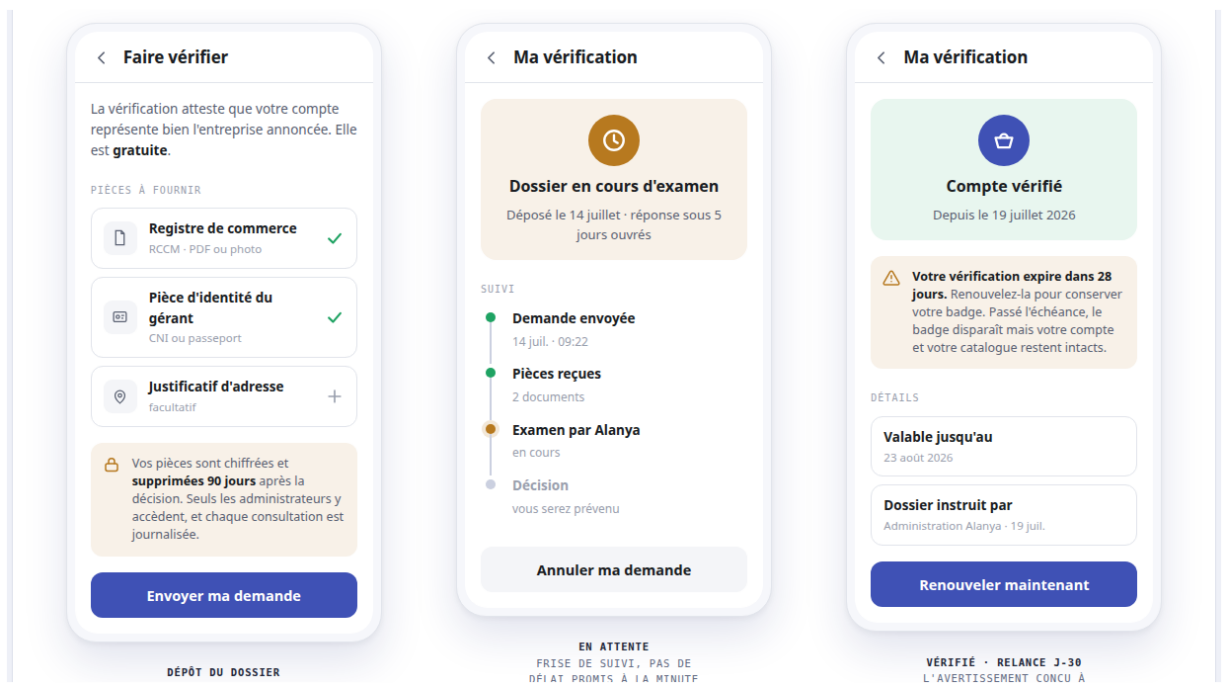
Les deux axes du socle étant indépendants, la vérification ne concerne pas que les commerces :

Genre	Rendu	Cas typique
Business		Une entreprise, une boutique, un cabinet
Personnel		Une personnalité publique, un journaliste

CHAPITRE 14

Le parcours du demandeur

14.1 Trois états



Maquette interactive. claude.ai/code/artifact/fb9bb169
Source versionnée : docs/architecture/maquettes/certification.html

14.2 Les pièces demandées

Proposition, à valider — elle est calquée sur le contexte camerounais et ouest-africain :

Pièce	Statut	Pourquoi
Registre de commerce (RCCM)	obligatoire	Prouve que l'entreprise existe
Pièce d'identité du gérant	obligatoire	Relie la personne à l'entreprise
Justificatif d'adresse	facultatif	Corrobore le lieu déclaré

Pour un compte *personnel* vérifié, la liste change : pièce d'identité, plus un élément de notoriété publique — page officielle, article de presse, site institutionnel.

Annoncer la rétention dès le dépôt. L'écran de dépôt dit, avant l'envoi, que les pièces sont chiffrées, supprimées 90 jours après la décision, et que chaque consultation est journalisée. Le dire après serait perçu comme une justification ; le dire avant est ce qui fait déposer.

14.3 Ce qui est promis, et ce qui ne l'est pas

« Réponse sous 5 jours ouvrés » plutôt qu'un compte à rebours. Un délai affiché à la minute transforme le sixième jour en manquement ; une fourchette laisse la marge que l'instruction demande réellement.

CHAPITRE 15

Le parcours de l'administrateur

15.1 L'écran d'instruction

Dossier #418 — Pharmacie Centrale

En attente

3 dossiers en file

COMPTE

Pharmacie Centrale

NUMÉRO ALANYA

0042 8817

GENRE DE COMPTE

Business — déclaré

RAISON SOCIALE DÉCLARÉE

PHARMACIE CENTRALE SARL

INSCRIT DEPUIS

11 mars 2026 · 4 mois

DEMANDE DÉPOSÉE

14 juillet 2026 · 09:22

PIÈCES JOINTES

Registre de commerce

RCCM · pdf · 1,4 Mo · chiffré

Ouvrir

Pièce d'identité du gérant

CNI · jpeg · 820 Ko · chiffré

Ouvrir

Suppression automatique le 17 octobre 2026 — 90 jours après la décision. Chaque ouverture est journalisée avec votre identifiant.

À gauche ce que le compte déclare, à droite ce qu'il apporte comme preuve. L'administrateur compare les deux ; c'est tout le travail.

15.2 Trois issues, pas deux

Approuver

Le badge apparaît, l'échéance est posée.

Refuser avec motif

Le motif est **obligatoire** et repart dans la notification. Sans lui, le demandeur redépose le même dossier incomplet et la file se remplit de doublons.

Demander une pièce

Le dossier *reste* en attente, le demandeur est prévenu de ce qui manque.

La troisième issue manquait. Ni `alanya-contexte-conception.md` ni `ideas/todo.tex` ne la mentionnaient. C'est pourtant le cas le plus fréquent en pratique : le dossier n'est pas mauvais, il est incomplet. Sans cette issue, l'administrateur doit refuser puis expliquer — ce qui est vécu comme un rejet.

15.3 Qui décide

Tout administrateur (`type_compte >= 1`). C'est cohérent avec l'existant : bannir et débannir sont déjà ouverts à tout administrateur (`src/routes/admin.js`). Chaque décision garde son auteur dans `reviewed_by` et sa date — la traçabilité remplace la restriction.

VOLET 6

Identité et connexion par QR code

Ajout de contact instantané par scan, et connexion d'un second appareil

CHAPITRE 16

Le besoin

16.1 Ce qui existe déjà

Alanya n’a aujourd’hui **qu’un seul chemin** pour ajouter un contact préféré : la recherche manuelle par nom, pseudo ou Alanya ID, depuis la feuille d’ajout du profil. Aucun lien d’invitation, aucun code, aucun geste plus rapide.

Vérifié dans le code. `lib/widgets/add_contact_sheet.dart` interroge d’abord le cache local (`filterUsersBySearch`), puis `GET /users/search` après 400 ms sans résultat. L’ajout appelle `POST /contacts/:userId` — la même route que celle que ce volet réutilisera pour l’ajout par QR.

Côté connexion, Alanya n’a **qu’un seul appareil implicite** : se connecter sur un second téléphone fonctionne déjà techniquement (rien ne l’empêche), mais rien ne le montre non plus. Il n’existe aucun écran « mes appareils », aucun moyen de savoir où son compte est ouvert, ni de fermer une session à distance.

Vérifié dans le code. `lib/screens/profile/account_security_screen.dart` ne contient que deux sections : *Email* et *Changer le mot de passe*. Rien sur les appareils.

16.2 Ce qu’on ajoute

Deux fonctionnalités, toutes deux fondées sur un QR code :

1. **Un code personnel éphémère.** Chaque utilisateur génère, en ouvrant « Mon code », un QR à son identité — logo Alanya au centre, traits arrondis, une vraie carte de visite plutôt qu’un carré noir et blanc. Le scanner d’un autre utilisateur reconnaît la personne et l’ajoute *instantanément* en contact préféré. Le code vit **dix minutes** et s’éteint au premier scan réussi ; l’écran en régénère un neuf tout seul, et le propriétaire est prévenu — et invité à ajouter l’autre **en retour** — chaque fois que son code est utilisé.
2. **La connexion d’un second appareil.** Un nouvel appareil affiche un code ; un appareil déjà connecté le scanne pour l’autoriser. L’utilisateur peut ensuite voir, depuis son compte, la liste de tous les appareils connectés — et en déconnecter un à distance.

Une seule brique, deux usages. Les deux fonctionnalités ne partagent pas qu’un mot dans leur nom. Elles partagent un **composant scanner**, un **générateur de QR stylé**, et une manière commune de résoudre ce qu’un code scanné signifie. C’est pour cette raison qu’elles sont conçues dans un seul volet — le même raisonnement qui a réuni compte officiel et diffusion dans l’étape 3.

16.3 Les règles produit

Qui peut scanner	Un utilisateur déjà connecté , dans les deux cas. Scanner n'est jamais un geste public ; c'est toujours un compte qui en reconnaît un autre.
Vie du code personnel	Dix minutes, une seule personne. Généré à la demande à l'ouverture de « Mon code », consommé au premier scan réussi. L'écran affiche un compte à rebours et régénère automatiquement — à l'expiration comme après consommation.
Ajout de contact	Instantané , sans écran de confirmation. Le filet de sécurité est une carte de résultat annulable, pas une étape supplémentaire.
Retour au propriétaire	Prévenu à chaque scan de son code, où qu'il soit dans l'application, et invité à ajouter l'autre en retour — notification si l'app est en arrière-plan, simple information si le lien existait déjà.
Contacts ajoutés par QR	Reconnaissables partout : pastille sur l'avatar, mention datée sur la fiche, filtre « Par QR » dans les contacts préférés <i>et</i> dans la liste des discussions.
Note de contexte	Optionnelle, saisie juste après le scan — « rencontré au salon de Douala » — ou au moment d'ajouter en retour, et relue sur la fiche du contact. Elle appartient à la relation : l'autre ne la voit jamais.
Codes permanents	Réservés aux futurs comptes business (volet 1). Aucun compte personnel n'a de code qui dure.
Connexion d'un appareil	Jamais instantanée. L'appareil qui scanne voit toujours l'identité de l'appareil qu'il autorise, et doit confirmer explicitement.
Durée du QR de connexion	Environ 90 secondes , puis régénération automatique — comme un code à usage unique, pas comme une carte de visite.
Emplacements	Profil (mon code), feuille d'ajout de contact (scanner), Compte et sécurité (appareils connectés), écran de connexion (nouvel appareil).

Pourquoi un code qui meurt. Un code permanent est une affiche ; un code de dix minutes est une poignée de main. Le code personnel sert à des rencontres — on le montre, on l'envoie, la personne en face scanne, c'est fini. L'usage unique garantit que le code n'ajoute que la personne à qui on vient de le tendre : une capture d'écran qui circule ensuite ne mène plus nulle part. Et comme l'écran régénère aussitôt, montrer son code à trois personnes d'affilée reste trois gestes de *leur* côté, zéro du sien.

Vérifié dans le code — pourquoi le permanent est verrouillé. L'infrastructure du code permanent (`users.qr_public_id`) existe et reste en place, mais son garde refuse aujourd'hui tout le monde — et pour une raison précise : la colonne `users.type_compte`, qu'on prendrait volontiers pour un type de compte métier, est en réalité un **rôle d'administration** (0 utilisateur, 1 admin, 2 super-admin, cf. la route `PUT /users/:id/role` de `src/routes/admin.js`, côté Alanya-Backend). Rien dans la base ne sait encore dire « ce compte est un commerce ». Le volet 1 (socle de compte) apportera `account_type` ; le verrou des codes permanents s'y branchera alors.

CHAPITRE 17

Mon code personnel

17.1 Vue d'ensemble

Un seul écran, deux volets accessibles d'un geste : *Mon code* et *Scanner* — comme on bascule entre appareil photo avant et arrière.

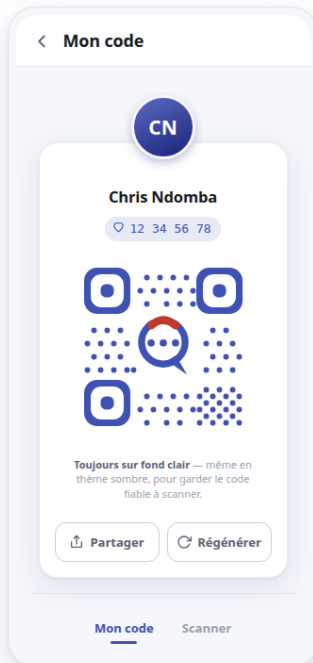
ALANYA · IDENTITÉ ET CONNEXION PAR QR CODE · ÉTAPE 6

Un code, une identité — pas un carré noir et blanc

Le code de chaque utilisateur porte le monogramme Alanya en son centre, des yeux et des modules arrondis. Le scanner reconnaît la personne et l'ajoute en contact préféré instantanément — sans écran de confirmation intermédiaire.

Les deux écrans

Un seul écran, deux volets accessibles par un geste : « Mon code » et « Scanner ».



MON CODE
MÉDAILLON, LOGO AU CENTRE,
PARTAGE ET RÉGÉNÉRATION



SCANNER
RECONNAISSANCE INSTANTANÉE,
SUCCÈS DOUX ET ANNULABLE

Ajout instantané, sans écran de confirmation. C'est la règle produit demandée — scanner suffit. Le filet de sécurité est le bandeau « Annuler » (5 s), pas un écran intermédiaire.

Le code reste lisible en toute condition. Correction d'erreur niveau H (30 % de redondance) pour tolérer le logo central sans nuire à la reconnaissance.

Régénérer invalide l'ancien code. Utile si un code a été capturé par erreur ou partagé publiquement — il ne mène alors plus à personne.

Tokens : lib/core/theme/app_colors.dart qr_flutter - eyeStyle/dataModuleStyle personnalisés, correction H mobile_scanner - scan plein écran
Maquette de conception - aucun code applicatif

Maquette interactive. claude.ai/code/artifact/69e9cbec

Source versionnée : docs/architecture/maquettes/qr-identite.html

17.2 Écran — Mon code

Une carte blanche : médaillon d'avatar débordant le haut, nom, Alanya ID, puis le QR code lui-même — fond clair, points arrondis à la couleur de marque, logo Alanya au centre. Sous le

code, un **compte à rebours** (« Expire dans 9 :41 ») et la mention « Valable 10 minutes et pour une seule personne ». En dessous, deux actions : **Partager** et **Nouveau code**.

À zéro — comme après un scan réussi — l'écran régénère un code neuf de lui-même : l'utilisateur n'a jamais un code mort sous les yeux, exactement le comportement déjà retenu pour le QR de connexion (chapitre 18). « Nouveau code » reste disponible pour qui veut couper court sans attendre ; aucun dialogue de confirmation, renouveler un code qui expire tout seul n'a rien d'irréversible.

Pourquoi le code reste toujours clair. Même en thème sombre, la carte du QR reste sur fond blanc. Un code sur fond sombre scanne moins bien — la fiabilité du scan prime sur la cohérence du thème, à cet endroit précisément.

17.3 Être prévenu, ajouter en retour

Quand quelqu'un utilise votre code — scan ou lien —, vous êtes prévenu immédiatement : un dialogue propose d'ajouter la personne **en retour** (« untel vous a ajouté à ses contacts préférés avec votre code QR. L'ajouter en retour ? », *Ajouter / Non merci*). Si l'application est en arrière-plan ou fermée, une notification prend le relais. Et si vous aviez déjà cette personne en contact, pas de question : une simple information.

Le dialogue n'habite aucun écran. Le code a pu être partagé par lien : à l'instant où il est utilisé, son propriétaire peut être n'importe où dans l'application — pas forcément sur l'écran « Mon code ». Le dialogue est donc global, affiché où que l'on soit.

L'écran « Mon code », lui, réagit à sa manière : le code venant d'être consommé, il en affiche un neuf aussitôt — le propriétaire peut enchaîner plusieurs personnes sans un geste.

17.4 Reconnaître un contact ajouté par QR

Un contact ajouté par QR reste reconnaissable après coup, partout où il apparaît :

1. une **pastille QR** au coin bas-gauche de l'avatar — le bas-droit reste réservé au point de présence en ligne, les deux cohabitent — dans **toutes** les listes : contacts préférés, feuille d'ajout, nouvelle discussion, sélection de membres, partage de contact... et jusqu'à la liste des discussions ;
2. une **mention datée** sur la fiche du contact : « Ajouté par QR code · 30 juil. 2026 » ;
3. un **filtre** « Tous / Par QR » sur l'écran des contacts préférés, chaque option affichant son compte.

L'ajout *en retour* est marqué de la même façon : les deux directions du lien portent la même origine.

Maquette interactive. claude.ai/code/artifact/0e095614
Source versionnée : [docs/architecture/maquettes/qr-distinction.html](#)

La liste des discussions porte le même filtre « Par QR » que l'écran des contacts préférés : un chip parmi les filtres existants (Tous, Discussions, Groupes...), qui ne retient que les conversations avec un contact rencontré par QR.

Enfin, la carte de résultat du scan propose un champ facultatif : une **note de contexte**, quelques mots pour se souvenir de la rencontre. Elle se retrouve sur la fiche du contact, en italique sous la mention d'origine — et nulle part ailleurs : c'est un aide-mémoire personnel, jamais montré à l'intéressé.

17.5 Partager son code

Partager ouvre un choix explicite : le **lien** (cliquable, accompagné de l'Alanya ID) ou l'**image** (la carte telle qu'affichée, compte à rebours compris). Dans les deux cas la légende annonce la couleur — « Ce code est valable 10 minutes et pour une seule personne » — suivie de l'Alanya ID : le code meurt en dix minutes, l'Alanya ID reste le chemin permanent, les deux voyagent ensemble.

Pas d'enregistrement dans la galerie. Enregistrer la carte en image n'est volontairement pas proposé : une image morte au bout de dix minutes finirait en code cassé au fond d'une pellicule. L'option reviendra avec les codes permanents des comptes business — eux supportent l'impression et l'affichage.

17.6 Écran — Scanner

Plein écran, viseur au centre, façon appareil photo. Dès qu'un code d'identité est reconnu, une **carte de résultat** se pose en bas de l'écran : la personne ajoutée (avatar, nom), « Ajouté à vos contacts », et trois suites possibles — **Message**, **Voir détails**, **Annuler**. La caméra reste vivante derrière la carte : on peut enchaîner plusieurs personnes sans ressortir de l'écran. Pas d'écran intermédiaire, pas de confirmation à taper.

Trois cas particuliers, traités comme des succès doux plutôt que des erreurs : scanner son propre code (rien ne se passe, un message discret l'indique — et le code *survit*), scanner un contact déjà favori (la carte s'affiche quand même, sans bouton Annuler : rien n'a été écrit), scanner un code expiré, déjà utilisé ou illisible (message clair, la caméra reste active).

Un bouton **Importer une image** couvre enfin le code reçu en capture d'écran ou en photo : l'image choisie dans la galerie est analysée exactement comme le cadre de la caméra — même résultat, même carte.

17.7 Points d'entrée

1. Le profil, une icône QR à côté de l'Alanya ID (`lib/screens/profile/profile_screen.dart`, en-tête).
2. La feuille d'ajout de contact, un bouton « Scanner un code » au-dessus de la recherche existante (`lib/widgets/add_contact_sheet.dart`).

CHAPITRE 18

Connexion d'un second appareil

18.1 Le principe

Contrairement à l'ajout de contact, une connexion ne s'approuve **jamais** au simple scan. Le nouvel appareil affiche un code ; c'est l'appareil *déjà* connecté qui le scanne et confirme — jamais l'inverse.

Pourquoi une confirmation, jamais une reconnaissance instantanée. Si le scan suffisait à connecter, un attaquant pourrait afficher le QR de connexion d'un appareil qu'il contrôle — sur un écran, un site, une affiche — et attendre qu'une victime le scanne par réflexe avec son propre compte déjà connecté. C'est une attaque documentée contre les connexions par QR (le « quishing »). L'écran de confirmation, qui montre *quel* appareil demande la connexion avant de l'accepter, est le seul rempart : il doit toujours s'afficher, sans exception ni raccourci.

18.2 Écran — Se connecter par QR

Sur le **nouvel** appareil, non encore connecté : un code, un compte à rebours, un statut (« En attente de scan... »). Le code se régénère seul à l'expiration — l'utilisateur n'a jamais un code mort sous les yeux.

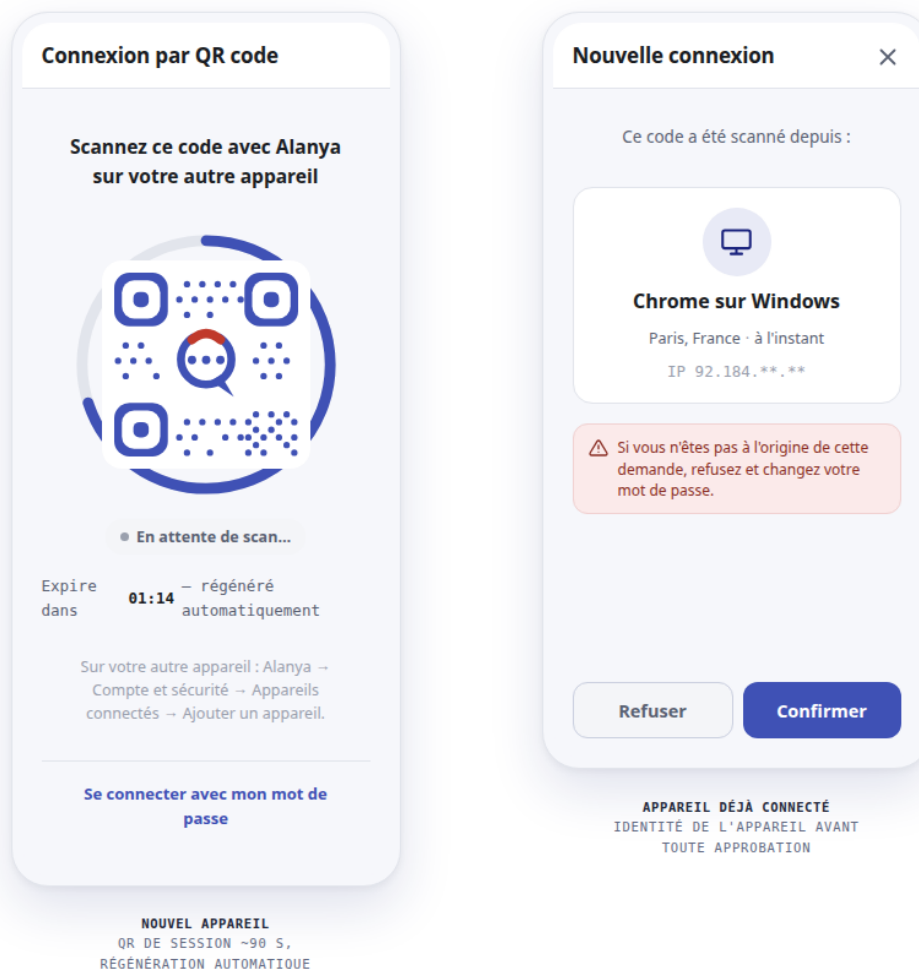
ALANYA · IDENTITÉ ET CONNEXION PAR QR CODE · ÉTAPE 6

Connecter un second appareil, jamais en un seul geste

Contrairement à l'ajout de contact, une connexion ne s'approuve jamais au simple scan. L'appareil déjà connecté doit voir qui il autorise et confirmer explicitement — le même principe que la liste des appareils, consultable à tout moment depuis le compte.

La poignée de main

Le nouvel appareil affiche, l'appareil de confiance scanne et confirme — deux écrans, un seul geste refusable.



Jamais d'approbation automatique au scan. L'écran de confirmation est obligatoire — sans lui, un QR de connexion affiché sur un site tiers et scanné par erreur suffirait à connecter un attaquant (« quishing »).

Le repli FCM ne fonctionne pas pour le nouvel appareil — il n'a pas encore d'alanyaID à cibler. La v1 mobile n'utilise que le polling REST (~2 s) ; le canal socket reste prêt côté serveur pour un futur onglet web, qui peut rester ouvert et écouter en continu.

Maquette interactive. claude.ai/code/artifact/a69b2050

Source versionnée : docs/architecture/maquettes/qr-connexion.html

18.3 Écran — Confirmer la connexion

Sur l'appareil **déjà** connecté, après le scan : une fiche décrivant ce qui vient d'être scanné — type d'appareil, ville approximative, à l'instant — suivie de deux boutons, **Refuser** et **Confirmer**. Un avertissement rappelle de refuser et de changer son mot de passe en cas de doute.

Pourquoi l'adresse compte plus que le nom de l'appareil. Le nom et la plateforme sont *annoncés* par l'appareil qui demande la connexion : rien n'empêche un attaquant d'appeler le sien « Alanya — Vérification de sécurité ». La seule information que le serveur *constate* lui-même est l'adresse d'où part la demande. C'est donc la seule sur laquelle fonder sa décision, et l'écran le dit explicitement.

Une ville, pas une adresse technique. Cette adresse est traduite en lieu approximatif — « Douala, Cameroun » — avant d'être montrée. Sous sa forme brute (41.202.219.14), elle n'apprendrait rien à qui n'est pas technicien : on afficherait la seule information fiable de l'écran sous la seule forme où elle est inutilisable. Sous forme de ville, le doute devient une évidence — on sait immédiatement si la demande part d'où l'on se trouve. Si la traduction n'aboutit pas, l'adresse brute reste affichée plutôt que rien.

18.4 Écran — Mes appareils connectés

Un écran à part, indépendant de toute connexion en cours, atteint depuis *Compte et sécurité* : tous les appareils sur lesquels le compte est ouvert — pas seulement ceux ajoutés par QR — avec un repère « Cet appareil » et un bouton **Déconnecter** par ligne, à effet immédiat.

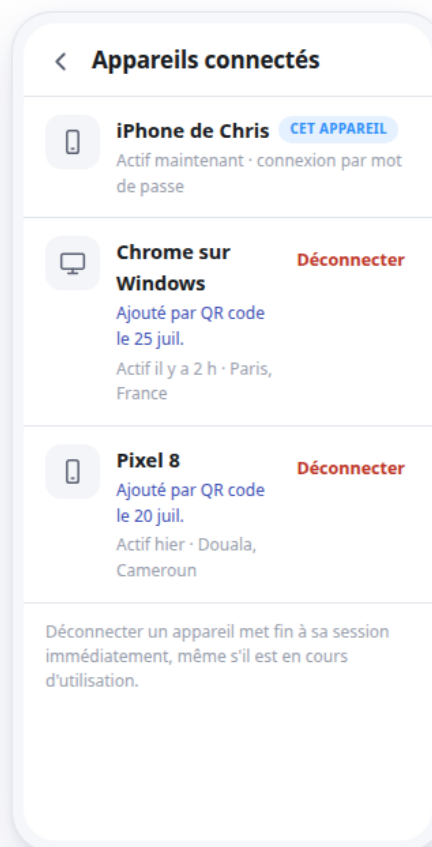
ALANYA · IDENTITÉ ET CONNEXION PAR QR CODE · ÉTAPE 6

Connecter un second appareil, jamais en un seul geste

Contrairement à l'ajout de contact, une connexion ne s'approuve jamais au simple scan. L'appareil déjà connecté doit voir qui il autorise et confirmer explicitement — le même principe que la liste des appareils, consultable à tout moment depuis le compte.

Mes appareils, à tout moment

Un écran, atteint depuis Compte et sécurité — indépendant de toute connexion QR en cours.



MES APPAREILS
TOUS LES LOGINS, PAS
SEULEMENT CEUX PAR QR

Un identifiant matériel, pas un UUID d'app. `device_id` vient de `device_info_plus` (déjà une dépendance) pour qu'un même téléphone reste le même appareil après réinstallation — pas de doublon fantôme dans la liste.

Tous les logins, pas seulement ceux par QR. Une connexion par mot de passe alimente aussi cette table — sinon l'écran resterait vide pour la quasi-totalité des comptes.

Maquette interactive. claude.ai/code/artifact/a69b2050 — second panneau de la même maquette
Source versionnée : docs/architecture/maquettes/qr-connexion.html

18.5 Points d'entrée

1. L'écran de connexion, un bouton « Se connecter avec un code QR » (`screens/authentication/login_screen.dart`), pour l'appareil qui n'est pas encore connecté.
2. *Compte et sécurité*, une nouvelle section « Appareils connectés » (`screens/profile/account_security_screen.dart`), pour lier un nouvel appareil ou consulter la liste depuis un appareil déjà connecté.

PARTIE II

Approche technique

VOLET 1

Socle de compte

Extension de users, dérivation du badge, protection du nom

CHAPITRE 19

Modèle de données

19.1 Pourquoi étendre `users` plutôt que créer une table `accounts`

La conception initiale prévoyait une nouvelle table `accounts` servant de socle. La lecture du schéma réel invalide cette approche.

Constat vérifié dans le code. 14 tables référencent `users(alanyaID)` par clé étrangère : `preferredContact`, `blocked`, `userAccess`, `conversation`, `conv_participants`, `message`, `statut`, `statut_views`, `meeting`, `participant`, `callHistory`, `reserved_alanya_phone`, `user_push_devices` et `user_notification_prefs` — soit 34 contraintes au total (sources : `migrations/001_initial_schema.sql`, 010, 020, 021).

Introduire une table parallèle imposerait de réécrire ces 34 contraintes. On étend donc `users`, qui est le socle de compte.

Ce choix a un second bénéfice, qui n'est pas un hasard : la conception exige que le genre et l'état de vérification soient lisibles dans la requête de liste de conversations **sans jointure supplémentaire**. Cette requête joint déjà `users` pour récupérer le nom et l'avatar du correspondant (`src/controllers/conversationController.js`, ligne 24). Deux colonnes de plus sur `users` sont donc gratuites ; une table `accounts` aurait imposé une jointure de plus sur le chemin le plus chaud de l'application.

19.2 La migration

```
1  -- 023_account_socle.sql -- socle de compte unifie
2  --
3  -- Deux axes orthogonaux portés par 'users' :
4  --   account_type           : ce que le compte EST
5  --   verification_status    : ce qu'Alanya ATTESTE a son sujet
6  --
7  -- TINYINT et non ENUM : ajouter une valeur a un ENUM impose un ALTER
   TABLE
8  -- sur une table qui grossit. L'enumeration vit cote Node.
9
10 ALTER TABLE users
11   ADD COLUMN account_type TINYINT NOT NULL DEFAULT 0
12   COMMENT '0=personnel 1=business 2=officiel',
13   ADD COLUMN verification_status TINYINT NOT NULL DEFAULT 0
14   COMMENT '0=aucun 1=en_attente 2=verifie 3=refuse 4=revoque
15           5=expire',
16   ADD COLUMN verified_until DATETIME NULL
17   COMMENT 'Echeance de la verification (NULL si non verifie)',
18   ADD COLUMN verification_reminder_sent TINYINT NOT NULL DEFAULT 0
19   COMMENT '1 = relance J-30 deja envoyee pour l echeance courante';
20
   -- Annuaire business a venir : filtrer par genre puis par etat.
```

```

21 ALTER TABLE users
22   ADD INDEX idx_users_account_type (account_type,
23                                     verification_status);
24
25 -- Balayage du scheduler d'expiration : ne remonter que les comptes
26 -- ayant une echeance, sans parcourir toute la table.
27 ALTER TABLE users
28   ADD INDEX idx_users_verified_until (verified_until);

```

Listing 19.1 – migrations/023_account_socle.sql

Aucune colonne badge_type. Le badge n'est jamais stocké. Il se calcule à l'affichage à partir des deux axes. Une valeur stockée finirait désynchronisée le jour où une certification expire ou est révoquée — ce qui est précisément le cas que le système doit gérer.

19.3 Les énumérations, côté Node

```

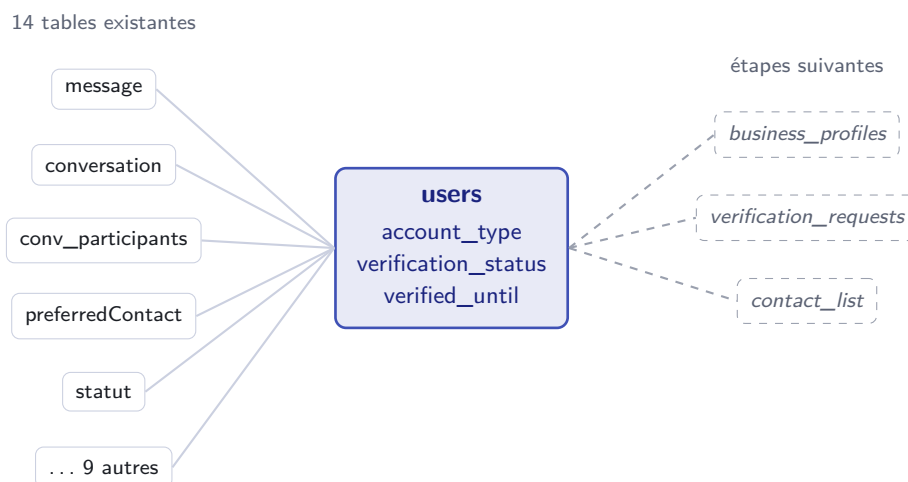
1  const ACCOUNT_TYPE = { PERSONNEL: 0, BUSINESS: 1, OFFICIEL: 2 };
2
3  const VERIFICATION = {
4    AUCUN:      0,
5    EN_ATTENTE: 1,
6    VERIFIE:    2,
7    REFUSE:     3,
8    REVOQUE:    4,
9    EXPIRE:     5,
10 };
11
12 module.exports = { ACCOUNT_TYPE, VERIFICATION };

```

Listing 19.2 – src/constants/accountTypes.js (nouveau)

Le dossier `src/constants/` existe déjà (`participantLimits.js`) : la convention est respectée.

19.4 Raccordement au schéma existant



CHAPITRE 20

Dérivation du badge

20.1 Une fonction pure, trois implémentations identiques

Le badge est une fonction de deux entiers vers une variante d’affichage. Aucun accès base, aucune requête réseau.

```
1 enum AccountBadge { aucun, cocheVerifiee, panierDeclare,
2   panierVerifie, officiel }
3
4 AccountBadge resolveAccountBadge(int accountType, int
5   verificationStatus) {
6   final verifie = verificationStatus == 2; // VERIFIE
7   switch (accountType) {
8     case 2: return AccountBadge.official;
9     case 1: return verifie ? AccountBadge.panierVerifie :
10       AccountBadge.panierDeclare;
11     default: return verifie ? AccountBadge.cocheVerifiee :
12       AccountBadge.aucun;
13   }
14 }
```

Listing 20.1 – lib/widgets/common/account_badge.dart (nouveau) — forme cible

account_type	verification_status	Variante	Rendu
0	$\neq 2$	aucun	—
0	$= 2$	cocheVerifiee	✓
1	$\neq 2$	panierDeclare	🛒
1	$= 2$	panierVerifie	⚙️
2	indifférent	officiel	🏆

Un seul widget, pas trois. Le composant doit être unique et utilisé aux quatre endroits listés en partie I. Trois implémentations concurrentes divergeront en deux semaines — c’est exactement ce qui est arrivé aux quatre indigos concurrents que le design system a dû unifier (lib/core/theme/app_colors.dart).

20.2 Le piège à corriger avant tout

Vérifié dans le code. Le champ `typeCompte` du modèle `User` est **codé en dur à 0** partout où un `User` est reconstruit depuis une charge utile partielle : `lib/core/services/call_service.dart` ligne 207, `lib/widgets/chat/contact_message_preview.dart` ligne 46, et `lib/screens/chats/contact_detail_screen.dart` ligne 104.

Les nouveaux champs suivront le même chemin et subiront le même sort : un badge dérivé afficherait « compte personnel non vérifié » à ces trois endroits, quel que soit le compte réel. Ces trois sites doivent être corrigés dans le même lot que l'ajout du badge, sinon le défaut est invisible en développement et visible en production.

20.3 Composant existant à ne pas confondre

`lib/widgets/common/app_badge.dart` existe déjà mais n'a aucun rapport : c'est `CountBadge`, la pastille de comptage des messages non lus. Le nouveau composant est donc un fichier distinct, `account_badge.dart`.

CHAPITRE 21

Règles appliquées par le serveur

21.1 Exclusion business / administrateur

Règle : `account_type != 0 ⇒ type_compte = 0`.

Pourquoi deux colonnes distinctes. `users.type_compte` existe déjà et porte le rôle d'administration (0 utilisateur, ≥ 1 admin, ≥ 2 super-admin). Il est lu par `src/middleware/adminAuth.js` (lignes 41 et 60), `lib/providers/admin_provider.dart` (lignes 245–246) et `talky-admin/lib/auth.ts`. Réutiliser cette colonne pour le genre de compte aurait cassé l'administration existante ; d'où la colonne séparée `account_type`.

Le contrôle est à poser aux deux endroits qui écrivent ces champs, dans `src/controllers/admin/users.js` : `setAccountType` (rôle admin, existant) et le nouveau point d'entrée qui fixera `account_type`. Rejet en 409 avec un message explicite plutôt qu'un écrasement silencieux.

21.2 Validation des noms d'affichage

Le filtrage doit vivre **côté serveur**, et non dans l'application mobile : le mobile n'est pas le seul écrivain. L'administration crée aussi des comptes (`src/controllers/admin/userCreate.js`), et la modification de profil passe par l'API.

```
1 // Blocs Unicode à refuser dans 'nom' et 'pseudo' :
2 //   U+2705, U+2713-U+2714 (coches)           U+2B50, U+2606, U+2B51
   //   (etoiles)
3 //   U+1F396-U+1F397, U+1F3C5 (medailles)   U+269C (fleur de lys /
   //   sceau)
4 //   Variation selectors utilisées pour composer un faux badge.
5 //
6 // Plus : refus des variantes normalisées du nom "Alanya", en tenant
   //   compte
7 //   de la casse, des accents, des espaces insérées, et des homoglyphes
8 //   cyrilliques U+0410 U+0430 (a), U+0435 (e), U+043E (o), U+0443
   //   (y),
9 //   U+0441 (c) -- visuellement identiques, distincts pour la base.
```

Listing 21.1 – `src/utils/displayNameGuard.js` (nouveau) — principe

Les homoglyphes. Refuser la chaîne « Alanya » ne suffit pas. Le même mot dont le A initial est un A cyrillique (U+0410) au lieu du A latin (U+0041) est une chaîne *différente* pour la base de données, et rigoureusement identique à l'œil. La comparaison doit donc se faire sur une forme normalisée, après repli des homoglyphes.

21.3 Interdire la réponse au compte officiel

Un seul point d’ancrage suffit. Les deux chemins d’envoi de message — REST (src/controllers/messageController.js ligne 178) et Socket.IO (src/socket/handlers/chat/messageSend.js ligne 142) — appellent tous deux la même fonction `evaluateDirectMessageSend` de `src/utils/blockUtils.js`. Une seule modification couvre donc les deux.

Cette fonction résout déjà le correspondant d’une conversation 1-1 et renvoie une action (`deliver`, `reject` ou `silent`). Il suffit d’y ajouter un cas :

```

1  const evaluateDirectMessageSend = async (conversationID, senderID) =>
2    {
3      const peerId = await getDirectConversationPeer(conversationID,
4        senderID);
5      if (peerId == null) return { isDirect: false };
6
7      // Nouveau : le compte officiel diffuse, il ne converse pas.
8      const [rows] = await pool.execute(
9        'SELECT account_type FROM users WHERE alanyaID = ?', [peerId],
10      );
11      if (rows[0] && rows[0].account_type === ACCOUNT_TYPE.OFFICIEL) {
12        return { isDirect: true, peerId, action: 'reject', code:
13          'OFFICIAL_READONLY' };
14      }
15
16      // ... suite inchangée : évaluation du blocage
17    };

```

Listing 21.2 – `src/utils/blockUtils.js` — ajout dans `evaluateDirectMessageSend`

Côté mobile, la zone de saisie de `lib/screens/chats/chat/chat_input.dart` est remplacée par un bandeau, et les actions « répondre » / « réagir » sont retirées des bulles.

À trancher à l’étape 3. Faut-il une conversation en lecture seule, ou un canal de diffusion distinct des conversations ? Le choix dépend de la stratégie de diffusion, qui est le sujet de l’étape sur le compte officiel. La règle serveur ci-dessus vaut dans les deux cas.

21.4 Expiration : réutiliser le mécanisme existant

Le comportement demandé — relance à J-30 puis suspension — a déjà son patron exact dans le code.

Ordonnanceur	src/services/meetingScheduler.js : un setInterval d'une minute, une requête qui remonte les échéances proches, et une colonne reminder_sent pour ne jamais notifier deux fois. Le squelette se transpose tel quel en verificationScheduler.js.
E-mail	src/services/mailService.js : sendMail et renderHtmlEmail sur le gabarit src/templates/email-template.html, déjà en service pour l'OTP de réinitialisation.
Notification	src/services/notificationService.js, déjà utilisé par l'ordonnanceur de réunions.
Avertissement in-app	Dérivé de verified_until, déjà présent dans la charge utile utilisateur — aucun appel supplémentaire.

```

1  -- 1. Relance J-30, une seule fois par echeance
2  SELECT alanyaID, nom, email, verified_until
3  FROM users
4  WHERE verification_status = 2
5         AND verified_until IS NOT NULL
6         AND verified_until > UTC_TIMESTAMP()
7         AND verified_until <= DATE_ADD(UTC_TIMESTAMP(), INTERVAL 30 DAY)
8         AND verification_reminder_sent = 0;
9
10 -- 2. Suspension a l'echeance
11 UPDATE users
12 SET verification_status = 5,                -- EXPIRE
13     verification_reminder_sent = 0          -- rearme pour un futur
14     renouvellement
15 WHERE verification_status = 2
16     AND verified_until IS NOT NULL
17     AND verified_until <= UTC_TIMESTAMP();

```

Listing 21.3 – Les deux requêtes du scheduler

Pourquoi pas de file de jobs ici. La conception initiale demandait une file de jobs (BullMQ / Redis) dès le départ. Elle n'existe pas aujourd'hui dans le backend. Pour les relances de vérification, le volume est faible et espacé : un ordonnanceur en processus suffit, et il a déjà fait ses preuves sur les rappels de réunion. La file redeviendra nécessaire au fan-out des diffusions du compte officiel — c'est là qu'il faudra la traiter, pas ici.

CHAPITRE 22

Impact fichier par fichier

22.1 Backend — Alanya-Backend

Fichier	Nature	Objet
migrations/023_account_socle.sql	nouveau	Les quatre colonnes et deux index
src/constants/accountTypes.js	nouveau	Énumérations partagées
src/utils/displayNameGuard.js	nouveau	Filtrage des noms, réservation d'« Alanya »
src/services/verificationScheduler.js	nouveau	Relance J-30 et suspension
src/utils/blockUtils.js	modifié	Refus de réponse au compte officiel
src/controllers/userController.js	modifié	Exposer les deux axes ; appliquer le filtrage des noms
src/controllers/conversationController.js	modifié	Ajouter les deux axes à la projection du correspondant
src/controllers/preferredContactController.js	modifié	Idem sur la liste de contacts
src/controllers/messageController.js	modifié	Idem sur l'expéditeur d'un message
src/controllers/admin/users.js	modifié	Piloter <code>account_type</code> ; exclusion business/admin ; filtre par genre
src/controllers/admin/userCreate.js	modifié	Création du compte officiel ; filtrage des noms
server.js	modifié	Démarrage du <code>verificationScheduler</code>

22.2 Mobile — Talky (Flutter)

Fichier	Nature	Objet
lib/widgets/common/account_badge.dart	nouveau	Le composant unique et sa dérivation
lib/talky_models.dart	modifié	accountType, verificationStatus, verifiedUntil sur User
lib/core/db/app_database.dart	modifié	Colonnes sur LocalUsers, schemaVersion et onUpgrade
lib/core/services/local_cache_repository.dart	modifié	Propager les champs dans le cache
lib/core/services/call_service.dart	corrigé	typeCompte: 0 codé en dur, ligne 207
lib/widgets/chat/contact_message_preview.dart	corrigé	Idem, ligne 46
lib/screens/chats/contact_detail_screen.dart	corrigé	Idem, ligne 104
lib/screens/chats/chats_screen.dart	modifié	Badge dans la liste de conversations
lib/screens/chats/chat_detail_screen.dart	modifié	Badge dans l'en-tête; masquage de la saisie
lib/screens/chats/chat/chat_input.dart	modifié	Bandeau « diffusion » à la place du champ
lib/screens/chats/chat/chat_bubbles.dart	modifié	Retrait de « répondre » et « réagir »
lib/l10n/*	modifié	Libellés des badges, du bandeau et des avertissements

Régénération obligatoire. Toute modification de `app_database.dart` impose `dart run build_runner build --delete-conflicting-outputs`, et la montée de `schemaVersion` doit passer par `onUpgrade` — pas par une recréation, sous peine de vider le cache local des utilisateurs déjà installés.

22.3 Administration — Alanya Admin

Fichier	Nature	Objet
types/index.ts	modifié	accountType, verificationStatus, verifiedUntil
app/(dashboard)/users/page.tsx	modifié	Colonne badge et filtre par genre de compte
app/(dashboard)/users/[id]/page.tsx	modifié	Pilotage du genre et de l'état
app/(dashboard)/users/new/page.tsx	modifié	Création du compte officiel
components/ui/badge.tsx	modifié	Variante de badge de compte

Conversion des noms. Le client HTTP de l'administration convertit automatiquement les clés `snake_case` en `camelCase` (`lib/api.ts`). Les colonnes `account_type` et `verification_status` arriveront donc en `accountType` et `verificationStatus` sans travail supplémentaire.

CHAPITRE 23

Points de vigilance et questions ouvertes

23.1 À vérifier avant d'appliquer la migration

Le backend n'a pas d'ordonnanceur de migrations. Il n'existe ni exécuter, ni table de suivi : les fichiers de `migrations/` sont appliqués **à la main**. Rien ne garantit que l'état réel de la base corresponde au contenu du dossier. Avant d'appliquer 023, il faut constater l'état réel des colonnes.

La migration 013 contient du SQL invalide. `migrations/013_view_once.sql` comporte un point-virgule au milieu de l'instruction `ALTER TABLE` : la seconde colonne, `viewedAt`, n'a jamais pu s'appliquer par ce fichier. Elle figure pourtant dans le schéma consolidé `docs/schema_complet_alanyBD2027.sql`. Il faut trancher lequel des deux décrit la production.

23.2 Collision de numérotation

Le numéro 023 est également revendiqué par `docs/listes-contacts-plan.md`, qui prévoit `023_contact_lists.sql`. Le socle prenant 023, les listes de contacts passent à **024**. Le document doit être corrigé avant sa conversion, sinon les deux migrations entreraient en collision à l'implémentation.

23.3 Écart assumé avec la conception initiale

Conception initiale	Ce document
Table <code>accounts</code> dédiée « Fuir le type <code>ENUM</code> »	Extension de <code>users</code> — 14 tables et 34 contraintes en dépendent déjà Appliqué : <code>TINYINT</code> partout. À noter que le schéma existant en contient déjà (<code>migrations/020</code> et <code>021</code>) ; on ne revient pas dessus
File de jobs dès la conception	Reportée : l'ordonnanceur en processus suffit pour les relances. La file redevient nécessaire au fan-out des diffusions

23.4 Ce qui reste à décider

1. La bascule business → personnel est-elle libre, ou soumise à validation d'un administrateur ?
2. Un compte business dont la vérification a expiré reste-t-il visible dans l'annuaire business, sans badge ?
3. La durée par défaut d'une vérification — un an est l'usage, mais ce n'est pas tranché.
4. Y a-t-il un seul compte officiel, ou plusieurs (support, actualités...) ? Le modèle proposé en supporte plusieurs sans modification.

VOLET 2

Listes de contacts

Appartenance plusieurs-à-plusieurs, cache hors ligne, arbitrage des audiences

CHAPITRE 24

Modèle de données

24.1 Deux tables

Sur le modèle exact de `preferredContact` et `conv_participants` (`migrations/001_initial_schema.sql`).

```
1  -- 024_contact_lists.sql -- listes de contacts + membres (plusieurs a
   plusieurs)
2
3  CREATE TABLE IF NOT EXISTS contact_list (
4      idList      BIGINT      NOT NULL AUTO_INCREMENT,
5      alanyaID    INT          NOT NULL,                -- propriétaire de
   la liste
6      name        VARCHAR(60) NOT NULL,
7      color       VARCHAR(9)  NULL,                    -- #RRGGBB pour la
   puce
8      created_at  DATETIME     NOT NULL DEFAULT CURRENT_TIMESTAMP,
9      PRIMARY KEY (idList),
10     UNIQUE KEY  uq_list_name (alanyaID, name),
11     KEY idx_list_owner (alanyaID),
12     CONSTRAINT fk_list_owner FOREIGN KEY (alanyaID)
13         REFERENCES users(alanyaID) ON DELETE CASCADE
14 ) ENGINE=InnoDB DEFAULT CHARSET=utf8mb4 COLLATE=utf8mb4_unicode_ci;
15
16 CREATE TABLE IF NOT EXISTS contact_list_member (
17     idList      BIGINT      NOT NULL,
18     idFriend    INT          NOT NULL,
19     created_at  DATETIME     NOT NULL DEFAULT CURRENT_TIMESTAMP,
20     PRIMARY KEY (idList, idFriend),
21     KEY idx_clm_friend (idFriend),
22     CONSTRAINT fk_clm_list FOREIGN KEY (idList)
23         REFERENCES contact_list(idList) ON DELETE CASCADE,
24     CONSTRAINT fk_clm_friend FOREIGN KEY (idFriend)
25         REFERENCES users(alanyaID) ON DELETE CASCADE
26 ) ENGINE=InnoDB DEFAULT CHARSET=utf8mb4 COLLATE=utf8mb4_unicode_ci;
```

Listing 24.1 – `migrations/024_contact_lists.sql`

Renumérotation — 023 devient 024. La spécification d’origine visait `023_contact_lists.sql`. Le numéro 023 est pris par le socle de compte, qui doit être appliqué avant. Les migrations restent appliquées à la main : il n’y a ni exécuter, ni table de suivi.

Clé primaire composite plutôt qu’un identifiant technique. `contact_list_member` n’a pas de colonne `id` : sa clé est le couple (`idList`, `idFriend`). Cela interdit gratuitement le doublon d’appartenance — il n’y a rien à vérifier côté applicatif.

24.2 L'appartenance n'est pas contrainte par la base

La règle « un membre doit être un favori du propriétaire » ne peut pas s'exprimer en clé étrangère : elle porte sur le couple (`propriétaire de la liste`, `ami`), pas sur `idFriend` seul. Elle est donc vérifiée dans le contrôleur, à l'ajout, par une requête sur `preferredContact`.

Conséquence à ne pas oublier : si l'utilisateur **retire un favori**, ce contact reste dans ses listes. Deux options — le retirer en cascade applicative, ou le laisser et le filtrer à la lecture. La première est plus propre et se code en trois lignes dans `removePreferredContact` (`src/controllers/preferredContactController.js`)

CHAPITRE 25

API

25.1 Contrôleur — `src/controllers/contactListController.js`

Style copié de `preferredContactController.js` : cadrage sur `req.user.alanyaID`, `parseInt(req.params.id, 10)`, `pool.execute`, codes 400/403/404/409, et les utilitaires existants `sanitizeUrl` et `maskPresenceIfBlocked` (`src/utills/blockUtils.js`).

Handler	Rôle
<code>getLists</code>	Listes du propriétaire + <code>member_count</code> (LEFT JOIN ... GROUP BY)
<code>createList</code>	{ name, color } — 409 si <code>uq_list_name</code> est violé
<code>updateList</code>	Renommer / recolorer, avec contrôle de propriété
<code>deleteList</code>	Le CASCADE supprime les membres
<code>getListMembers</code>	JOIN vers <code>users</code> , même projection que <code>getPreferredContacts</code>
<code>addMember</code>	Vérifie la propriété et que l'ami est un favori — sinon 403
<code>removeMember</code>	—

25.2 Routes — `src/routes/contactLists.js`

Router Express avec auth sur **chaque** route et des blocs `@swagger`, comme `src/routes/contacts.js`.

GET /	<code>getLists</code>
POST /	<code>createList</code>
PUT /:idList	<code>updateList</code>
DELETE /:idList	<code>deleteList</code>
GET /:idList/members	<code>getListMembers</code>
POST /:idList/members/:friendID	<code>addMember</code>
DELETE /:idList/members/:friendID	<code>removeMember</code>

Montage dans `server.js` : `app.use('/api/contact-lists', contactListRoutes)`.

Aucun endpoint de conversation à créer. L'action « créer un groupe depuis une liste » réutilise **POST** `/api/conversations/group`, déjà en service et déjà appelé côté mobile par `createGroup` (`lib/api/chat_api.dart`, ligne 27, signature `{participantIDs, groupName, groupPhoto}`).

CHAPITRE 26

Client Flutter

26.1 Couche données

Fichier	Contenu
lib/api/contact_lists_api.dart (nouveau)	Extension ContactListsApi on TalkyApiClient, part of le client, sur le modèle de lib/api/users_api.dart. Déclarer le part dans talky_api_client.dart.
lib/talky_models.dart	Modèle ContactList : idList, name, color, memberCount, fromJson / toJson.
lib/core/db/app_database.dart	Tables LocalContactLists et LocalContactListMembers (clé composite {idList, idFriend}).
lib/core/services/local_cache_repository.dart	Sectionary.dart listes de contacts », calquée sur watch/syncPreferredContacts.

Version du schéma local — v15 devient v16. La spécification d'origine annonçait un passage de 14 à 15. `schemaVersion` vaut aujourd'hui **14** (lib/core/db/app_database.dart, ligne 253), et le socle de compte l'amène déjà à **15**. Les listes de contacts prennent donc **16** :

```
if (from < 16) {  
  await m.createTable(localContactLists);  
  await m.createTable(localContactListMembers);  
}
```

Puis régénérer : `dart run build_runner build --delete-conflicting-outputs`. La montée doit passer par `onUpgrade`, jamais par une recreation.

Le dépôt local expose : `watchContactLists()` (flux Drift, hors ligne d'abord), `syncContactLists()` (même logique `previousIds` / `newIds` de diff-suppression que `syncPreferredContacts`), `watchListMembers(idList)` et `syncListMembers(idList)`. Les mutations passent en ligne par l'API puis re-synchronisent, comme `addContact` / `removeContact`. Les deux tables sont ajoutées à `clearSession()`.

26.2 Interface

Fichier	Objet
<code>screens/profile/preferred_contacts_screen.dart</code>	Branche de puces au-dessus de la liste; état <code>_selectedListId</code> (null = Tous) combiné à la recherche existante (<code>filterUsersBySearch</code>).
<code>screens/profile/contact_lists_screen.dart</code> (nouveau)	CRUD des listes.
<code>screens/profile/contact_list_detail_screen.dart</code> (nouveau)	Membres, ajout / retrait, bouton « Créer un groupe ».
<code>screens/profile/profile_screen.dart</code>	Entrée « Listes de contacts ».
<code>lib/l10n/*</code>	<code>contactLists</code> , <code>createList</code> , <code>listName</code> , <code>renameList</code> , <code>deleteList</code> , <code>addToList</code> , <code>removeFromList</code> , <code>createGroupFromList</code> , <code>noLists...</code>

Composants à réutiliser, pas à réécrire. Le patron de puce existe déjà : `_buildFilterChip` dans `lib/screens/chats/chats_screen.dart`, **ligne 584** — et non 539–569 comme l'indiquait la spécification d'origine, dont la référence avait vieilli. Pour le reste : `AppAvatar`, `AppSearchField`, `EmptyState`, `AppBottomSheet`, et les jetons `AppSpacing` / `AppRadius` / `context.colors` / `context.l10n`.

CHAPITRE 27

Arbitrage : liste de contacts *n'est pas* audience de diffusion

La conception initiale (`alanya-contexte-conception.md`, §3) proposait *une seule* abstraction d'audience, réutilisée pour les segments de messages officiels, les segments de statuts officiels et les listes de contacts préférés — « ne pas la dupliquer ».

Décision prise : deux modèles distincts. Les deux objets n'ont ni le même propriétaire, ni la même échelle, ni la même nature.

	Liste de contacts	Audience de diffusion
Propriétaire	un utilisateur	un administrateur
Membres	ses favoris, énumérés	tous les comptes, décrits par un critère
Échelle	quelques dizaines	potentiellement toute la base
Nature	des lignes	une <i>requête</i>
Usage	filtrer, créer un groupe	cibler une diffusion

Les fusionner obligerait à choisir entre deux mauvaises options :

1. **matérialiser** l'audience de diffusion en lignes d'appartenance — c'est exactement le *fan-out on write* que le §9 du même document rejette ;
2. **doter** les listes personnelles d'un moteur de critères dont elles n'auront jamais l'usage.

Ce qui reste mutualisé. Deux schémas distincts, mais **un seul résolveur** côté serveur : une fonction qui prend l'un ou l'autre et renvoie la liste des destinataires. La mutualisation se fait au niveau du code d'exécution, où elle est utile, pas au niveau du schéma, où elle coûte.

CHAPITRE 28

Vérification

28.1 Backend

Appliquer `024_contact_lists.sql` sur MySQL, redémarrer, puis tester chaque endpoint via Swagger ou `curl` avec un jeton valide :

1. créer une liste → 201 ;
2. recréer le même nom → 409 ;
3. ajouter un contact **non favori** → 403 ;
4. lister les membres, en retirer un, supprimer la liste ;
5. vérifier que la suppression de la liste n'a supprimé **aucun** favori.

28.2 Client

`flutter pub get` → `dart run build_runner build --delete-conflicting-outputs` → `flutter run`.

1. Créer « Famille », y ajouter trois favoris ; la puce « Famille » filtre la liste.
2. « Créer un groupe » ouvre une conversation de groupe avec ces membres et le nom pré-rempli.
3. **Migration v15** → **v16** sur un appareil déjà installé : la mise à jour ne doit **pas** vider les données existantes.
4. **Hors ligne** : réseau coupé, listes et membres restent visibles depuis le cache Drift ; les mutations affichent un état hors ligne cohérent.
5. Retirer un favori appartenant à une liste, et vérifier le comportement retenu.

28.3 Points ouverts

1. Retrait d'un favori encore présent dans des listes : cascade applicative, ou filtrage à la lecture ?
2. Une limite au nombre de listes par utilisateur ? Rien ne l'impose techniquement, mais la rangée de puces se dégrade au-delà d'une dizaine.
3. Extensions volontairement hors périmètre, possibles sans changer les tables : appel de groupe depuis une liste, diffusion de statut restreinte à une liste.

VOLET 3

Compte officiel et diffusion

Matérialisation paresseuse, résolveur de critères, file de jobs

CHAPITRE 29

La stratégie de livraison

29.1 Le désaccord à trancher

Deux documents existants se contredisaient :

<code>ideas/broadcast.tex</code>	Pour chaque destinataire : créer ou réutiliser la conversation 1-1, insérer une ligne <code>message</code> , incrémenter <code>unreadCount</code> , envoyer le push. C'est un <i>fan-out on write</i> .
<code>alanya-contexte-conception.md</code> §9	« Fan-out on read, pas on write. Un enregistrement de diffusion unique + un état de lecture par utilisateur, matérialisé à la demande . »

En lisant les deux attentivement, la contradiction est moins frontale qu'elle en a l'air : le §9 ne dit pas de renoncer aux lignes de message, il dit de ne pas les écrire *au moment de l'envoi*. C'est une question de *quand*, pas de *quoi*.

29.2 Les trois options, chiffrées

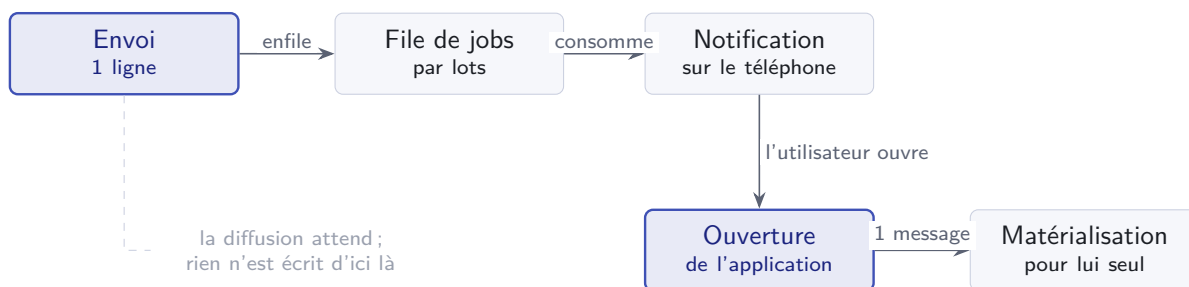
	Écriture immédiate	Matérialisation paresseuse	Lecture pure
À l'envoi	N conversations + N messages	1 ligne	1 ligne
Pipeline existant	réutilisé	réutilisé	à réécrire
Cache Drift	gratuit	gratuit	à refaire
Non-lus, accusés	gratuits	gratuits	à refaire
Effort	faible	moyen	élevé
Tient à 300 k	non	oui	oui

Décision : matérialisation paresseuse. À 300 000 comptes, l'écriture immédiate produirait 300 000 lignes `message` en une transaction. La lecture pure, elle, obligerait à réécrire tout le chemin de livraison — synchronisation, cache local, compteurs de non-lus, accusés de réception — pour un seul type de message.

29.3 Le moment de la matérialisation

Vérifié dans le code. `getMessagesSince` (`src/controllers/messageController.js`, ligne 922) ne synchronise que les conversations dont **le client envoie déjà le curseur**, et sort immédiatement en `{messages: [], hasMore: false}` si la liste est vide (ligne 941). Une conversation créée côté serveur n'est donc jamais découverte par ce chemin.

Le point d'accroche est donc `GET /api/conversations` — la liste des conversations, qui est le seul endroit où une conversation inconnue du client peut apparaître. Avant de répondre, on matérialise les diffusions en attente pour cet utilisateur.



29.4 Le garde-fou contre la dérive

Un critère évalué à la lecture pose un problème que l'écriture immédiate n'avait pas : un compte **créé après** la diffusion correspondrait au critère et recevrait un message qui ne lui était pas destiné.

Une clause suffit :

```
AND u.created_at <= b.sent_at
```

Pourquoi ne pas figer l'audience. On pourrait enregistrer la liste des destinataires à l'envoi. Ce serait exactement le *fan-out on write* qu'on vient d'écarter, avec les mêmes 300 000 lignes. La clause ci-dessus couvre le seul cas réaliste de dérive. Les autres — un compte devenu business après l'envoi, par exemple — sont acceptés : le critère décrit à *qui le message s'adresse*, et cette description reste vraie.

CHAPITRE 30

Modèle de données

30.1 Migration 025_broadcast.sql

```
1  -- 025_broadcast.sql -- compte officiel, diffusions et file de jobs
2  -- Requiert 023 (socle de compte) et 024 (listes de contacts).
3
4  CREATE TABLE IF NOT EXISTS broadcast (
5      id          BIGINT      NOT NULL AUTO_INCREMENT,
6      sender_id   INT         NOT NULL,      -- compte officiel
7              (account_type = 2)
8      created_by  INT         NOT NULL,      -- admin auteur de la
9              diffusion
10     kind         TINYINT     NOT NULL DEFAULT 0 COMMENT '0=message
11              1=statut',
12     content      TEXT        NULL,
13     type         TINYINT     NOT NULL DEFAULT 0 COMMENT '0=texte
14              1=image 2=video',
15     media_url    VARCHAR(255) NULL,
16     criteria     JSON        NOT NULL      COMMENT 'Critere de ciblage,
17              compile a la volee',
18     estimate     INT         NOT NULL DEFAULT 0 COMMENT 'Destinataires
19              estimees a l envoi',
20     statut_id    INT         NULL          COMMENT 'Ligne statut creee
21              (kind = 1)',
22     sent_at      DATETIME     NOT NULL DEFAULT CURRENT_TIMESTAMP,
23     PRIMARY KEY (id),
24     KEY idx_broadcast_sent (sent_at),
25     CONSTRAINT fk_bc_sender FOREIGN KEY (sender_id) REFERENCES
26         users(alanyaID),
27     CONSTRAINT fk_bc_admin  FOREIGN KEY (created_by) REFERENCES
28         users(alanyaID),
29     CONSTRAINT fk_bc_statut FOREIGN KEY (statut_id) REFERENCES
30         statut(ID) ON DELETE SET NULL
31 ) ENGINE=InnoDB DEFAULT CHARSET=utf8mb4 COLLATE=utf8mb4_unicode_ci;
32
33 -- Une ligne par destinataire AYANT REELLEMENT reçu : creee a la
34 -- materialisation, pas a l'envoi. Donne le taux d'ouverture
35      gratuitement.
36
37 CREATE TABLE IF NOT EXISTS broadcast_delivery (
38     broadcast_id BIGINT NOT NULL,
39     alanyaID     INT    NOT NULL,
40     conversation_id BIGINT NULL,
41     message_id   BIGINT NULL,
42     delivered_at  DATETIME NOT NULL DEFAULT CURRENT_TIMESTAMP,
43     PRIMARY KEY (broadcast_id, alanyaID),
44     KEY idx_bd_user (alanyaID),
45     CONSTRAINT fk_bd_broadcast FOREIGN KEY (broadcast_id)
```

```
34 REFERENCES broadcast(id) ON DELETE CASCADE ,
35 CONSTRAINT fk_bd_user FOREIGN KEY (alanyaID)
36 REFERENCES users(alanyaID) ON DELETE CASCADE
37 ) ENGINE=InnoDB DEFAULT CHARSET=utf8mb4 COLLATE=utf8mb4_unicode_ci;
38
39 -- Filigrane : jusqu'ou cet utilisateur a deja ete servi.
40 ALTER TABLE users
41 ADD COLUMN last_broadcast_id BIGINT NOT NULL DEFAULT 0
42 COMMENT 'Derniere diffusion evaluee pour ce compte (materialisee
    ou non)';
```

Listing 30.1 – Diffusions et livraisons

Pourquoi un filigrane sur users. Sans lui, chaque GET /api/conversations devrait comparer l'utilisateur à toutes les diffusions jamais envoyées. Avec lui, la requête ne remonte que `broadcast.id > u.last_broadcast_id`. Le filigrane avance **même quand le critère ne correspond pas** — on ne réévalue jamais deux fois la même diffusion pour le même compte.

Le taux d'ouverture sort tout seul. `broadcast_delivery` n'existe que pour les comptes ayant rouvert l'application depuis l'envoi. Le taux affiché dans l'historique est donc `COUNT(delivery) / broadcast.estimate` — aucune instrumentation supplémentaire.

CHAPITRE 31

Le résolveur de critères

31.1 Format stocké

```
1 {
2   "v": 1,
3   "op": "and",
4   "conditions": [
5     { "field": "pays",      "op": "in",      "value": [5, 7] },
6     { "field": "account_type", "op": "eq",      "value": 1 },
7     { "field": "created_at",  "op": "before", "value": "-3 months" },
8     { "field": "last_seen",   "op": "after",  "value": "-30 days" }
9   ]
10 }
```

Listing 31.1 – Exemple de valeur de `broadcast.criteria`

Le champ `v` porte la version du format : une diffusion relue dans deux ans doit rester interprétable même si le vocabulaire a évolué.

La règle du §7 sur le JSON est respectée, pas contournée. `alanya-contexte-conception.md` déconseille les colonnes JSON « dès qu'elles font l'objet d'un `WHERE` ». Ici, `criteria` n'est **jamais** filtré en SQL : il est lu, puis *compilé* en clauses. Aucune requête ne cherche à l'intérieur.

31.2 Deux évaluateurs pour un même critère

C'est le point qui rend l'ensemble simple. Le résolveur a deux modes, et l'un des deux n'écrit pas de SQL du tout :

Mode	Usage	Fonctionnement
Compilation SQL	estimation, parcours pour le push	Produit un fragment <code>WHERE</code> paramétré, appliqué sur <code>users</code> .
Évaluation en mémoire	« ce compte précis correspond-il ? »	Pure fonction JavaScript sur la ligne <code>users</code> déjà chargée. Aucune requête.

La matérialisation et la visibilité des statuts n'ont besoin que du **second** mode : l'utilisateur est connu, sa ligne est déjà en main, il suffit de tester ses attributs.

Invariant à tester. Les deux modes doivent toujours répondre la même chose. C'est exactement le genre d'accord qui se rompt silencieusement à la première évolution. Un test qui, pour un jeu de comptes, compare le résultat des deux évaluateurs sur chaque critère du catalogue est indispensable.

31.3 Sécurité : compiler n'est pas concaténer

Un critère venu du client, compilé en SQL, est une surface d'injection s'il est traité naïvement.

- Le champ (`field`) est résolu par une **liste blanche** vers un nom de colonne. Toute valeur inconnue est un rejet, pas un passage.
- L'opérateur vient d'une énumération fermée par type de champ.
- La valeur est **toujours** un paramètre lié (?), jamais interpolée.

Ce que le résolveur produit

Le critère est stocké en JSON puis **compilé** en clauses SQL. Il n'est jamais filtré en base — la règle du §7 sur les colonnes JSON est respectée.

```
-- critere stocke sur la ligne broadcast, compile a la volee
SELECT COUNT(*) FROM users u
WHERE u.exclus = 0
  AND u.created_at <= b.sent_at -- garde anti-derive
  AND u.idPays = 5
  AND u.account_type = 1;
```

Ce comptage n'est demandé qu'ici. À l'envoi, la liste des destinataires n'est jamais construite : le critère ne sert qu'à ce `COUNT(*)` et au fan-out push par lots.

Historique

Chaque diffusion conserve son critère, pour qu'on puisse relire six mois plus tard à qui elle s'adressait.

ENVOYÉE	NATURE	CONTENU	CIBLE	DESTINATAIRES	OUVERTURES
24 juil. 09:12	• Message	Maintenance programmée dimanche de 2 h à 4 h	pays = Cameroun	128 430	71 %
19 juil. 17:40	• Statut	Les listes de contacts arrivent	tous	309 004	44 %
11 juil. 08:05	• Message	Votre vérification expire dans 30 jours	business A vérifié A échéance < 30 j	1 246	92 %

31.4 Ce qu'on n'accepte pas comme critère

Tous les champs retenus sont indexés :

Critère	Colonne	Index
Ancienneté	<code>users.created_at</code>	<code>idx_users_created_at</code> (migration 005)
Activité	<code>users.last_seen</code>	<code>idx_users_online</code> (migration 001)
Pays	<code>users.idPays</code>	clé étrangère
Genre, état	<code>account_type</code> , <code>verification_status</code>	<code>idx_users_account_type</code> (migration 023)

Critère écarté. « Nombre de messages envoyés » exigerait une agrégation sur `message`, la plus grosse table de la base, à chaque estimation. À écarter, ou à pré-calculer dans une colonne entretenue — pas à compiler à la volée.

CHAPITRE 32

Les statuts officiels

32.1 Le correctif de visibilité

État actuel. `getStatus` (`src/controllers/statutController.js`, lignes 29–30) impose deux jointures INNER sur `preferredContact` : l’auteur doit avoir le lecteur en favori **et** réciproquement.

`ideas/broadcast.tex` proposait d’ajouter une colonne `isBroadcast` sur `statut`. Elle est inutile depuis le socle : l’information est déjà sur l’auteur.

```
1 FROM statut s
2 JOIN users u ON s.alanyaID = u.alanyaID
3 LEFT JOIN preferredContact pc1 ON pc1.alanyaID = s.alanyaID AND
   pc1.idFriend = ?
4 LEFT JOIN preferredContact pc2 ON pc2.alanyaID = ? AND
   pc2.idFriend = s.alanyaID
5 WHERE s.expiredAt > NOW()
6 AND (
7     u.account_type = 2 -- officiel : pas de
   reciprocite exigee
8 OR (pc1.idPrefContact IS NOT NULL -- sinon : regle inchangee
9 AND pc2.idPrefContact IS NOT NULL)
10 )
11 ...
```

Listing 32.1 – `getStatus` — jointures assouplies

32.2 Les statuts n’ont pas besoin de matérialisation

Asymétrie qu’il faut voir pour ne pas sur-concevoir : un statut est **une seule ligne lue par tout le monde**. Il est nativement en *fan-out on read*. Un message, lui, vit dans une conversation propre à chaque destinataire.

Un statut officiel se réduit donc à : une ligne `statut` portée par le compte officiel, plus le filtre de critère appliqué au lecteur — en mode évaluation en mémoire, sur la ligne `users` déjà chargée par la requête. Aucune table, aucun travail de fond.

CHAPITRE 33

La file de jobs

33.1 Pourquoi elle devient obligatoire ici

Quelle que soit la stratégie de livraison, envoyer les **notifications** reste un parcours de N destinataires. C'est le seul véritable fan-out qui subsiste, et à 300 000 comptes il ne peut pas tenir dans le cycle d'une requête HTTP.

Nuance à ne pas gommer. On lit souvent que la matérialisation paresseuse évite de parcourir les destinataires. C'est faux : elle évite de leur **écrire des lignes de message**. Le parcours pour le push existe bien — mais il se fait par pages, en arrière-plan, sans rien persister.

33.2 Table plutôt que Redis

Le §7 de `alanya-contexte-conception.md` recommandait BullMQ sur Redis. On retient une **table MySQL** : aucun service supplémentaire à héberger, sauvegarder et surveiller, pour un besoin qui se limite aujourd'hui au push par lots.

```
1 CREATE TABLE IF NOT EXISTS job_queue (  
2   id          BIGINT      NOT NULL AUTO_INCREMENT,  
3   kind        VARCHAR(40) NOT NULL COMMENT 'broadcast_push, ...',  
4   payload     JSON        NOT NULL,  
5   run_after   DATETIME    NOT NULL DEFAULT CURRENT_TIMESTAMP,  
6   attempts    TINYINT     NOT NULL DEFAULT 0,  
7   max_attempts TINYINT     NOT NULL DEFAULT 5,  
8   locked_at   DATETIME    NULL,  
9   locked_by   VARCHAR(64) NULL,  
10  failed_at   DATETIME    NULL,  
11  last_error  TEXT         NULL,  
12  created_at  DATETIME    NOT NULL DEFAULT CURRENT_TIMESTAMP,  
13  PRIMARY KEY (id),  
14  KEY idx_job_pret (kind, failed_at, locked_at, run_after)  
15 ) ENGINE=InnoDB DEFAULT CHARSET=utf8mb4;
```

Listing 33.1 – File de jobs — même migration 025

33.3 Consommation concurrente

SKIP LOCKED (MySQL 8, déjà la cible du schéma) permet à plusieurs consommateurs de prendre des lots disjoints sans se bloquer :

```
START TRANSACTION;  
SELECT id, payload FROM job_queue  
WHERE kind = ? AND locked_at IS NULL AND failed_at IS NULL
```

```
    AND run_after <= NOW()
  ORDER BY run_after, id
  LIMIT 20
  FOR UPDATE SKIP LOCKED;

UPDATE job_queue
SET locked_at = NOW(), locked_by = ?, attempts = attempts + 1
WHERE id IN (...);
COMMIT;
```

Un job réussi est supprimé; un job échoué est déverrouillé avec `run_after` repoussé (recul exponentiel) jusqu'à `max_attempts`, puis marqué `failed_at` et laissé pour inspection.

33.4 Découpage du fan-out

L'enfileur parcourt les destinataires par **pagination par clé** sur `alanyaID` — jamais par `OFFSET`, qui se dégrade sur des centaines de milliers de lignes — et crée un job par lot de quelques centaines. Une diffusion à 300 000 comptes produit quelques centaines de jobs, pas 300 000.

Réutiliser l'existant. `src/services/meetingScheduler.js` fournit déjà le squelette d'une boucle de fond (`setInterval` d'une minute, démarrage depuis `server.js`). Le consommateur de jobs suit le même patron. `notificationService.js` envoie déjà les push par lots.

CHAPITRE 34

Impact fichier par fichier

34.1 Backend

Fichier	Nature	Objet
migrations/025_broadcast.sql	nouveau	broadcast, broadcast_delivery, job_queue, filigrane
src/services/criteriaResolver.js	nouveau	Compilation SQL <i>et</i> évaluation en mémoire
src/services/jobQueue.js	nouveau	Enfilage, consommation SKIP LOCKED, recul exponentiel
src/services/broadcastService.js	nouveau	Envoi, enfilage du push, matérialisation
src/controllers/admin/broadcast.js	nouveau	CRUD, estimation, historique
src/routes/admin.js	modifié	GET / POST /broadcasts, GET /broadcasts/:id, POST /broadcasts/estimate
src/controllers/conversationController.js	modifié	Matérialisation avant de répondre à la liste
src/controllers/statutController.js	modifié	Jointures en LEFT + critère
server.js	modifié	Démarrage du consommateur de jobs

34.2 Administration

L'écran existe déjà, branché sur des données simulées (app/(dashboard)/broadcasts/page.tsx, components/broadcasts/*, hooks/useBroadcasts.ts → lib/mock-data.ts). Il faut le brancher et remplacer le ciblage :

Fichier	Objet
components/broadcasts/BroadcastDialogForm	Remplacer les trois boutons radio targetType par le constructeur de conditions
components/broadcasts/CriteriaBuilderAjust (nouveau)	Ajust, retrait et édition des conditions
components/broadcasts/RecipientEstimateForm (nouveau)	Estimation en direct, appel anti-rebond sur /broadcasts/estimate
types/index.ts	targetType / targetCriteria deviennent criteria
lib/mock-data.ts	Retirer la bascule simulée pour les diffusions

Déjà anticipé. BroadcastFormData contient déjà un booléen `isStatus` (`types/index.ts`, ligne 324) : la distinction message / statut avait été prévue.

34.3 Mobile

Peu de travail, et c’est la récompense du choix de livraison : une diffusion arrive comme un message ordinaire.

Fichier	Objet
<code>lib/screens/chats/chat/chat_input.data</code>	Bandeau à la place de la saisie (étape 1)
<code>lib/screens/chats/chat/chat_bubbles.data</code>	Retrait de « répondre » et « réagir » (étape 1)
<code>lib/screens/status/statuses_screen.data</code>	Aucun changement — le statut officiel arrive par la requête corrigée

CHAPITRE 35

Points de vigilance et questions ouvertes

35.1 Chantier d'infrastructure nommé

La **file de jobs** n'existe pas aujourd'hui dans le backend. Ce n'est pas un détail d'implémentation : c'est une brique à construire, tester et surveiller, avec sa propre supervision (jobs en échec, jobs bloqués par un consommateur mort). À budgéter comme telle.

Verrous orphelins. Un consommateur tué laisse ses jobs `locked_at` non nul indéfiniment. Il faut une reprise : tout job verrouillé depuis plus de N minutes est considéré comme abandonné et redevient disponible.

35.2 Coût de la matérialisation sur le chemin chaud

`GET /api/conversations` est appelé à chaque ouverture de l'application. Y ajouter un travail conditionnel demande de la rigueur : la requête de filigrane doit être triviale quand il n'y a rien à faire — c'est-à-dire dans la quasi-totalité des appels.

35.3 Questions ouvertes

1. Une diffusion peut-elle être **annulée** après envoi ? Techniquement oui tant qu'elle n'est pas matérialisée pour tout le monde — mais elle l'aura déjà été pour une partie, et les push seront partis. Un rappel partiel est-il pire que rien ?
2. Faut-il **programmer** une diffusion à une date future ? La file de jobs le permet gratuitement via `run_after`.
3. Combien de temps conserve-t-on les lignes `broadcast_delivery` ? Elles grossissent avec chaque diffusion.
4. Un utilisateur peut-il **se désabonner** des annonces ? Aucune obligation légale pour des notifications de service, mais la question se posera.
5. Plusieurs comptes officiels (support, actualités) — le modèle le permet ; faut-il l'exposer dès maintenant dans l'administration ?

VOLET 4

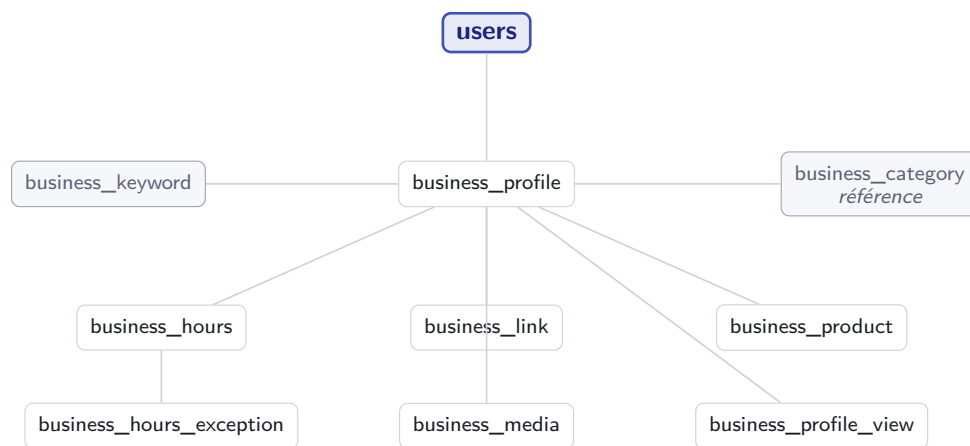
Comptes business

Index spatial, horaires, catalogue, transcodage des médias

CHAPITRE 36

Modèle de données

36.1 Vue d'ensemble



36.2 Le profil

```
1  -- 026_business.sql -- profil business étendu
2  -- Requier 023 (socle de compte).
3
4  CREATE TABLE IF NOT EXISTS business_category (
5      id          SMALLINT      NOT NULL AUTO_INCREMENT,
6      slug        VARCHAR(40)   NOT NULL,
7      libelle     VARCHAR(80)   NOT NULL,
8      ordre       SMALLINT      NOT NULL DEFAULT 0,
9      actif       TINYINT       NOT NULL DEFAULT 1,
10     PRIMARY KEY (id),
11     UNIQUE KEY uq_cat_slug (slug)
12 ) ENGINE=InnoDB DEFAULT CHARSET=utf8mb4 COLLATE=utf8mb4_unicode_ci;
13
14 CREATE TABLE IF NOT EXISTS business_profile (
15     alanyaID     INT            NOT NULL,          -- 1-1 avec le compte
16     legal_name   VARCHAR(120)  NOT NULL,
17     description  TEXT           NULL,
18     category_id  SMALLINT      NULL,
19     cover_url    VARCHAR(255)  NULL,
20     address      VARCHAR(255)  NULL,
21     -- SPATIAL INDEX impose NOT NULL : on prévoit la colonne des
22     -- maintenant,
23     -- l'ajouter apres coup sur une table remplie est penible.
24     geo          POINT         NOT NULL SRID 4326,
```

```

24  timezone      VARCHAR(64)  NULL,           -- sinon deduit via
      users.idPays
25  is_hidden     TINYINT      NOT NULL DEFAULT 0
26  COMMENT 'Profil conserve mais masque (retour en compte
      personnel)',
27  created_at    DATETIME     NOT NULL DEFAULT CURRENT_TIMESTAMP,
28  updated_at    DATETIME     NOT NULL DEFAULT CURRENT_TIMESTAMP
29                      ON UPDATE CURRENT_TIMESTAMP,
30  PRIMARY KEY (alanyaID),
31  SPATIAL INDEX idx_bp_geo (geo),
32  KEY idx_bp_categorie (category_id, is_hidden),
33  CONSTRAINT fk_bp_user FOREIGN KEY (alanyaID)
34      REFERENCES users(alanyaID) ON DELETE CASCADE,
35  CONSTRAINT fk_bp_cat FOREIGN KEY (category_id)
36      REFERENCES business_category(id)
37 ) ENGINE=InnoDB DEFAULT CHARSET=utf8mb4 COLLATE=utf8mb4_unicode_ci;

```

Listing 36.1 – migrations/026_business.sql — extrait

Deux contraintes MySQL à ne pas découvrir en route. SPATIAL INDEX exige que la colonne soit NOT NULL et porte un SRID explicite. Un commerce sans coordonnées connues doit donc recevoir une position par défaut — celle du chef-lieu de son pays — plutôt qu'un NULL. Et utf8mb4 est indispensable : le premier emoji dans un nom d'entreprise ferait échouer l'insertion sur un jeu plus étroit.

36.3 Liens, médias, mots-clés

```

1  -- Pas une colonne par reseau social : ajouter TikTok demain ne doit
2  -- pas demander de migration.
3  CREATE TABLE IF NOT EXISTS business_link (
4      id          BIGINT      NOT NULL AUTO_INCREMENT,
5      alanyaID    INT         NOT NULL,
6      kind        TINYINT     NOT NULL COMMENT '0=site 1=facebook
7      2=instagram ...',
8      url         VARCHAR(255) NOT NULL,
9      ordre       SMALLINT    NOT NULL DEFAULT 0,
10     PRIMARY KEY (id),
11     KEY idx_bl_owner (alanyaID, ordre),
12     CONSTRAINT fk_bl_user FOREIGN KEY (alanyaID)
13         REFERENCES business_profile(alanyaID) ON DELETE CASCADE
14 ) ENGINE=InnoDB DEFAULT CHARSET=utf8mb4;
15
16 CREATE TABLE IF NOT EXISTS business_keyword (
17     alanyaID    INT         NOT NULL,
18     keyword     VARCHAR(40) NOT NULL COMMENT 'Deja normalise :
19     minuscules, accents replies',
20     PRIMARY KEY (alanyaID, keyword),
21     KEY idx_bk_mot (keyword),
22     CONSTRAINT fk_bk_user FOREIGN KEY (alanyaID)
23         REFERENCES business_profile(alanyaID) ON DELETE CASCADE
24 ) ENGINE=InnoDB DEFAULT CHARSET=utf8mb4;
25
26 -- Images ET videos de presentation. Une seule table : la difference
27 -- au champ 'kind' et aux colonnes de transcodage, nulles pour une
28 -- image.

```



```

26 CREATE TABLE IF NOT EXISTS business_media (
27   id          BIGINT      NOT NULL AUTO_INCREMENT,
28   alanyaID    INT         NOT NULL,
29   kind        TINYINT     NOT NULL DEFAULT 0 COMMENT '0=image
      1=video',
30   role        TINYINT     NOT NULL DEFAULT 0 COMMENT '0=galerie
      1=couverture',
31   url         VARCHAR(255) NOT NULL,
32   thumb_url   VARCHAR(255) NULL COMMENT 'Vignette : obligatoire
      pour une video',
33   width       SMALLINT    NULL,
34   height      SMALLINT    NULL,
35   duration_ms INT         NULL COMMENT 'Video uniquement',
36   size_bytes  BIGINT      NULL,
37   -- Cycle de vie du transcodage, pilote par la file de jobs (etape
      3).
38   status       TINYINT     NOT NULL DEFAULT 0
      COMMENT '0=en attente 1=pret 2=echec',
39   ordre       SMALLINT    NOT NULL DEFAULT 0,
40   created_at  DATETIME    NOT NULL DEFAULT CURRENT_TIMESTAMP,
41   PRIMARY KEY (id),
42   KEY idx_bm_owner (alanyaID, role, ordre),
43   KEY idx_bm_statut (status),
44   CONSTRAINT fk_bm_user FOREIGN KEY (alanyaID)
45     REFERENCES business_profile(alanyaID) ON DELETE CASCADE
46 ) ENGINE=InnoDB DEFAULT CHARSET=utf8mb4;
47

```

Listing 36.2 – Tables satellites

Une seule table pour les images et les vidéos. Les séparer obligerait à fusionner deux listes à chaque affichage de galerie, et à dupliquer l'ordre d'affichage. `kind` distingue les deux ; les colonnes propres à la vidéo (`duration_ms`, `thumb_url`, `status`) restent nulles pour une image.

`status` n'est pas décoratif. Une vidéo est *visible mais pas encore lisible* tant que le transcodage n'a pas abouti. Sans cet état, la fiche afficherait une vidéo qui ne démarre pas — et le commerçant conclurait que l'application est cassée.

36.4 Horaires

```

1 CREATE TABLE IF NOT EXISTS business_hours (
2   id          BIGINT      NOT NULL AUTO_INCREMENT,
3   alanyaID    INT         NOT NULL,
4   weekday     TINYINT     NOT NULL COMMENT '1=lundi ... 7=dimanche',
5   -- Minutes depuis minuit : 480 = 08:00. Un TIME suffirait, mais
      l'entier
6   -- rend le passage de minuit trivial (fin < debut => le lendemain).
7   start_min   SMALLINT    NULL,
8   end_min     SMALLINT    NULL,
9   all_day     TINYINT     NOT NULL DEFAULT 0 COMMENT '1 = 24h/24,
      start/end ignores',
10  PRIMARY KEY (id),
11  KEY idx_bh_jour (alanyaID, weekday),
12  CONSTRAINT fk_bh_user FOREIGN KEY (alanyaID)
13    REFERENCES business_profile(alanyaID) ON DELETE CASCADE
14 ) ENGINE=InnoDB DEFAULT CHARSET=utf8mb4;

```

```

15
16 -- Absence de ligne pour un jour = ferme ce jour-la.
17 -- Plusieurs lignes pour un meme jour = plusieurs creneaux.
18
19 CREATE TABLE IF NOT EXISTS business_hours_exception (
20     id          BIGINT          NOT NULL AUTO_INCREMENT,
21     alanyaID    INT             NOT NULL,
22     day         DATE            NOT NULL,
23     closed      TINYINT         NOT NULL DEFAULT 1,
24     start_min   SMALLINT        NULL,    -- horaires speciaux si closed = 0
25     end_min     SMALLINT        NULL,
26     label       VARCHAR(80)     NULL,    -- "fete nationale"
27     PRIMARY KEY (id),
28     UNIQUE KEY uq_bhe (alanyaID, day),
29     CONSTRAINT fk_bhe_user FOREIGN KEY (alanyaID)
30         REFERENCES business_profile(alanyaID) ON DELETE CASCADE
31 ) ENGINE=InnoDB DEFAULT CHARSET=utf8mb4;

```

Listing 36.3 – Horaires et exceptions

Trois conventions qui évitent des cas particuliers. **Pas de ligne** pour un jour signifie *fermé* — inutile de stocker l'absence. **Plusieurs lignes** pour le même jour donnent les créneaux multiples, sans colonne supplémentaire. Et `end_min < start_min` signifie *le lendemain*, ce qui règle le passage de minuit sans champ dédié.

36.5 Catalogue

```

1 CREATE TABLE IF NOT EXISTS business_product (
2     id          BIGINT          NOT NULL AUTO_INCREMENT,
3     alanyaID    INT             NOT NULL,
4     name        VARCHAR(120)    NOT NULL,
5     description  TEXT            NULL,
6     -- Entier en unite mineure + code devise. JAMAIS un flottant.
7     price_minor BIGINT          NULL,
8     currency    CHAR(3)         NOT NULL DEFAULT 'XAF',
9     unit        VARCHAR(40)     NULL COMMENT 'boite de 20, 500 ml',
10    media_url    VARCHAR(255)    NULL,
11    availability  TINYINT         NOT NULL DEFAULT 0
12    COMMENT '0=disponible 1=sur commande 2=epuise',
13    ordre       SMALLINT        NOT NULL DEFAULT 0,
14    is_hidden    TINYINT         NOT NULL DEFAULT 0,
15    created_at   DATETIME        NOT NULL DEFAULT CURRENT_TIMESTAMP,
16    PRIMARY KEY (id),
17    KEY idx_bpr_owner (alanyaID, is_hidden, ordre),
18    CONSTRAINT fk_bpr_user FOREIGN KEY (alanyaID)
19        REFERENCES business_profile(alanyaID) ON DELETE CASCADE
20 ) ENGINE=InnoDB DEFAULT CHARSET=utf8mb4 COLLATE=utf8mb4_unicode_ci;

```

Listing 36.4 – Produits

Le prix n'est jamais un flottant. 1 200 FCFA se stocke 120000 en unité mineure, avec XAF à côté. Un FLOAT ou un DOUBLE sur des prix finit toujours par produire un centime fantôme, et le franc CFA n'a pas de subdivision — raison de plus pour que l'unité soit explicite plutôt que devinée.

CHAPITRE 37

Un lieu, ou plusieurs

37.1 Ce qui est retenu

Un seul lieu par compte. L'adresse, la position et les horaires sont portés par `business_profile`. Une chaîne de trois boutiques crée aujourd'hui trois comptes.

37.2 La variante multi-lieux

Elle est documentée ici parce qu'elle est envisagée comme **fonctionnalité payante** et qu'il vaut mieux en connaître le coût avant de la promettre.

	Un lieu (retenu)	Plusieurs lieux
Adresse, position, horaires	sur <code>business_profile</code>	déplacés dans <code>business_location</code>
Annuaire	cherche des <i>comptes</i>	cherche des <i>lieux</i> , remonte le compte
Index spatial	sur <code>business_profile</code>	sur <code>business_location</code>
Éditeur d'horaires	un jeu	un jeu par lieu
Fiche	une adresse	sélecteur de lieu en tête de fiche

Coût de bascule : création de `business_location`, migration des colonnes d'adresse, de position et d'horaires depuis `business_profile`, déplacement de l'index spatial, et réécriture de la requête d'annuaire. Le reste — catalogue, liens, médias, mots-clés — n'est pas touché.

Comment se garder la porte ouverte à moindre frais. Concevoir dès maintenant les requêtes d'annuaire pour renvoyer un couple (*compte, lieu*) plutôt qu'un compte seul. Avec un seul lieu, le second terme est constant ; le jour de la bascule, seule la source change, pas les appelants.

Le déblocage vient de l'abonnement, pas du badge. Si les lieux multiples étaient conditionnés à `verification_status = 2`, le badge deviendrait une fonctionnalité qui se vend — exactement le recouplage entre identité et argent que le découplage de l'étape 5 cherche à éviter. La condition doit porter sur `subscriptions`. En pratique l'utilisateur voit la même chose ; dans le modèle, la porte n'est pas ouverte par la même clé.

CHAPITRE 38

Annuaire et recherche de proximité

38.1 L'état actuel

Il n'y a aucune donnée géographique. Le schéma ne contient pas une seule colonne de position. La page « Géolocalisation » de l'administration est une table de **54 pays codée en dur** (talky-admin/lib/geo-utils.ts), alimentée par des *noms de pays en chaîne* issus de /api/admin/stats. Il n'existe pas d'endpoint géographique côté backend.

L'annuaire de proximité part donc de zéro. Ce n'est pas une évolution de l'existant, c'est une brique nouvelle.

38.2 La requête

```
1 SELECT b.alanyaID, u.nom, b.legal_name, c.libelle,
2         ST_Distance_Sphere(b.geo, ST_SRID(POINT(?, ?), 4326)) AS metres
3 FROM business_profile b
4 JOIN users u ON u.alanyaID = b.alanyaID
5 LEFT JOIN business_category c ON c.id = b.category_id
6 WHERE b.is_hidden = 0
7       AND u.exclus = 0
8       AND MBRContains(
9         ST_Buffer(ST_SRID(POINT(?, ?), 4326), ?),    -- prefiltre sur
10         l'index
11         b.geo)
12       AND (? IS NULL OR b.category_id = ?)
13 ORDER BY metres
14 LIMIT 50;
```

Listing 38.1 – Commerces ouverts, par proximité

Pourquoi le préfiltre. `ST_Distance_Sphere` est une fonction : seule, elle impose un balayage complet. L'index spatial ne sert que si la requête contient une contrainte de *contenance* (`MBRContains`) qu'il sait exploiter. On restreint donc d'abord à un rectangle, puis on trie sur la distance exacte à l'intérieur.

38.3 « Ouvert maintenant »

Ce filtre ne se calcule pas en SQL sans douleur : il dépend du jour courant *dans le fuseau du commerce*, des créneaux multiples, du cas 24h/24, du passage de minuit et des exceptions. On charge les horaires des résultats retenus — au plus cinquante — et on évalue en mémoire. La même fonction sert à l'affichage « ferme à 19h » sur la fiche.

CHAPITRE 39

Tableau de bord d'audience

39.1 Journal d'événements

```
1 CREATE TABLE IF NOT EXISTS business_profile_view (  
2   id          BIGINT NOT NULL AUTO_INCREMENT,  
3   alanyaID    INT     NOT NULL,           -- le commerce vu  
4   viewer_id   INT     NULL,              -- NULL si non connecte  
5   source      TINYINT NOT NULL DEFAULT 0  
6               COMMENT '0=annuaire 1=recherche 2=conversation 3=lien  
7               partage',  
8   viewed_at   DATETIME NOT NULL DEFAULT CURRENT_TIMESTAMP,  
9   PRIMARY KEY (id),  
10  KEY idx_bpv_owner_date (alanyaID, viewed_at),  
11  CONSTRAINT fk_bpv_biz FOREIGN KEY (alanyaID)  
12  REFERENCES business_profile(alanyaID) ON DELETE CASCADE  
   ) ENGINE=InnoDB DEFAULT CHARSET=utf8mb4;
```

Listing 39.1 – Vues de fiche

Une table qui grossit vite. Chaque ouverture de fiche est une ligne. Il faut décider dès maintenant : anti-rebond côté serveur (une vue par visiteur et par heure), agrégation quotidienne dans une table de synthèse, et purge du détail au-delà de quelques mois. Sans cela, cette table dépassera **message** en volume.

39.2 Ce que le commerçant voit

Vues sur la période, évolution, répartition par provenance et par pays, conversations entamées depuis la fiche. Tout est dérivé de cette table et de **conversation**, sans saisie du commerçant.

CHAPITRE 40

Médias : stockage et transcodage

40.1 Ce que l'existant sait déjà faire

Vérifié dans le code. `src/middleware/upload.js` accepte déjà les images (jpeg, png, webp, gif) et les vidéos (mp4, webm, quicktime, 3gpp), plafonne à **50 Mo** par fichier, range par type dans `uploads/media/{images,audio,video,files}` et sert le tout en statique sur `/uploads`. Côté mobile, `video_thumbnail` et `image_thumbnail_service` produisent déjà des vignettes pour les messages.

Le téléversement n'est donc pas à construire. Ce qui manque, c'est tout ce qui vient après.

40.2 Trois problèmes que la fiche business introduit

Le volume	Un message vidéo est vu une fois par un destinataire. Une vidéo de fiche est servie à <i>chaque</i> ouverture de la fiche, à tous les visiteurs. Le rapport n'est pas du même ordre.
Le poids d'origine	50 Mo est acceptable pour un envoi ponctuel entre deux personnes. C'est intenable pour un média public consulté depuis un réseau mobile.
La diversité des formats	Un enregistrement issu d'un téléphone Android bas de gamme et un autre d'un iPhone récent n'ont ni le même codec, ni la même orientation, ni la même cadence.

40.3 La chaîne de traitement



Le transcodage est le **deuxième client** de la file de jobs, après le fan-out des notifications — exactement ce que prévoyait le §7 du document d'amorce. C'est un argument de plus pour que la file soit construite à l'étape 3 et non plus tard.

40.4 Limites à fixer

Aucune n'est technique : ce sont des décisions produit, à arrêter avant l'implémentation.

Images de galerie	10 par fiche ?
Vidéos de présentation	2 par fiche, 60 secondes chacune ?
Résolution servie	1080p suffit pour un téléphone ; conserver l'original ?
Poids après transcodage	viser quelques mégaoctets, pas quelques dizaines

Le disque local va montrer ses limites. Les médias de fiches sont publics, consultés en boucle et conservés indéfiniment — à la différence des médias de conversation. C'est le premier usage qui justifiera un stockage objet et un cache en frontal. Le disque local tiendra le temps de la mise en service, pas au-delà.

CHAPITRE 41

Impact fichier par fichier

41.1 Backend

Fichier	Nature	Objet
migrations/026_business.sql	nouveau	Profil, catégories, liens, mots-clés, horaires, produits, vues
src/controllers/businessController.js	nouveau	Fiche, liens, médias, mots-clés
src/controllers/businessHoursController.js	nouveau	Créneaux et exceptions
src/controllers/businessProductController.js	nouveau	Catalogue
src/controllers/directoryController.js	nouveau	Annuaire, proximité, filtres
src/utils/openingHours.js	nouveau	Évaluation « ouvert maintenant », fuseau compris
src/routes/business.js	nouveau	/api/business, /api/directory
server.js	modifié	Montage des routes

41.2 Mobile

Fichier	Objet
lib/screens/business/business_profile_screen.dart (nouveau)	Écran de profil d'un visiteur
lib/screens/business/business_edit_screen.dart (nouveau)	Écran d'édition par le propriétaire
lib/screens/business/hours_editor_screen.dart (nouveau)	À budgéter à part
lib/screens/business/catalog_screen.dart (nouveau)	Carte et fiche produit
lib/screens/business/directory_screen.dart (nouveau)	Annuaire et proximité
lib/screens/business/dashboard_screen.dart (nouveau)	Au lieu de la fiche
lib/core/db/app_database.dart	Cache local du profil et du catalogue (Drift v16 → v17)

Réutiliser ce qui existe. geolocator et flutter_map sont déjà dans pubspec.yaml — ils servent au partage de position dans les conversations. L'annuaire n'ajoute aucune dépendance. Idem pour cached_network_image sur les médias du catalogue.

CHAPITRE 42

Points de vigilance et questions ouvertes

42.1 Chantiers d'infrastructure nommés

1. **L'index spatial** — aucune donnée géographique n'existe aujourd'hui. Colonne NOT NULL, SRID explicite, et une position par défaut pour les commerces sans coordonnées.
2. **Le journal de vues** — anti-rebond, agrégation, purge. À concevoir avec la table, pas après.
3. **Le transcodage vidéo** — deuxième client de la file de jobs de l'étape 3. Sans lui, les vidéos de présentation sont inutilisables sur un réseau mobile.
4. **Le stockage des médias** — galerie, vidéos et catalogue multiplient les fichiers, publics et servis en boucle. Le disque local avec `multer` (`src/middleware/upload.js`) tiendra le temps de la mise en service ; c'est le premier candidat au stockage objet.

42.2 Questions ouvertes

1. La liste des 20 à 30 catégories reste à arrêter. Elle est difficile à changer ensuite : elle mérite une revue dédiée.
2. Un compte business a-t-il des réglages de confidentialité différents ? Sa fiche est publique par nature — le « vu à » et le « en ligne » ont-ils encore un sens ?
3. Quelles limites : nombre de produits, de médias, de mots-clés ? Sans plafond, une fiche peut devenir une charge à elle seule.
4. Un commerce peut-il être signalé ? Une arnaque commerciale n'est pas le même traitement qu'un signalement d'utilisateur, et rien n'existe aujourd'hui.
5. Que devient le catalogue au retour en compte personnel ? Le champ `is_hidden` le conserve — confirmer que c'est bien le comportement voulu.

VOLET 5

Certification

Dossiers, coffre à documents, abonnement laissé dormant

CHAPITRE 43

Modèle de données

43.1 Ce que le socle a déjà posé

Rien à recréer. La migration 023 porte déjà sur `users` : `verification_status` (six états), `verified_until`, `verification_reminder_sent`, et l'index `idx_users_verified_until`. Le `verificationScheduler` de l'étape 1 assure déjà la relance à J-30 et le passage en *expiré* à l'échéance.

Ce document n'ajoute que le **dossier** et ses pièces — plus les tables commerciales, laissées inactives.

43.2 Migration 027_verification.sql

```
1  -- 027_verification.sql -- dossiers de verification
2  -- Requierit 023 (socle de compte).
3
4  CREATE TABLE IF NOT EXISTS verification_request (
5      id                BIGINT    NOT NULL AUTO_INCREMENT,
6      alanyaID          INT       NOT NULL,
7      target_type       TINYINT   NOT NULL DEFAULT 1
8                          COMMENT 'Genre vise : 0=personnel notable
9                                  1=business',
9      claimed_name      VARCHAR(160) NULL COMMENT 'Raison sociale telle que
10      declaree',
10     status            TINYINT   NOT NULL DEFAULT 0
11                          COMMENT '0=en attente 1=pièce demandée 2=approuvé
12                                  3=refuse 4=annule',
12     reviewed_by       INT       NULL,
13     decided_at        DATETIME  NULL,
14     reason            VARCHAR(255) NULL COMMENT 'Obligatoire si status = 3',
15     created_at        DATETIME  NOT NULL DEFAULT CURRENT_TIMESTAMP,
16     PRIMARY KEY (id),
17     KEY idx_vr_file (status, created_at),
18     KEY idx_vr_user (alanyaID, created_at),
19     CONSTRAINT fk_vr_user FOREIGN KEY (alanyaID)
20         REFERENCES users(alanyaID) ON DELETE CASCADE,
21     CONSTRAINT fk_vr_reviewer FOREIGN KEY (reviewed_by)
22         REFERENCES users(alanyaID) ON DELETE SET NULL
23 ) ENGINE=InnoDB DEFAULT CHARSET=utf8mb4 COLLATE=utf8mb4_unicode_ci;
24
25 CREATE TABLE IF NOT EXISTS verification_document (
26     id                BIGINT    NOT NULL AUTO_INCREMENT,
27     request_id        BIGINT    NOT NULL,
28     doc_type          TINYINT   NOT NULL
29                          COMMENT '0=registre commerce 1=pièce identité
30                                  2=adresse 3=notoriété',
```

```

30  storage_key  VARCHAR(255) NOT NULL COMMENT 'Chemin du fichier
      chiffre',
31  mime        VARCHAR(80)  NOT NULL,
32  size_bytes  BIGINT       NOT NULL,
33  -- Echeance portee par la ligne : plus simple qu'un script de
      retention
34  -- separe, et lisible par quiconque ouvre la table.
35  purge_after DATETIME     NULL COMMENT 'Renseigne a la decision :
      +90 jours',
36  purged_at   DATETIME     NULL,
37  uploaded_at DATETIME     NOT NULL DEFAULT CURRENT_TIMESTAMP,
38  PRIMARY KEY (id),
39  KEY idx_vd_purge (purge_after, purged_at),
40  CONSTRAINT fk_vd_request FOREIGN KEY (request_id)
41  REFERENCES verification_request(id) ON DELETE CASCADE
42 ) ENGINE=InnoDB DEFAULT CHARSET=utf8mb4;
43
44 -- Journal d'accès : chaque ouverture d'une piece est tracee.
45 CREATE TABLE IF NOT EXISTS verification_document_access (
46  id          BIGINT NOT NULL AUTO_INCREMENT,
47  document_id BIGINT NOT NULL,
48  admin_id    INT    NOT NULL,
49  accessed_at DATETIME NOT NULL DEFAULT CURRENT_TIMESTAMP,
50  ip          VARCHAR(45) NULL,
51  PRIMARY KEY (id),
52  KEY idx_vda_doc (document_id, accessed_at),
53  CONSTRAINT fk_vda_doc FOREIGN KEY (document_id)
54  REFERENCES verification_document(id) ON DELETE CASCADE,
55  CONSTRAINT fk_vda_admin FOREIGN KEY (admin_id)
56  REFERENCES users(alanyaID)
57 ) ENGINE=InnoDB DEFAULT CHARSET=utf8mb4;

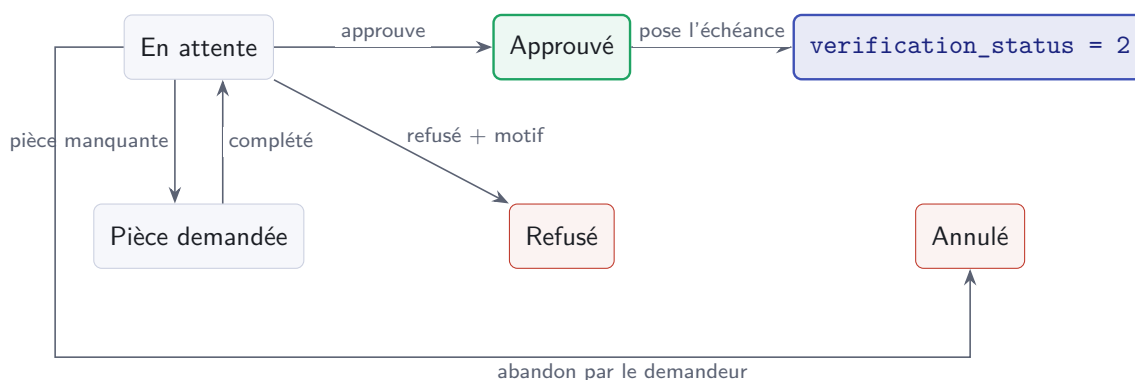
```

Listing 43.1 – Dossiers et pièces

Pourquoi purge_after sur la ligne. Porter l'échéance dans l'enregistrement lui-même évite un script de rétention séparé, qui serait la première chose oubliée lors d'un déploiement. Le même ordonnanceur que l'expiration balaie `purge_after <= NOW() AND purged_at IS NULL`.

Le journal d'accès n'est pas une option. Des pièces d'identité consultables par n'importe quel administrateur sans trace, c'est le genre de détail qui ne se remarque qu'après un incident. La table pèse peu et se purge avec le reste.

43.3 La machine à états du dossier



Un seul dossier ouvert à la fois. Un compte ne peut avoir qu'une demande en *attente* ou *pièce demandée*. Sans cette règle, un demandeur impatient en dépose trois et trois administrateurs les instruisent en parallèle. Contrainte applicative — une clé unique partielle n'existe pas en MySQL.

CHAPITRE 44

Sécurité des pièces

44.1 Chiffrement au repos

Les pièces sont des **données personnelles sensibles**. Trois exigences :

Chiffrées au repos	Chiffrement par fichier, clé hors de la base. Un accès en lecture au disque ne doit pas suffire.
Jamais servies en statique	<code>server.js</code> sert <code>/uploads</code> en statique (<code>express.static</code>). Les pièces ne doivent pas y vivre : elles passent par un point d'entrée authentifié qui déchiffre à la volée.
Journalisées	Chaque ouverture écrit une ligne avec l'administrateur et l'horodatage.

Le piège du dossier uploads. `src/middleware/upload.js` range tout sous `uploads/` et `server.js` expose ce dossier en statique. Déposer une pièce d'identité par le chemin habituel la rendrait **publiquement téléchargeable** pour qui connaît l'URL. Il faut un stockage distinct, hors de l'arborescence servie.

44.2 La purge

Un balayage quotidien, sur le patron du `verificationScheduler` :

```
SELECT id, storage_key FROM verification_document
WHERE purge_after IS NOT NULL
      AND purge_after <= NOW()
      AND purged_at IS NULL
LIMIT 200;
```

Le fichier est effacé du stockage, puis `purged_at` est renseigné. La ligne survit — elle atteste qu'une pièce a existé et qu'elle a été détruite, ce qui est précisément ce qu'on veut pouvoir montrer.

CHAPITRE 45

Abonnement et paiement — conçus, dormants

45.1 Pourquoi les concevoir maintenant

Parce que la version 1 est gratuite, ces tables ne servent à rien aujourd’hui. On les décrit tout de même, pour deux raisons : leur forme conditionne des choix qu’on fait dès à présent (le déblocage de fonctionnalités), et les ajouter plus tard sur une base peuplée coûte plus cher que de les prévoir vides.

```
1 CREATE TABLE IF NOT EXISTS subscription (
2   id                BIGINT    NOT NULL AUTO_INCREMENT,
3   alanyaID          INT       NOT NULL,
4   plan              TINYINT   NOT NULL DEFAULT 0,
5   status            TINYINT   NOT NULL DEFAULT 0
6   COMMENT '0=inactive 1=active 2=late 3=resilient',
7   current_period_end DATETIME NULL,
8   reminder_sent     TINYINT   NOT NULL DEFAULT 0,
9   created_at        DATETIME  NOT NULL DEFAULT CURRENT_TIMESTAMP,
10  PRIMARY KEY (id),
11  KEY idx_sub_user (alanyaID, status),
12  KEY idx_sub_echeance (current_period_end),
13  CONSTRAINT fk_sub_user FOREIGN KEY (alanyaID)
14    REFERENCES users(alanyaID) ON DELETE CASCADE
15 ) ENGINE=InnoDB DEFAULT CHARSET=utf8mb4;
16
17 CREATE TABLE IF NOT EXISTS payment_transaction (
18   id                BIGINT    NOT NULL AUTO_INCREMENT,
19   subscription_id    BIGINT    NOT NULL,
20   provider_ref       VARCHAR(120) NULL,
21   -- Le fournisseur jouera ses webhooks. Sans cette contrainte
22   -- unique,
23   -- double credit garanti.
24   idempotency_key    VARCHAR(120) NOT NULL,
25   amount_minor       BIGINT    NOT NULL,
26   currency           CHAR(3)    NOT NULL DEFAULT 'XAF',
27   status            TINYINT   NOT NULL DEFAULT 0,
28   created_at        DATETIME  NOT NULL DEFAULT CURRENT_TIMESTAMP,
29  PRIMARY KEY (id),
30  UNIQUE KEY uq_pt_idempotence (idempotency_key),
31  KEY idx_pt_sub (subscription_id, created_at),
32  CONSTRAINT fk_pt_sub FOREIGN KEY (subscription_id)
33    REFERENCES subscription(id) ON DELETE CASCADE
34 ) ENGINE=InnoDB DEFAULT CHARSET=utf8mb4;
```

Listing 45.1 – 027_verification.sql — tables inactives en v1

45.2 Renouvellement manuel, pas prélèvement

`subscription` est modélisée pour un renouvellement **manuel** avec relances, et non pour un prélèvement automatique. Sur mobile money, les autorisations expirent et les échecs sont silencieux ; une formule annuelle relancée est plus réaliste qu'un auto-débit qui casse sans prévenir. D'où `reminder_sent`, exactement comme sur la vérification.

45.3 La règle à ne jamais enfreindre

Ce qui débloque une fonctionnalité, c'est l'abonnement. Aucune fonctionnalité ne doit être conditionnée à `verification_status`. Les lieux multiples de l'étape 4, par exemple, se débloquent par `subscription.status = 1`.

Si le badge ouvrait des portes, il deviendrait un produit qui se vend, et le retirer pour défaut de paiement reviendrait à retirer une attestation d'identité à quelqu'un qui est toujours la même personne. C'est le recouplage que le §4 du document d'amorce écarte, et c'est ce qui permet aujourd'hui de livrer la certification gratuitement sans rien fermer.

45.4 Une contrainte de plateforme à anticiper

Un badge payant est un service numérique : l'App Store et le Play Store peuvent exiger un achat in-app, avec leur commission. C'est la raison pour laquelle les offres équivalentes coûtent plus cher sur mobile que sur le web. À trancher **avant** la soumission, pas pendant — et c'est un argument de plus pour que la v1 reste gratuite.

CHAPITRE 46

Impact fichier par fichier

46.1 Backend

Fichier	Nature	Objet
migrations/027_verification.sql	nouveau	Dossiers, pièces, journal, tables dormantes
src/services/documentVault.js	nouveau	Chiffrement, déchiffrement à la volée, purge
src/controllers/verificationController.js	nouveau	Dépôt, suivi, annulation
src/controllers/admin/verification.js	nouveau	File, instruction, décision
src/routes/verification.js	nouveau	/api/verification
src/routes/admin.js	modifié	File et décision
src/services/verificationScheduler.js	modifié	Ajout du balayage de purge
server.js	modifié	Montage des routes

46.2 Administration et mobile

Fichier	Objet
app/(dashboard)/verifications/page.tsx (nouveau)	File des dossiers
app/(dashboard)/verifications/[id]/page.tsx (nouveau)	Instruction
app/(dashboard)/layout.tsx	Entrée de navigation, avec le compteur en attente
lib/screens/profile/verification_screen.tsx (nouveau)	Dépôt et suivi
lib/screens/profile/settings_screen.tsx	Entrée « Faire vérifier mon compte »

CHAPITRE 47

Points de vigilance et questions ouvertes

47.1 Chantiers nommés

1. **Le coffre à documents** — chiffrement, service authentifié, purge, journal. Rien de cela n'existe ; c'est la brique la plus sensible des cinq étapes.
2. **La sortie de uploads/** — les pièces ne doivent pas transiter par le chemin de téléversement habituel, servi en statique.

47.2 Questions ouvertes

1. **Durée de validité par défaut** : un an est l'usage, mais ce n'est pas tranché. Elle conditionne le rythme des relances et la charge d'instruction.
2. La liste des pièces ci-dessus est une proposition : elle demande une validation métier, notamment sur le RCCM comme pièce obligatoire.
3. Un dossier refusé peut-il être redéposé immédiatement, ou après un délai ? Sans délai, un demandeur peut saturer la file.
4. Que se passe-t-il si un compte vérifié **change de nom** ? L'attestation portait sur une identité ; un changement devrait au minimum alerter.
5. Qui traite une **contestation** après refus ? Aucun circuit n'existe, et c'est précisément à cela que servent les 90 jours de rétention.

VOLET 6

Identité et connexion par QR code

Jeton opaque régénérable, session de connexion éphémère, révocation réelle du refresh token

CHAPITRE 48

Enveloppe de payload et résolution

Tous les QR d'Alanya encodent la même chose : une URL portant un **jeton opaque**, jamais une donnée brute. Ni l'`alanyaID` interne (auto-incrémenté, donc énumérable), ni l'Alanya ID (`alanyaPhone`) — ce dernier est aussi un identifiant de connexion, et l'exposer sur un support scannable par n'importe qui affaiblirait sa fonction actuelle. Trois familles de chemins, reconnues par une seule expression (`_QR_PATH_RE` dans `qrController.js`) :

Chemin encodé	Ce qu'il résout
<code>/q/c/:token</code>	un contact — code éphémère d'un compte personnel, en mémoire (section 51.1)
<code>/q/u:qrPublicId</code>	un contact — code permanent (<code>users.qr_public_id</code>), verrouillé jusqu'aux comptes business (section 49.1)
<code>/q/l:sessionId:scanSecret</code>	une session de connexion en attente, en mémoire (chapitre 52)

Une seule base d'URL, dans un seul fichier. `src/utils/qrToken.js` porte `QR_BASE_URL` (variable d'environnement, par défaut `https://www.alanya237.com`) ainsi que les trois fabricants de payload, `identityPayload`, `contactPayload` et `loginPayload`. Ils vivent ensemble et non dans leurs contrôleurs respectifs parce qu'un seul contrôleur, `qrController`, les *reparse* tous : deux constantes séparées ont déjà divergé une fois, laissant les QR de connexion sur l'ancien domaine. Le même fichier porte aussi `generateOpaqueToken` et `secretMatches`, la comparaison à temps constant des secrets. Le point qui sépare `sessionId` de `scanSecret` n'est jamais ambigu : les deux jetons sont en `base64url`, dont l'alphabet ne contient pas le point.

Les trois types partagent un seul point d'entrée :

POST `/api/qr/resolve` Corps : `{ payload }`, la chaîne **brute** scannée — le client ne l'interprète jamais. Réponse : `{type:'contact', ...}` (chapitre 51) ou `{type:'login', session}`.

Le domaine n'est pas vérifié au parsing. `_parsePayload` n'extrait que le *chemin* de l'URL scannée : l'autorité de résolution est le jeton, pas l'hôte. Un QR déjà imprimé sous un ancien domaine continue donc de fonctionner, et un QR forgé sous un autre domaine ne gagne rien — il faudrait qu'il porte un jeton valide, ce que le contrôle d'autorisation vérifie ensuite.

Le scanneur doit toujours être authentifié. Les deux flux exigent que **qui scanne** soit déjà connecté — pas seulement qui possède un compte. Scanner un contact sans être soi-même connecté n'a pas de sens (à qui l'ajouter ?) ; scanner une session de connexion sans l'être serait la faille elle-même. Le middleware `authCustom` garde donc `/api/qr/resolve` au même titre que le reste de l'API, doublé de `qrResolveLimiter` (30 résolutions par minute et par IP) — une défense en profondeur contre l'énumération des jetons, pas une contrainte fonctionnelle.

CHAPITRE 49

Modèle de données

49.1 Le jeton d'identité permanent, et son verrou

Une colonne sur `users`, indépendante de `alanyaPhone` : régénérable sans effet sur la connexion, sans capacité de connexion elle-même. Elle est posée par la migration 026 ci-dessous, mais **réservée aux futurs comptes business** : le garde `_qrPermanentAutorise` (`qrController.js`) refuse aujourd'hui tout le monde. **GET** `/api/qr/me` et **POST** `/api/qr/me/regenerate` répondent 403 `BUSINESS_ONLY` ; la résolution d'un `/q/u/` et sa page publique répondent « inconnu ou expiré » — y compris pour les `qr_public_id` déjà renseignés par des comptes d'essai antérieurs au verrouillage. Les comptes personnels vivent sur les jetons éphémères de la section 51.1.

users.type_compte n'est pas un type de compte. Le réflexe serait de brancher ce verrou sur `users.type_compte`. Ce champ est en réalité un **rôle d'administration** — 0 utilisateur, 1 admin, 2 super-admin, posé par `PUT /users/:id/role` dans `src/routes/admin.js` — pas un type de compte métier : s'y brancher ouvrirait les codes permanents... aux administrateurs. Rien en base ne sait encore dire « ce compte est un commerce » ; c'est le volet 1 (socle de compte, `account_type`) qui l'apportera, et `_qrPermanentAutorise` s'y branchera alors — le TODO est posé dans le code, à côté du garde.

49.2 La nature de `device_id`

Un identifiant matériel, pas un UUID d'application. `lib/core/services/storage_service.dart`, méthode `getOrCreateDeviceId()`, génère un UUID aléatoire persisté par `flutter_secure_storage` — stable tant que l'app reste installée, mais **réinitialisé à la réinstallation**. Un appareil réinstallé apparaîtrait alors comme un second appareil fantôme dans la liste. Le `device_id` de la table `appareils` est donc un identifiant **matériel**, calculé par `TalkyApiClient._currentHardwareId()` et distinct de l'UUID applicatif, que le push et le socket continuent d'utiliser de leur côté.

Plateforme	Source de <code>device_id</code>
Android	<code>Settings.Secure.ANDROID_ID</code> , lu par un canal natif Kotlin — <code>MethodChannel com.alanya/device_identity</code> , méthode <code>getAndroidId</code> , déclaré dans <code>android/app/src/main/kotlin/com/alanya237/alanya/MainActivity.kt</code> . Unique par (appareil, signature d'app, utilisateur) et survivant à une réinstallation.
iOS	<code>identifierForVendor</code> , via <code>device_info_plus</code> .
Autres	repli direct sur l'UUID applicatif.

Pourquoi un canal natif et non `device_info_plus`. `device_info_plus` n'expose plus `ANDROID_ID` depuis sa **v9** — le paquet reste une dépendance et sert toujours au libellé lisible de l'appareil (`_currentDeviceModel()`) et à l'identifiant `ForVendor` d'iOS, mais il ne peut plus fournir la clé d'appareil sur Android. Le piège à éviter est `AndroidDeviceInfo.id`, qu'on prendrait volontiers pour son remplaçant : ce champ est `Build.ID`, c'est-à-dire l'**empreinte du firmware** : identique sur tous les téléphones d'un même modèle, et modifiée à chaque mise à jour système. Comme l'unicité porte sur le couple (`alanyaID`, `device_id`), deux téléphones du même modèle appartenant au *même* compte se retrouveraient sur une seule ligne — en déconnecter un déconnecterait l'autre — pendant qu'une simple mise à jour ferait réapparaître chaque téléphone comme neuf. Et un identifiant que tout possesseur du même modèle connaît n'est plus un identifiant. Strictement inutilisable comme clé d'appareil ; d'où le canal natif.

La chaîne de repli, jusqu'au bout. Si `ANDROID_ID` est indisponible (exception ou valeur vide), et hors Android/iOS (web, macOS, Windows, Linux), `_currentHardwareId()` retombe sur l'UUID applicatif `StorageService().getOrCreateDeviceId()` — puis, en dernier recours, sur `'INDEFINI'`. Mieux vaut un identifiant par installation qu'aucun : sans `device_id`, aucune ligne `appareils` n'est créée, donc aucun jeton n'est émis (chapitre 50) et la connexion échoue. Le prix de ce repli est la seule imperfection assumée du dispositif : sur ces plateformes, une réinstallation crée une seconde ligne.

Asymétrie de persistance entre plateformes. Le Keychain iOS — et donc potentiellement l'UUID applicatif lui-même — survit généralement à une réinstallation ; les données d'application Android, dont l'UUID applicatif, sont effacées à la désinstallation. `ANDROID_ID` est donc le seul des deux identifiants qui apporte une vraie amélioration côté Android. Côté iOS, `identifierForVendor` ne survit pas à la désinstallation de la *dernière* application du vendeur : le téléphone réapparaît alors comme un nouvel appareil. Limite assumée, et signalée à l'utilisateur dans l'écran « Appareils connectés ».

Conséquence assumée : `appareils.device_id` et `user_push_devices.deviceId` ne sont **pas** directement joignables — ce sont deux identifiants différents. La table `appareils` porte donc un `push_device_id` nullable, réservé à l'affichage optionnel du statut « notifications activées » dans la liste des appareils. **Aucun code ne l'écrit à ce jour** : la colonne est posée pour cet usage futur, et l'écran doit se comporter comme si elle était toujours vide.

49.3 Le jeton et la table `appareils`

```

1  -- Identite QR (users.qr_public_id) + registre des appareils connectes
2  -- (appareils), alimente par toute connexion (login, register, QR).
3  --
4  -- MySQL 8 ne supporte pas ADD COLUMN IF NOT EXISTS (cf. migration
5  -- 008) : si
6  -- relancee apres un premier passage reussi, ignorer les erreurs 1060
7  -- (Duplicate column name) / 1061 (Duplicate key name).
8
9  ALTER TABLE users
10     ADD COLUMN qr_public_id VARCHAR(64) NULL,
11     ADD UNIQUE KEY uq_users_qr_public_id (qr_public_id);
12
13 CREATE TABLE IF NOT EXISTS appareils (
14     id BIGINT AUTO_INCREMENT PRIMARY KEY,
15     alanyaID INT NOT NULL,
16     device_id VARCHAR(128) NOT NULL,
17     push_device_id VARCHAR(128) NULL,
18     device_name VARCHAR(120) NULL,

```

```

18  -- VARCHAR et non ENUM : le client envoie aussi macOS / Windows /
    Linux, et
19  -- ajouter une plateforme ne doit pas demander un ALTER TABLE.
20  platform VARCHAR(20) NOT NULL DEFAULT 'unknown',
21  ip_address VARCHAR(64) NULL,
22  login_method ENUM('password','register','qr') NOT NULL,
23  created_at DATETIME NOT NULL DEFAULT CURRENT_TIMESTAMP,
24  last_active_at DATETIME NOT NULL DEFAULT CURRENT_TIMESTAMP,
25  revoked_at DATETIME NULL,
26  UNIQUE KEY uq_appareils_user_device (alanyaID, device_id),
27  KEY idx_appareils_user (alanyaID, revoked_at),
28  CONSTRAINT fk_appareils_user FOREIGN KEY (alanyaID)
29    REFERENCES users(alanyaID) ON DELETE CASCADE
30 ) ENGINE=InnoDB DEFAULT CHARSET=utf8mb4;

```

Listing 49.1 – migrations/026_qr_appareils.sql

Ordre de déploiement — la migration d’abord, le code ensuite. Les trois gardes d’authentification (`src/middleware/auth.js`, `src/middleware/authCustom.js`, `src/socket/handlers/auth.js`) joignent appareils à **chaque** requête authentifiée. Déployer le code sans la table, c’est un `ER_NO_SUCH_TABLE` sur toute l’API — messages, appels et `/auth/me` compris, pas seulement le QR. Le dépôt n’a pas de *runner* de migrations : elles s’appliquent à la main, c’est donc à vérifier explicitement au déploiement. L’avertissement figure en tête du fichier SQL.

Pourquoi ni `user_push_devices` ni `userAccess`. Deux autres tables évoquent déjà « appareil », chacune pour une raison différente. `user_push_devices` (migration 020) a une portée strictement push — heartbeat, `appState`, jeton FCM : des écritures fréquentes et peu sensibles. `userAccess` est un journal en écriture seule, jamais muté, consultable seulement par un admin. Aucune des deux n’est un état de session vivant et révocable. Mélanger `revoked_at` — un champ sensible, rarement écrit — avec le flux à haute fréquence de `user_push_devices` aurait été le genre de raccourci qui coûte cher plus tard.

platform en VARCHAR(20), login_method en ENUM. Les deux colonnes sont un vocabulaire fermé ; une seule est déclarée comme tel, et l’écart est délibéré. `platform` est **ouvert dans le temps** : le client envoie déjà `android`, `ios`, `web`, `macos`, `windows`, `linux` et `unknown`, et une plateforme de plus ne doit pas coûter un `ALTER TABLE` sur une table de session — d’où le `VARCHAR(20)`, sur le modèle de `user_push_devices.platform`. `login_method`, lui, est **fermé par nature** : on ne se connecte que par mot de passe, par inscription ou par QR, et un quatrième moyen serait de toute façon un chantier, pas une valeur de plus. La validation n’est donc pas laissée à la seule base : `deviceSessionService` normalise `platform` contre une liste blanche (`_normalizePlatform`, repli sur `'unknown'`) et lève sur un `login_method` inconnu, avant l’insertion.

49.4 L’origine d’un contact

```

1  -- Origine d'un contact prefere -- 'search' (recherche manuelle) ou
    'qr'
2  -- (scan ou lien) -- et note contextuelle optionnelle saisie juste
    apres un
3  -- scan. Alimentent la pastille, les filtres << Par QR >> et la fiche.
4
5  ALTER TABLE preferredContact
6    ADD COLUMN added_via VARCHAR(16) NOT NULL DEFAULT 'search',
7    ADD COLUMN added_note VARCHAR(200) NULL;

```

Listing 49.2 – migrations/027_pref_contact_added_via.sql

VARCHAR et non ENUM, même leçon que `appareils.platform` ci-dessus : le vocabulaire est ouvert dans le temps (un partage de fiche, un import de répertoire ajouteraient chacun leur origine), et une valeur de plus ne doit pas coûter un ALTER TABLE. Le DEFAULT 'search' requalifie d'un coup tout l'existant — exact, puisque la recherche était jusqu'ici le seul chemin d'ajout.

`added_note` est la note de contexte, posée *après* l'ajout par **PUT** `/api/contacts/:id/note` (corps {note}, 200 caractères, vide = effacement, 404 si la personne n'est pas un contact préféré). Elle vit sur la ligne de la relation — `alanyaID` → `idFriend` — et n'est donc jamais visible de l'intéressé ; GET `/contacts` la restitue en `addedNote`, le client Flutter la reflète en local (`local_users.preferredNote`, schéma v17) et la fiche l'affiche en italique sous la mention d'origine.

Migration 027 avant le code, comme la 026. `contactService` *nomme* la colonne dans son INSERT et ses SELECT : déployer le code sans la migration casse l'ajout de contact — REST comme QR. Même discipline que pour la 026 : appliquer d'abord, déployer ensuite. L'avertissement figure en tête du fichier SQL.

Pas de table pour la session éphémère. La session de connexion (le pont entre le nouvel appareil et celui qui l'autorise) ne vit **pas** dans une table SQL — voir la justification au chapitre 52, une fois son mécanisme introduit. Le jeton de contact éphémère (section 51.1) suit le même régime.

CHAPITRE 50

Rendre la révocation réelle

Le point de départ, avant ce volet. L'authentification était **100 % sans état** : un jeton d'accès (15 min) et un jeton de rafraîchissement (30 jours, secret distinct), tous deux de simples JWT signés. Aucune table **sessions** ni **tokens** n'existait, et `POST /api/auth/refresh` (`authCustomController.js`) se contentait de vérifier la signature pour réémettre une nouvelle paire — sans jamais rien invalider. Un jeton de rafraîchissement qui fuyait restait valide 30 jours, quoi qu'il arrive. Ce chapitre est la réponse à ce constat : la même route refuse désormais de renouveler quoi que ce soit si l'`appareilId` du jeton désigne une ligne révoquée.

Chiffré. Sur 78 comptes en production, **69** ont encore `users.device_ID = 'INDEFINI'` (la valeur par défaut de la colonne) et seulement **9** portent un vrai identifiant — hérité non pas du login, mais d'un effet de bord de `pushDevicesController.js::registerPushDevice`, qui écrase par rétrocompatibilité l'ancienne colonne mono-appareil à chaque enregistrement push réussi. Ce chemin ne couvre ni les comptes qui refusent les notifications, ni le moment précis de la connexion. La table `appareils` ne peut donc pas être un sous-produit d'un autre flux : elle doit être alimentée **directement** par `register`, `login` et l'approbation d'une session QR.

Table dédiée, pas une extension de `user_push_devices`. Réutiliser `user_push_devices` pour porter aussi la session d'authentification a été envisagé, puis écarté : les deux concepts ont des cycles de vie incompatibles sur une même ligne. Un enregistrement push doit pouvoir survivre à la révocation d'une session — c'est précisément ce qui permet de notifier un appareil qu'il vient d'être déconnecté. Les fusionner aurait rendu cette notification impossible au moment même où elle est la plus utile.

Le JWT est étendu d'un champ `appareilId` :

```
1 const generateAccessToken = (payload) =>
2   jwt.sign({ ...payload, type: 'access' }, JWT_SECRET, { expiresIn:
3     '15m' });
4   // payload devient { alanyaID, email, appareilId }
```

Listing 50.1 – `src/middleware/authCustom.js` — extension du payload

La vérification de `appareils.revoked_at IS NULL` se fait dans les gardes d'authentification, à **chaque** requête — pas seulement au rafraîchissement. Elles sont **trois**, toutes sous `src/middleware/authCustom.js`, `src/middleware/auth.js` et `src/socket/handlers/auth.js`. Oublier la troisième laisserait un appareil révoqué en 401 sur tout le REST mais parfaitement vivant en temps réel : il continuerait de recevoir les messages du compte.

```
1 SELECT u.alanyaID, u.alanyaPhone, u.email
2 FROM users u
3 LEFT JOIN appareils a ON a.id = ? AND a.alanyaID = u.alanyaID
4 WHERE u.alanyaID = ? AND u.exclus = 0
5 AND (a.id IS NULL OR a.revoked_at IS NULL);
```

Listing 50.2 – La jointure commune aux trois gardes

Pourquoi une jointure *externe*, et ce qu'elle concède. Le `LEFT JOIN` et la clause `a.id IS NULL` laissent passer les jetons émis **avant** la migration 026, qui ne portent pas d'`appareilId` — sans quoi le déploiement déconnecterait tout le monde d'un coup. La contrepartie est explicite : la révocation n'est réelle que pour les sessions ouvertes *après* la migration. Les anciennes s'éteignent d'elles-mêmes à l'expiration de leur jeton de rafraîchissement, au plus tard 30 jours après.

Le coût assumé. Cela ajoute un `SELECT` indexé (`idx_appareils_user`) sur le chemin le plus chaud de l'application — en pratique, une jointure de plus sur une requête déjà faite. Alternative plus légère : ne vérifier qu'au rafraîchissement, avec un délai de grâce allant jusqu'à 15 minutes. Choix retenu : vérifier à chaque requête. Un bouton « Déconnecter » qui met jusqu'à un quart d'heure à faire effet se comporterait, du point de vue de l'utilisateur, comme un bouton cassé.

50.1 Ce que fait vraiment la révocation

`DELETE /api/auth/device-sessions/:id` pose `revoked_at = NOW()`, puis enchaîne deux gestes de nature très différente :

1. **Un signal de confort** — l'événement `auth:device_revoked` part vers *tout* le compte, dans la room `user_${alanyaID}`, avec l'**identifiant matériel** de l'appareil coupé. Chaque client compare cette valeur à son propre `currentHardwareId()` : seul l'appareil visé se reconnaît, et il efface alors sa session locale — stockage sécurisé compris — au lieu d'attendre son prochain 401. Diffuser à tout le compte plutôt que cibler une socket est volontaire : l'appareil révoqué peut très bien ne pas avoir de socket ouverte à cet instant, et son identifiant matériel ne dit rien d'exploitable aux autres appareils du *même* compte.
2. **La garantie dure** — le 401 des trois gardes. Elle ne dépend d'aucune coopération du client. `disconnectAppareilSockets` (`src/utils/userSocketRegistry.js`) ferme en outre les sockets encore ouvertes de cet appareil, *après* l'`emit` pour que l'événement parte avant la fermeture.

La clé de fermeture des sockets est `appareilId`, pas `device_id`. `disconnectAppareilSockets` filtre sur `socket.appareilId` — l'identifiant de *ligne*, porté par le JWT et posé sur la socket à son authentification. Filtrer sur un `socket.deviceId` ne marcherait pas : cette propriété-là porte l'**UUID applicatif** du push, qui n'a aucun rapport avec `appareils.device_id` (section 49.2). Les deux champs se ressemblent assez pour qu'on les confonde, et l'erreur serait silencieuse : aucune socket ne correspondrait, aucune exception ne serait levée, et la fermeture ne ferait simplement rien.

Un appareil révoqué ne se *réactive* jamais. Quand un appareil précédemment révoqué se reconnecte, `deviceSessionService.recordLogin` ne remet pas `revoked_at` à `NULL` : il **supprime** la ligne morte et en insère une neuve. La raison tient à l'`id`. Réactiver la ligne conserverait son identifiant, donc l'`appareilId` porté par les anciens jetons : le jeton de rafraîchissement de 30 jours qu'on venait précisément de couper redeviendrait valide, et l'appareil volé retrouverait l'accès sans rien faire d'autre qu'attendre une reconnexion légitime du propriétaire. Un `id` neuf ne figure dans aucun jeton déjà émis.

Pas de ligne `appareils`, pas de jeton. `register` et `login` exigent un identifiant d'appareil (`hardware_id`, à défaut `device_ID`) et refusent la valeur sentinelle '`INDEFINI`' : sans lui, c'est un 400. Et si `recordLogin` échoue malgré tout, ils répondent **503** sans émettre le moindre jeton, plutôt que de laisser passer une connexion. Une session sans ligne `appareils` serait en effet invisible dans « Appareils connectés » et irrévocable — exactement ce que ce chapitre entier cherche à éliminer. Mieux vaut une connexion qui échoue franchement qu'une session fantôme. Le même refus s'applique à l'approbation d'une session QR.

CHAPITRE 51

Ajout de contact instantané

`addPreferredContact` (`src/controllers/preferredContactController.js`) faisait déjà exactement ce qu'il faut : rejet si l'utilisateur se scanne lui-même, vérification que la cible existe et n'est pas exclue, vérification de non-blocage, vérification de non-appartenance déjà acquise, puis INSERT. Cette logique est extraite en service — `src/services/contactService.js`, fonction `addContactByFriendId(alanyaID, friendID, {addedVia})` — partagé par le contrôleur REST existant **et** par le résolveur QR, sans dupliquer les règles. L'option `addedVia` ('search' par défaut, 'qr' pour les deux chemins QR) alimente la colonne de la migration 027, et `_contactBody` restitue `addedVia` et `addedAt` dans chaque réponse — la ligne telle qu'elle s'affichera. Le même fichier exporte `sanitizeUrl`, le filtre des sentinelles historiques de `users.avatar_url` ('NON DEFINI', 'null'...), pour la même raison : recopié, il diverge et l'avatar s'affiche différemment d'un écran à l'autre.

L'origine restituée est celle de l'ajout initial. Sur `already`, le service relit `added_via` et `created_at` de la ligne *existante* : rescanter quelqu'un qu'on avait ajouté par recherche ne réécrit pas l'histoire, et la mention datée de la fiche reste celle du premier geste.

Le service ne connaît aucun code HTTP. `addContactByFriendId` renvoie un résultat discriminé — `{ok, reason, contact}` avec `reason` ∈ `added, self, not_found, blocked, already` — et laisse chaque appelant choisir sa sémantique. C'est indispensable ici, parce que les deux appelants *divergent volontairement* sur deux cas : là où la route REST POST `/api/contacts/:id` répond 400 (soi-même) et 409 (déjà contact), la résolution d'un QR répond 200 dans les deux cas. Un service qui aurait choisi les codes à leur place aurait forcé le QR à mentir sur ce qui s'est passé.

51.1 Le jeton éphémère, en mémoire

Le code que l'écran « Mon code » affiche pour un compte personnel est un jeton de **dix minutes**, à **usage unique**, qui ne touche jamais la base : nouveau fichier `src/socket/state/qrContactTokens.js`, calque assumé de `qrLoginSessions.js` (chapitre 52, où le choix « une Map en mémoire, pas une table » est justifié — mono-processus compris).

Structure	Map token → {token, alanyaID, createdAt, expiresAt}, jeton <code>generateOpaqueToken(16)</code>
Durée de vie	TTL_MS = 10 min, renvoyée au client en <code>ttlMs</code>
Un seul jeton actif	index alanyaID → jeton ; <code>create()</code> invalide le précédent du même compte
Usage unique	<code>consume()</code> = lecture + suppression ; <code>get()</code> reste non destructif
Nettoyage	expiration paresseuse, balayage d'un échantillon à chaque création (<code>SWEEP_PER_CREATE = 20</code>), éviction FIFO au-delà de <code>MAX_TOKENS = 10 000</code>

La création est autolimitante. Un seul jeton vivant par utilisateur : en générer un nouveau tue le précédent. Marteler **POST** `/api/qr/contact-token` ne fait donc jamais grossir la `Map` — c’est pourquoi la route, déjà authentifiée, n’a pas de limiteur dédié. Le balayage et le plafond restent, même filet que `qrLoginSessions` : une `Map` bornée, pas seulement expirante.

Testé avec la session de connexion. `qrContactTokens.test.js` (création, invalidation du précédent, consommation unique, expiration paresseuse) est chaîné à `qrLoginSessions.test.js` dans le script `npm run test:qr`.

51.2 Résolution et consommation

`_resolveContactEphemere` (`qrController.js`) lit le jeton *sans* le consommer, applique `addContactByFriendId` avec `{addedVia: 'qr'}`, puis décide du sort du jeton selon l’issue :

Réponse de POST <code>/api/qr/resolve</code>	Cas — sort du jeton
<code>{type: 'contact', added: true, alreadyContact: false, contact}</code>	ajouté — jeton consommé
<code>{type: 'contact', added: false, alreadyContact: true, contact}</code>	déjà favori — succès doux, aucune écriture, jeton consommé quand même
200 <code>{type: 'contact', self: true}</code>	code correspondant à soi-même — le jeton survit
403	bloqué, dans un sens ou dans l’autre — le jeton survit
404	jeton inconnu, expiré, déjà utilisé, ou compte exclu

Qui a le droit de tuer un code. La règle produit « scanné = expiré » est lue littéralement : le jeton est consommé dès qu’il a rempli son office — contact ajouté, *ou* déjà présent (le scan a bien « réussi » du point de vue de qui scanne). Il survit en revanche à un scan de son propre code et à un scan par un compte bloqué : ni une maladresse du propriétaire, ni un bloqué, ne doivent pouvoir tuer le code d’autrui. Et la consommation vient **après** l’écriture, jamais avant : consommer d’abord laisserait un **INSERT** en échec détruire un code encore valide, pour rien.

Se scanner soi-même est un 200, pas un 400. La partie I range ce cas parmi les **succès doux** : « rien ne se passe, un message discret l’indique ». Une erreur 400 obligerait le client à distinguer, dans une même branche d’échec, le code illisible (400 lui aussi) de son propre code — et le moindre raccourci d’affichage transformerait un geste anodin en message d’erreur rouge. Le serveur répond donc 200 avec un drapeau `self`, que le client lit explicitement (`isSelf` dans `lib/models/qr_models.dart`). Même logique pour un contact déjà enregistré : la carte de résultat s’affiche, sans nouvelle écriture. Le champ s’appelle bien `contact` et non `user` : c’est la ligne `preferredContact` telle qu’elle apparaîtra dans la liste, `idPrefContact`, `addedVia` et `addedAt` compris, pas la fiche de l’utilisateur.

La page publique ne consomme jamais. `GET /q/c/:token` (`src/controllers/qrLandingController.js`, monté hors `/api` par `src/routes/qrLanding.js`) sert la page qu’un appareil photo générique ouvre en scannant le code : identité du propriétaire, bouton « Ouvrir dans Alanya » en schéma applicatif, mention « valable une dizaine de minutes et pour une seule personne ». Elle n’est pas authentifiée, ne sait pas qui la consulte, et **ne consomme pas** le jeton : seule la résolution par un utilisateur connecté dans l’app le fait. La page du code permanent (`GET /q/u/:token`) suit le verrou business : tant qu’il est fermé, elle répond « Ce code n’est plus valide ».

51.3 Prévenir le propriétaire

Après consommation, `_prevenirProprietaire` avertit le compte dont le code vient d'être utilisé — c'est ce qui alimente le dialogue « ajouter en retour » de la partie I. Deux canaux, jamais les deux à la fois :

1. l'événement socket `qr:contact_scanned`, vers la room `user_${alanyaID}` du propriétaire : `{by: {alanyaID, nom, pseudo, avatar_url}, alreadyMutual, at}`. `alreadyMutual` dit si le propriétaire a *déjà* le scanneur en contact — le client affiche alors une simple information au lieu du dialogue oui/non ;
2. la notification push `qr_contact_scanned` (`notificationService.notifyQrContactScanned`), **seulement** si `hasForegroundSocket` (`src/utils/userSocketRegistry.js`) ne trouve aucune socket du compte déclarée au premier plan : app visible, le dialogue in-app suffit et la push ferait doublon ; app en arrière-plan ou fermée, la push prend le relais.

Le tout est *best-effort* : l'appel n'est pas attendu et ses erreurs sont journalisées sans jamais faire échouer l'ajout — le scanneur a son contact, prévenir le propriétaire est un bonus, pas une condition.

51.4 L'ajout en retour

Accepter le dialogue rejoue simplement la route existante, **POST** `/api/contacts/:id`, avec un corps `{addedVia: 'qr'}`. Le contrôleur n'accepte que cette valeur en **liste blanche** — tout le reste retombe sur `'search'` : cette métadonnée est cosmétique (pastille, filtre, mention datée), pas de la sécurité, mais on ne laisse pas le client inventer des origines. Les deux directions du lien portent ainsi la même origine, et **GET** `/api/contacts` restitue `addedVia` et `addedAt` pour toute la liste.

CHAPITRE 52

La session de connexion éphémère

52.1 Une structure en mémoire, pas une table

Vérifié dans le déploiement. Alanya-Backend tourne en **un seul processus Node** — `npm start` lance `node server.js`, le déploiement fait `pm2 restart alanya-backend` sans mode cluster, et aucun `ecosystem.config.js` n'existe. Un état partagé uniquement en mémoire est donc visible de toutes les requêtes, sans risque qu'une requête tombe sur un autre processus qui ignore la session.

La session de connexion vit dans une `Map` en mémoire, sur le modèle exact de `src/socket/state/pendingCalls.js` : `TTL_MS`, `createdAt` / `expiresAt`, et un nettoyage porté par la structure elle-même plutôt que par un job externe. Nouveau fichier : `src/socket/state/qrLoginSessions.js` — dont le jeton de contact éphémère (section 51.1) est le calque.

Ce que ça évite. Ni migration, ni colonne, ni politique de rétention à trancher (une ligne de table qui traînerait des semaines n'existe plus, par construction — l'entrée disparaît de la `Map` à l'expiration). Ce que ça coûte, en échange : une session en cours ne survit pas à un redémarrage du serveur — l'utilisateur relance simplement le QR, un cas déjà géré par la régénération automatique ci-dessous.

Une `Map` bornée, pas seulement expirante. L'expiration est *paresseuse* : pas de `setInterval`, une entrée périmée disparaît à la première lecture qui la rencontre. Cela suffit tant qu'on lit ; or la route de création n'est pas authentifiée, et son `authLimiter` a `skipSuccessfulRequests` : les créations *réussies* ne consomment aucun quota. Un client qui créerait sans jamais interroger ferait donc croître la `Map` indéfiniment, dans le processus qui héberge aussi tous les sockets. Chaque création balaie donc un échantillon d'entrées (`SWEEP_PER_CREATE`) et évince en FIFO au-delà de `MAX_SESSIONS` — une `Map` JavaScript itère dans l'ordre d'insertion, l'éviction est donc gratuite.

52.2 Deux secrets par session, jamais interchangeables

C'est la pièce la moins évidente du dispositif, et celle dont dépend tout le reste : **le QR ne suffit pas à récupérer les jetons.**

Secret	Dans le QR ?	À qui, et pour quoi
<code>scanSecret</code>	oui	À l'appareil qui scanne : il prouve qu'on a vu le code. Exigé par <code>/resolve</code> , <code>/approve</code> et <code>/deny</code> .
<code>pollToken</code>	jamais	À l'appareil qui a créé la session, et à lui seul — remis une unique fois, dans la réponse à la création. Exigé pour interroger le statut et retirer les jetons.

Sans cette séparation, photographier le QR suffirait. Avec un secret unique, quiconque prend en photo l'écran du nouvel appareil pourrait interroger la session en boucle et **récupérer les jetons à la place du demandeur**, à l'instant précis où l'utilisateur approuve — l'écran de confirmation, lui, aurait montré un appareil parfaitement légitime. Le `pollToken` coupe cette attaque à la racine : il n'a jamais été affiché nulle part.

En-tête X-Poll-Token, et non paramètre d'URL. GET `/api/auth/qr-session/:sessionId` lit le `pollToken` dans un **en-tête** HTTP. Une *query string* serait fonctionnellement identique et discrètement désastreuse : nginx et Cloudflare journalisent l'URL complète par défaut, et la réponse à cette requête précise transporte `accessToken` et `refreshToken`. Le `sessionId` étant public — il est dans le QR — le `pollToken` est le *seul* facteur : il ne doit pas finir en clair dans un journal d'accès.

Livraison des jetons à usage unique. Dès que l'interrogation renvoie `approved` avec la paire de jetons, l'entrée est **effacée** de la Map (`clear`). Les jetons ne repassent jamais par ce canal : un second appel, avec le même `pollToken`, ne trouve plus rien. Une session inconnue et un `pollToken` faux donnent d'ailleurs la même réponse — `{status:'expired'}` — pour qu'on ne puisse pas, avec le seul `sessionId` public, sonder l'existence d'une session vivante. Même principe côté scan : un `scanSecret` invalide et une session inconnue renvoient tous deux 404.

52.3 Cycle de vie

Statut	Déclencheur
pending	créée par le nouvel appareil, <code>expiresAt</code> = maintenant + 90 s
scanned	un appareil authentifié a résolu le jeton
approving	réserve interne pendant l'approbation, jamais exposée
approved	confirmation explicite — crée la ligne <code>appareils</code> , puis livre les jetons une seule fois
denied	refus explicite. L'entrée est conservée jusqu'à son TTL : la supprimer afficherait « expiré » au demandeur, un message trompeur
expired	statut <i>synthétique</i> — l'entrée n'est plus dans la Map, qu'elle ait expiré ou déjà livré ses jetons

À expiration, le nouvel appareil régénère seul une nouvelle session — l'utilisateur ne voit jamais un code mort.

Réserve atomique de l'approbation. L'approbation n'est pas instantanée : elle enchaîne plusieurs `await` — lecture du compte, écriture dans `appareils`, signature des jetons. Deux confirmations quasi simultanées franchiraient donc toutes deux la garde de statut, et la **seconde écraserait le résultat de la première** : l'appareil demandeur ouvrirait le compte du second approbateur. `qrLoginSessions.beginApproval()` bascule le statut en `approving` *avant* le premier `await` ; comme rien n'est asynchrone entre la lecture et l'écriture du statut, la transition est atomique en JavaScript mono-thread. La seconde confirmation reçoit alors un 409. `abortApproval()` rend la session à son statut précédent si l'approbation échoue en cours de route — compte introuvable, `recordLogin` en échec — pour qu'un incident ne condamne pas un code encore valide.

Le cas où la session expire pendant l'approbation. `approve()` peut échouer après que la ligne `appareils` a été créée. Le contrôleur teste donc son retour : en cas d'échec, il **supprime la ligne** qu'il vient d'insérer et répond 410 `QR_SESSION_EXPIRED`. Sans ce test, on annoncerait « approuvé », on émettrait une paire de jetons que personne ne viendrait chercher, et on laisserait un appareil orphelin dans la liste « Appareils connectés ».

52.4 L'exception d'authentification socket

Constat vérifié dans le code. Chaque handler socket existant — chat, appels, réunions, accusés de réception — ouvre par la même garde : `if (!socket.authenticated) return;`. C'est la règle universelle du fichier `server.js` et de `src/socket/handlers/`.

Le nouvel appareil n'est, par définition, pas encore authentifié quand il attend une approbation. Un nouveau handler `src/socket/handlers/qrLogin.js` déroge donc — seule exception du dépôt — à cette garde : `socket.on('qr_session:join', ...)` valide directement contre la Map de `qrLoginSessions.js` plutôt que contre une session utilisateur, puis rejoint la room `qr_session_${sessionId}`.

Une exception bornée par construction. Elle exige le `pollToken` (section 52.2), pas le `scanSecret` : photographier le QR ne suffit donc pas à entrer dans la room. Le socket ne devient jamais `authenticated`, ne rejoint jamais `user_*` et ne reçoit donc aucun autre événement du serveur. La room ne transporte que le *statut*, jamais de jeton — la livraison des jetons reste sur le *poll* HTTP à usage unique. Et la session s'auto-détruit au bout de 90 secondes. Quatre bornes, aucune reposant sur la bonne volonté du client.

Le repli FCM ne fonctionne pas ici. `notificationService.js::sendToUser` cible par `alanyaID` — une donnée que le nouvel appareil n'a, par construction, pas encore. Le repli habituel (push en complément du socket, comme pour les appels) est donc **impossible** sur ce chemin précis. Le seul canal qui fonctionne avant authentification est celui que l'appareil a lui-même ouvert : la room socket, ou le polling REST.

Décision v1 : polling seul côté mobile. Le socket authentifié de `TalkyApiClient` suppose systématiquement un jeton d'accès valide — toute la mécanique de reconnexion et de *watchdog* est bâtie autour d'une session déjà établie. Faire cohabiter un mode « anonyme, scellé à un jeton de 90 s » dans cette même classe est un risque de régression sur un composant déjà complexe. **Le client mobile v1 n'utilise que le polling REST** (`GET /api/auth/qr-session/:sessionId`, toutes les 2 s) — le délai perçu est négligeable face au temps humain de déverrouiller l'autre téléphone et de confirmer. La room `qr_session_${sessionId}` reste implémentée côté serveur : elle ne coûte rien et prépare la voie web (chapitre 54.6), qui peut se permettre un socket ouvert en continu.

ttlMs plutôt qu'une simple date de fin. La création renvoie `expiresAt` et `ttlMs`. Le client décompte à partir de `ttlMs` et de l'instant de *réception*, sans jamais comparer son horloge à celle du serveur. Un téléphone dont l'horloge avance de plus de 90 secondes — ce qui n'a rien d'exotique — verrait sinon son code expiré dès son affichage, et le régénérerait en boucle sans jamais laisser le temps de le scanner.

Confirmation explicite, jamais d'approbation automatique. Rappel technique du choix produit : `POST /api/qr/resolve` sur un jeton de type `login` ne fait que passer le statut à `scanned` et renvoyer les informations de l'appareil demandeur — il est *strictement en lecture seule* côté autorisation. Seul `POST /api/auth/qr-session/:sessionId/approve`, appelé après une action explicite de l'utilisateur sur un écran dédié, crée la ligne `appareils` et émet les jetons. Sans cette séparation en deux appels, le scan seul suffirait à connecter — exactement le scénario de « quishing » que la partie I met en garde.

Les informations affichées viennent d'un appelant non authentifié. `deviceName` et `platform` sont fournis par l'appareil qui *crée* la session — lequel n'est, par construction, pas encore connecté. Ils s'affichent pourtant tels quels sur l'écran de confirmation, seul rempart contre le quishing. La plateforme est donc contrainte à un vocabulaire fermé **dès la création** (et non au moment de l'insertion en base, trop tard car après l'approbation), et le libellé libre est borné à 60 caractères puis présenté côté client comme « déclaré par l'appareil », jamais comme un fait vérifié.

CHAPITRE 53

API

Trois routeurs : `src/routes/qr.js`, monté sur `/api/qr` par `server.js`; `src/routes/authCustom.js`, qui accueille les chemins `/api/auth/qr-session*` et `/api/auth/device-sessions*` — exactement comme il accueille déjà `push-devices` et `notification-prefs`; et `src/routes/qrLanding.js`, les pages **publiques** servies à la racine du domaine (`/q/c/`, `/q/u/`, section 51.3) ainsi que les deux fichiers d'association des liens universels, `/.well-known/assetlinks.json` et `/.well-known/apple-app-site-association` (404 tant que l'empreinte de signature Android, resp. le Team ID Apple, ne sont pas fournis en variables d'environnement — le lien ouvre alors le navigateur, dégradation et non panne).

Endpoint	Garde	Effet
POST <code>/api/qr/contact-token</code>	<code>authCustom</code>	jeton éphémère du compte; renvoie <code>{token, payload, ttlMs, expiresAt}</code> et invalide le précédent
GET <code>/api/qr/me</code>	<code>authCustom</code>	<i>mint-if-absent</i> de <code>qr_public_id</code> ; 403 <code>BUSINESS_ONLY</code> tant que le verrou est fermé
POST <code>/api/qr/me/regenerate</code>	<code>authCustom</code>	nouveau jeton permanent, l'ancien devient invalide; même 403 que ci-dessus
POST <code>/api/qr/resolve</code>	<code>authCustom + qrResolveLimiter</code>	résout un jeton scanné (contact éphémère, permanent ou login)
GET <code>/q/c/:token, /q/u/:token</code>	aucune (public)	pages d'accueil des codes contact — jamais de consommation ni d'écriture
POST <code>/api/auth/qr-session</code>	<code>authLimiter</code> (public)	crée une session; renvoie <code>scanSecret, pollToken, payload, expiresAt, ttlMs</code>
GET <code>/api/auth/qr-session/:sessionId</code>	en-tête <code>X-Poll-Token</code> (public)	interrogation par le nouvel appareil; livre les jetons une seule fois
POST <code>/api/auth/qr-session/:sessionId/approve</code>	<code>authCustom + scanSecret</code>	crée appareils, émet les jetons
POST <code>/api/auth/qr-session/:sessionId/deny</code>	<code>authCustom + scanSecret</code>	marque denied
GET <code>/api/auth/device-sessions</code>	<code>authCustom</code>	liste des appareils non révoqués
DELETE <code>/api/auth/device-sessions/:id</code>	<code>authCustom</code>	révocation immédiate

Module	Rôle
src/controllers/qrController.js	/api/qr/* — jeton éphémère, verrou du permanent, résolution, notification du propriétaire
src/controllers/qrLandingController.js	pages publiques /q/c/ et /q/u/
src/controllers/qrAuthController.js	/api/auth/qr-session* et /api/auth/device-sessions* : un seul contrôleur, parce que les deux familles manipulent le même objet — la session d'un appareil — l'une pour l'ouvrir, l'autre pour la fermer
src/services/deviceSessionService.js	recordLogin / touchLastActive / normalizePlatform — la table appareils, partagée avec login et register
src/services/contactService.js	addContactByFriendId (option addedVia), partagé avec POST /api/contacts/:id
src/utils/qrToken.js	jetons opaques, payloads, comparaison à temps constant
src/socket/state/qrLoginSessions.js	la Map des sessions de connexion éphémères
src/socket/state/qrContactTokens.js	la Map des jetons de contact éphémères, un seul actif par compte

53.1 Limitation de débit

Route	Limiteur	Pourquoi ce calibre
POST /api/qr/contact-token	aucun limiteur dédié	Route authentifiée et création autolimitante : un seul jeton actif par compte, chaque création invalide le précédent (section 51.1).
POST /api/auth/qr-session	authLimiter (10 / 15 min / IP)	Création, <i>une</i> par code affiché. Le limiteur a skipSuccessfulRequests : seuls les échecs consomment le quota.
POST /api/qr/resolve	qrResolveLimiter (30 / min / IP)	Un humain qui scanne dépasse rarement quelques codes par minute.
GET /api/auth/qr-session/:sessionId	un limiteur dédié , de l'ordre de 60 / min / IP	Interrogation toutes les 2 s pendant 90 s : environ 45 appels <i>par code affiché</i> .

Ne jamais poser authLimiter sur l'interrogation de statut. Le réflexe serait d'aligner l'interrogation sur la création, toutes deux publiques. Il casserait la fonctionnalité : **authLimiter** autorise 10 requêtes par 15 minutes, quand un seul code en consomme environ 45 en 90 secondes. L'utilisateur verrait son écran se figer au bout de vingt secondes, puis échouer — et le symptôme ressemblerait à une panne réseau, pas à un plafond. Cette route a besoin d'un limiteur *dimensionné pour du polling*, de l'ordre de 60 requêtes par minute et par IP : assez large pour deux appareils derrière un même NAT, assez étroit pour rendre coûteux le sondage massif de **sessionId**. **À ce jour, aucun limiteur dédié n'est posé sur cette route** : elle n'est couverte que par le **generalLimiter** global (300 / min / IP), qui la protège d'un abus grossier mais n'est pas calibré pour elle.

CHAPITRE 54

Client Flutter

54.1 Couche données

Fichier	Contenu
lib/api/qr_api.dart (<i>nouveau</i>)	Extension <code>QrApi</code> on <code>TalkyApiClient</code> , part of le client, sur le modèle de <code>auth_api.dart</code> / <code>users_api.dart</code> : <code>createContactQr</code> (le code éphémère de « Mon code »), <code>resolveQr</code> , et les dormants <code>getMyQr</code> / <code>regenerateQr</code> — 403 BUSINESS_ONLY tant que le verrou du permanent est fermé, gardés pour les comptes business.
lib/api/qr_auth_api.dart (<i>nouveau</i>)	Extension <code>QrAuthApi</code> : <code>createQrSession</code> , <code>pollQrSession</code> (qui pose l'en-tête <code>X-Poll-Token</code>), <code>approveQrSession</code> , <code>denyQrSession</code> , <code>listDeviceSessions</code> , <code>revokeDeviceSession</code> . Séparée de la précédente parce qu'elle vise <code>/api/auth/*</code> et qu'une partie de ses appels se fait sans jeton d'accès.
lib/models/qr_models.dart (<i>nouveau</i>)	<code>QrIdentity</code> , <code>QrContactCode</code> (<code>token</code> , <code>payload</code> , <code>ttl</code>), <code>QrResolveResult</code> (<code>isSelf</code> , <code>added</code> , <code>alreadyContact</code>), <code>QrContactScan</code> (<code>by</code> , <code>alreadyMutual</code>), <code>QrLoginCreated</code> , <code>QrLoginPollResult</code> , <code>AppareilSession</code> .
lib/api/users_api.dart (<i>modifié</i>)	<code>addContact(userId, {viaQr})</code> : l'ajout <i>en retour</i> envoie <code>{addedVia: 'qr'}</code> à la route existante.
lib/providers/auth_provider.dart (<i>corrigé</i>)	<code>login()</code> / <code>register()</code> transmettent désormais <code>hardwareId: await TalkyApiClient.currentHardwareId()</code> — l'identifiant matériel du chapitre 49, jusqu'ici absent malgré le paramètre déjà accepté.
lib/api/socket_api.dart (<i>modifié</i>)	Écoute <code>auth:device_revoked</code> , <code>auth:conflict</code> et <code>qr:contact_scanned</code> — ce dernier exposé en flux <code>qrContactScans</code> , consommé par le dialogue global <i>et</i> par l'écran « Mon code ».

Ce qui passe par Drift, et ce qui n'y passe pas. Le code personnel ne touche pas le cache : éphémère par nature, il se redemande au serveur à chaque affichage. La liste des appareils non plus : donnée de **sécurité**, un cache périmé — appareil révoqué encore affiché, appareil suspect masqué — serait activement trompeur. L'**origine** des contacts, elle, est exactement une donnée de cache : `lib/core/db/app_database.dart` passe de `schemaVersion` 15 à **16**, `local_users` gagne `addedViaQr` (booléen) et `preferredAddedAt` (date, nullable) — que des `addColumn`, le cache existant n'est pas vidé.

User.addedViaQr est nullable à dessein. Côté modèle (`lib/talky_models.dart`), `addedViaQr` est un `bool? : null` signifie « la source ne portait pas cette information » (résultat de recherche, instantané d'appel), pas « faux ». `_partialUserToCompanion` (`local_cache_repository.dart`) traduit `null` en `Value.absent()` : une écriture partielle laisse la colonne telle quelle au lieu de l'écraser. Un booléen non nullable aurait fait perdre la pastille au premier passage d'une source qui ignore la relation — silencieusement, et de façon intermittente.

54.2 Interface

Fichier	Objet
<code>screens/profile/qr_scanner_screen.dart</code> (nouveau)	Scanner plein écran (<code>mobile_scanner</code>), gabarit <code>screens/chats/camera_screen.dart</code> . Autonome : appelle <code>resolveQr</code> , branche selon <code>type</code> . Carte de résultat par-dessus la caméra restée vivante, import d'une image de la galerie (<code>analyzeImage</code>).
<code>screens/profile/qr_code_screen.dart</code> (nouveau)	« Mon code » : deux onglets, <i>Mon code</i> et <i>Scanner</i> ; code éphémère, compte à rebours, partage lien ou image, régénération.
<code>widgets/qr_scan_result_card.dart</code> (nouveau)	Carte de résultat du scanner : avatar, Message / Voir détails / Annuler, disparition différée. Sans bouton d'appel : à côté d'« Annuler », une action bruyante et irréversible invite à la faute de doigt.
<code>widgets/qr_added_sheet.dart</code> (nouveau)	La même carte, en feuille modale : confirmation d'un ajout venu d'un <i>lien</i> — l'utilisateur revient d'un navigateur et doit voir <i>qui</i> a été ajouté.
<code>widgets/qr_pin.dart</code> (nouveau)	La pastille « ajouté par QR » (section 54.4).
<code>core/services/qr_contact_flow.dart</code> (nouveau)	Flux d'ajout partagé scan / lien : écriture du cache, carte ou bandeau, annulation (<code>undoAdd</code>).
<code>core/services/qr_deep_link_service.dart</code> (nouveau)	Réception des liens <code>/q/c/</code> et <code>/q/u/</code> (section 54.6).
<code>widgets/alanya_qr_view.dart</code> (nouveau)	Rendu unique des QR — identité comme connexion.
<code>screens/profile/qr_login_confirm_screen.dart</code> (nouveau)	Fiche de l'appareil scanné, Confirmer / Refuser.
<code>screens/authentication/qr_login_screen.dart</code> (nouveau)	QR de session, compte à rebours fondé sur <code>ttlMs</code> , <code>Timer.periodic</code> de <i>polling</i> toutes les 2 s.
<code>screens/profile/connected_devices_screen.dart</code> (nouveau)	Liste appareils. Le repère « Cet appareil » vient du drapeau <code>current</code> calculé par le serveur (comparaison de <code>appareils.id</code> à <code>l'appareilId</code> du jeton), pas d'une comparaison locale.
<code>screens/profile/profile_screen.dart</code>	Icône QR dans l'en-tête.
<code>widgets/add_contact_sheet.dart</code>	Bouton « Scanner un code », pastille sur les résultats.
<code>main.dart</code>	Dialogue global « ajouter en retour », rejeu des liens reçus hors session.
<code>screens/profile/preferred_contacts_screen.dart</code>	Filtre « Tous / Par QR » avec comptes.
<code>screens/chats/contact_detail_screen.dart</code>	Mention « Ajouté par QR code · date » sur la fiche.
<code>screens/chats/chats_screen.dart</code>	Pastille dans la liste des discussions (section 54.4).
<code>screens/profile/account_security_screen.dart</code>	Sections « Appareils connectés » et « Lier un appareil ».

Rendu du QR. `qr_flutter` (`QrImageView`) permet `eyeStyle` et `dataModuleStyle` personnalisés (points arrondis, couleur `brandPrimary`), un `embeddedImage` pour le logo, et un niveau de correction d'erreur `QrErrorCorrectLevel.H` (30 % de redondance) — nécessaire pour tolérer l'occlusion du logo central sans nuire à la lecture. Le scan repose sur `mobile_scanner` (CameraX et ML Kit sur Android, AVFoundation/Vision sur iOS), retenu plutôt que `qr_code_scanner` pour son socle natif et son entretien plus actif dans l'écosystème Flutter récent.

Un seul widget de rendu, et pourquoi. `alanya_qr_view.dart` existe parce que les deux rendus avaient *déjà* divergé — yeux ronds contre carrés, bicolore contre monochrome, deux fichiers de logo différents — donnant deux identités visuelles à la même fonctionnalité. Le fond blanc en thème sombre et le niveau H y sont posés une fois pour toutes : ce ne sont pas des choix esthétiques mais des conditions de lisibilité, et les laisser à chaque écran, c'était les perdre au premier oubli.

54.3 Mon code : décompte, consommation, régénération

L'écran « Mon code » appelle `createContactQr()` à l'ouverture, puis décompte à partir du `ttlMs` serveur et de l'*instant de réception* — jamais en comparant son horloge à `expiresAt`, pour la raison déjà donnée à la section 52.4 (note « `ttlMs` plutôt qu'une simple date de fin »). À zéro, il régénère sans geste ; un compteur de génération écarte les réponses tardives d'un code déjà remplacé. Il s'abonne aussi au flux `qrContactScans` : à la consommation, le code affiché est mort par définition, l'écran en recrée un aussitôt — c'est ce qui permet d'enchaîner plusieurs personnes. Le décompte est peint *sur* la carte capturée pour le partage d'image : l'image dit elle-même qu'elle est temporaire.

Partage : le lien ou l'image, jamais un mélange. Sur Android, un partage de type `image/*` transporte bien `EXTRA_TEXT`, mais la plupart des messageries l'ignorent dès qu'il y a une pièce jointe — le lien et l'Alanya ID disparaissaient silencieusement. L'écran fait donc choisir explicitement : *Partager le lien* (texte : invitation, mention de validité, Alanya ID, payload) ou *Partager l'image* (la carte en PNG, texte joint pour les applications qui l'honorent). Aucun enregistrement en galerie : une image morte en dix minutes y finirait en code cassé — l'option reviendra avec les codes permanents business.

54.4 La pastille, rendue à un seul endroit

`lib/widgets/qr_pin.dart` dessine la pastille — coin **bas-gauche**, le bas-droit appartient au point de présence, liseré à la couleur du fond porteur — et `AppAvatar` (`lib/widgets/common/app_avatar.dart`) l'expose en option `qrBadge`. Toutes les listes passant par `AppAvatar` (contacts préférés, feuille d'ajout, nouvelle discussion, sélection de membres, partage de contact, clavier d'appel...), la marque est identique partout : la poser écran par écran, c'était la perdre au premier oubli — même argument que `alanya_qr_view.dart` ci-dessus.

La liste des discussions n'a pas de User. Ses lignes sont des conversations, pas des contacts : `chats_screen.dart` observe donc `watchPreferredContacts()` et en dérive un `Set` d'identifiants ajoutés par QR, mis à jour au fil de l'eau — une lecture Drift déjà chaude, aucune requête par ligne, et la pastille suit automatiquement un ajout ou une annulation.

54.5 Le dialogue global d'ajout en retour

L'abonnement au flux `qrContactScans` qui porte le dialogue oui/non vit dans `AuthWrapper` (`main.dart`), pas sur un écran : le code a pu être partagé par lien, et son propriétaire peut être n'importe où dans l'application quand le scan survient — un abonné visible seulement sur « Mon code »

raterait l'événement (flux *broadcast*, sans mémoire tampon). `alreadyMutual` vrai court-circuite le dialogue en simple information. Accepter appelle `addContact(..., viaQr: true)` puis met le cache à jour en écriture partielle ; un 409 (déjà ajouté entre-temps, par double scan ou depuis un autre appareil) est présenté comme un succès, pas comme une erreur.

54.6 Liens profonds

Le QR encode une URL `https` précisément pour qu'un scan hors application mène quelque part (la page publique du chapitre 51) ; les liens profonds referment la boucle en ramenant cette URL dans l'app :

1. **Android** : `intent-filter` sur `www.alanya237.com` avec `pathPrefix /q/u/` et `/q/c/` (`AndroidManifest.xml`) ;
2. **iOS** : le fichier `apple-app-site-association` servi par le backend déclare `/q/u/*` et `/q/c/*` ;
3. **schéma applicatif** `alanya://q/<genre>/<jeton>`, utilisé par le bouton « Ouvrir dans Alanya » de la page publique — il fonctionne sans empreinte de signature ni Team ID.

`QrDeepLinkService` (`app_links`) extrait du lien un couple (**genre, jeton**) — `extractIdentityToken`, genres `c` et `u` — sans jamais le résoudre lui-même : `QrContactFlow` reconstruit le payload et appelle `POST /api/qr/resolve`, exactement comme après un scan caméra. Un lien reçu hors session est mis de côté (`stashToken`) et rejoué à la connexion ; un lien de type `login` est ignoré par construction — l'approbation d'un appareil exige un scan et l'écran de confirmation, jamais un lien cliqué.

Rien dans l'API n'est spécifique au mobile : une session de connexion se crée, s'interroge et s'approuve par de simples appels REST, et la room socket `qr_session_${sessionId}` (section 52.4) fonctionne identiquement pour n'importe quel client. La colonne `appareils.platform` traite déjà `'web'` comme une valeur de premier ordre — exactement le même choix que fait `user_push_devices.platform` depuis la migration 020.

Un onglet de navigateur serait même un **meilleur** client que l'application mobile pour ce flux : il reste ouvert en continu, ce qui rend le canal socket — volontairement laissé de côté côté mobile (section 52.4) — immédiatement utile plutôt qu'à préparer pour plus tard. Seul le rendu du code changerait (un `canvas` ou un `svg` plutôt que `qr_flutter`). Aucun écran web n'est conçu dans ce volet.

CHAPITRE 55

Impact fichier par fichier

55.1 Backend — Alanya-Backend

Fichier	Nature	Objet
migrations/026_qr_appareils.sql	nouveau	qr_public_id, appareils
migrations/027_pref_contact_added_via.sql	nouveau	preferredContact.added_via
src/utils/qrToken.js	nouveau	Jetons opaques, QR_BASE_URL, les trois payloads, secretMatches
src/services/contactService.js	nouveau	addContactByFriendId (option addedVia) et sanitizeUrl, extraits d'addPreferredContact
src/services/deviceSessionService.js	nouveau	recordLogin, touchLastActive, normalizePlatform
src/socket/state/qrLoginSessions.js	nouveau	Map en mémoire des sessions de connexion, sur le modèle de pendingCalls.js
src/socket/state/qrContactTokens.js	nouveau	Map des jetons de contact éphémères (10 min, usage unique, un seul actif par compte); test qrContactTokens.test.js chaîné dans test:qr
src/controllers/qrController.js	nouveau	/api/qr/* — jeton éphémère, verrou du permanent, résolution, notification du propriétaire
src/controllers/qrLandingController.js	nouveau	Pages publiques /q/c/ et /q/u/
src/controllers/qrAuthController.js	nouveau	/api/auth/qr-session* et /api/auth/device-sessions*
src/routes/qr.js	nouveau	Monte /api/qr, dont contact-token
src/routes/qrLanding.js	nouveau	Monte /q/* et les fichiers d'association /.well-known/*
src/socket/handlers/qrLogin.js	nouveau	qr_session:join, seule exception à la garde d'authentification
server.js	modifié	Montage des routes (dont les pages publiques à la racine), enregistrement du handler socket
src/routes/authCustom.js	modifié	Routes qr-session* et device-sessions*
src/middleware/rateLimiter.js	modifié	qrResolveLimiter
src/middleware/authCustom.js	modifié	Payload JWT étendu, vérification revoked_at
src/middleware/auth.js	modifié	Idem, pour rester cohérent avec authCustom.js
src/socket/handlers/auth.js	modifié	Même vérification côté socket; socket.appareilId
src/controllers/authCustomController.js	modifié	register / login : recordLogin obligatoire, 503 sinon; refreshToken :

Trois fichiers que le dossier avait prévus et qui n'existent pas. La conception initiale annonçait `src/constants/qrSessionStatus.js`, `src/services/qrIdentityService.js` et `src/services/qrLoginService.js`. Aucun n'a été écrit, et c'est délibéré : les statuts sont des chaînes déjà exhaustivement énumérées par les transitions de `qrLoginSessions.js`, et les deux services n'auraient fait que déléguer, l'un à `qrController`, l'autre à `qrLoginSessions`. Trois indications vides valent moins qu'un contrôleur qu'on lit d'un bout à l'autre.

55.2 Mobile — Talky (Flutter)

Fichier	Nature	Objet
lib/api/qr_api.dart	nouveau	Jeton éphémère et résolution
lib/api/qr_auth_api.dart	nouveau	Sessions de connexion et appareils
lib/models/qr_models.dart	nouveau	Modèles de la couche données QR, dont <code>QrContactCode</code> et <code>QrContactScan</code>
lib/screens/profile/qr_code_screen.dart	nouveau	Mon code (compte à rebours, partage lien / image)
lib/screens/profile/qr_scanner_screen.dart	nouveau	Scanner (carte de résultat, import galerie)
lib/widgets/alanya_qr_view.dart	nouveau	Rendu unique des QR
lib/widgets/qr_scan_result_card.dart	nouveau	Carte de résultat
lib/widgets/qr_added_sheet.dart	nouveau	La même carte, en feuille modale (ajout par lien)
lib/widgets/qr_pin.dart	nouveau	Pastille « ajouté par QR »
lib/core/services/qr_contact_flow.dart	nouveau	Flux d'ajout partagé scan / lien, annulation
lib/core/services/qr_deep_link_service.dart	nouveau	Liens <code>/q/c/</code> et <code>/q/u/</code> → (genre, jeton)
lib/screens/profile/qr_login_confirm_screen.dart	nouveau	Confirmation
lib/screens/authentification/qr_login_screen.dart	nouveau	QR de session
lib/screens/profile/connected_devices_screen.dart	nouveau	Mes appareils
android/.../alanya237/alanya/MainActivity.kt	modifié	Canal natif <code>com.alanya/device_identity</code> → <code>ANDROID_ID</code>
android/.../main/AndroidManifest.xml	modifié	<code>intent-filter /q/u/</code> et <code>/q/c/</code>
lib/talky_api_client.dart	modifié	<code>currentHardwareId()</code> , part des deux nouvelles extensions
lib/providers/auth_provider.dart	corrigé	<code>hardwareId</code> transmis au login / register
lib/api/auth_api.dart	modifié	Champ <code>hardware_id</code> dans le corps
lib/api/users_api.dart	modifié	<code>addContact(..., viaQr:)</code>
lib/api/socket_api.dart	modifié	<code>auth:device_revoked</code> , <code>auth:conflict</code> , <code>qr:contact_scanned</code>
lib/talky_models.dart	modifié	<code>SocketEvents</code> , <code>authDeviceRevoked</code> / <code>qrContactScanned</code> , <code>AccountDeviceLogin</code> , <code>User.addedViaQr</code> / <code>preferredAddedAt</code>
lib/core/db/app_database.dart	modifié	<code>schemaVersion 15</code> → <code>16</code> : <code>addedViaQr</code> , <code>preferredAddedAt</code>
lib/core/services/local_cache_repository.dart	modifié	Écritures partielles de l'origine, <code>watchPreferredContacts</code>
lib/main.dart	modifié	Dialogue global « ajouter en

Administration — hors périmètre. Aucun écran `talky-admin` n'est nécessaire à ce volet. Une vue support « appareils connectés d'un compte » serait une extension naturelle mais n'a pas été demandée — signalée au chapitre suivant, pas conçue ici.

CHAPITRE 56

Vérification

56.1 Backend

Appliquer `026_qr_appareils.sql` et `027_pref_contact_added_via.sql` **avant** de déployer le code, redémarrer, puis :

1. `npm run test:qr` → les deux stores en mémoire passent (`qrLoginSessions`, `qrContactTokens`);
2. `POST /api/qr/contact-token` deux fois → deux jetons différents, et le *premier* ne résout plus (un seul actif par compte); `GET /api/qr/me` et `POST /regenerate` → 403 `BUSINESS_ONLY`; résoudre un `/q/u/` même valide en base → 404;
3. résoudre le jeton `/q/c/` d'un autre compte → ligne `preferredContact` avec `added_via = 'qr'`, événement `qr:contact_scanned` reçu par le propriétaire (`alreadyMutual` correct), et le **re-jouer** → 404 : le jeton est consommé. Propriétaire sans socket au premier plan → push `qr_contact_scanned`; au premier plan → pas de push;
4. scanner *son propre* jeton → 200 `{self:true}`, jamais une erreur — et le jeton **survit**; même survie après le 403 d'un compte bloqué;
5. `GET /q/c/:token` dans un navigateur → la page d'identité, et le jeton résout toujours ensuite (la page ne consomme pas); `GET /q/u/:token` → « Ce code n'est plus valide »;
6. `POST /api/contacts/:id` avec `{addedVia:'qr'}` → ligne marquée `qr`; avec `{addedVia:'pigeon'}` → ligne marquée `search` (liste blanche);
7. créer une session QR, la laisser expirer → `status = expired` au polling suivant;
8. interroger une session avec un `X-Poll-Token` erroné → `expired`, indistinguable d'une session inconnue; sans en-tête du tout → 400;
9. créer une session, la résoudre depuis un second compte, **approuver** → ligne `appareils` créée, jetons valides, `appareilId` présent dans leur charge utile. Rejouer l'interrogation → `expired` : les jetons ne sont livrés qu'une fois;
10. approuver **deux fois** la même session, quasi simultanément → la seconde reçoit 409, jamais une seconde paire de jetons;
11. `DELETE /api/auth/device-sessions/:id` sur un appareil actif → sa requête authentifiée **sui-vante** échoue immédiatement, socket fermée; le même appareil qui se reconnecte obtient une ligne au `id` **différent**.

56.2 Client

`flutter pub get` → `flutter run`.

1. Ouvrir « Mon code » → compte à rebours à 10 :00, fondé sur `ttlMs`; à zéro, un code neuf s'affiche sans geste.
2. Scanner le code d'un contact → ajout instantané, carte de résultat avec « Annuler » fonctionnel, caméra toujours vivante derrière; enchaîner un second scan sans ressortir de l'écran. Rejouer le même code → « code inconnu ou expiré ».

3. Côté propriétaire, au scan : dialogue « ajouter en retour » où qu'on soit dans l'app (accepter → pastille des deux côtés) ; app en arrière-plan → notification ; scanneur déjà en contact → simple information. L'écran « Mon code », s'il est ouvert, affiche un code neuf.
4. Vérifier la distinction : pastille sur l'avatar dans les listes *et* la liste des discussions, mention « Ajouté par QR code · date » sur la fiche, filtre « Tous / Par QR » avec les bons comptes — y compris après redémarrage de l'app (cache Drift).
5. Importer depuis la galerie une capture d'un code → même carte de résultat que par la caméra ; une image sans code et un code d'une autre app → deux messages distincts.
6. Scanner son propre code, puis un code expiré → messages distincts, caméra jamais bloquée.
7. Connecter un second appareil de bout en bout : code affiché, scan depuis le premier, fiche correcte, confirmation, jetons reçus par polling.
8. **Thème sombre** : la carte du QR personnel reste blanche.
9. Réinstaller l'app sur le même appareil physique : sur **Android**, il ne réapparaît *pas* comme un second appareil dans la liste (`ANDROID_ID` stable). Sur iOS, il réapparaît si c'était la dernière application du vendeur — limite connue.
10. Déconnecter un appareil depuis un autre : l'appareil visé se déconnecte **tout seul** à la réception d'`auth:device_revoked`, sans attendre un appel réseau. Refaire le test *application fermée* : il doit alors tomber au premier 401.
11. Régler l'horloge du téléphone avec 5 minutes d'avance, puis afficher un QR de connexion : le compte à rebours doit partir de 90 s (`ttlMs`) et non expirer immédiatement.

CHAPITRE 57

Points de vigilance et questions ouvertes

Ajout instantané : le garde-fou est devenu structurel. La règle produit — ajouter sans confirmation — garde pour seul filet côté scanneur la carte de résultat annulable. Mais le régime éphémère borne désormais l'exposition côté propriétaire : dix minutes de vie, usage unique, et un propriétaire prévenu à chaque consommation. Le scénario du code affiché en contexte public (vitrine, affiche) ne concerne plus que les codes permanents — à réexaminer au moment d'ouvrir le verrou business (section 49.1).

1. **Coût de la vérification en garde** (chapitre 50) : une jointure de plus par requête authentifiée, dans **trois** fichiers quasi dupliqués (`auth.js`, `authCustom.js`, `socket/handlers/auth.js`). Assumé ici ; leur fusion serait une occasion naturelle, mais indépendante de ce volet — et le risque, en l'état, est qu'une évolution future n'en corrige que deux sur trois.
2. **Pas de limiteur dédié sur l'interrogation de statut.** `GET /api/auth/qr-session/:sessionId` n'est couvert que par le `generalLimiter` global (300 / min / IP). Un limiteur calibré pour du *polling* (~60 / min / IP) reste à poser — en aucun cas `authLimiter`, qui casserait la fonctionnalité.
3. **`auth:conflict` a désormais un émetteur.** L'événement, qui existait côté mobile sans jamais être émis, est maintenant diffusé par `authCustomController.js` (connexion par mot de passe) et par `qrAuthController.js` (approbation d'une session QR). Son nom reste discutable : il annonce une *nouvelle connexion sur le compte*, pas un conflit. Le renommer reste ouvert, et coûterait un déploiement coordonné des deux côtés.
4. **Aucune limite** au nombre d'appareils simultanés n'est proposée. Rien dans la demande ne l'impose ; un garde-fou futur (avertissement au-delà d'un seuil, à la manière de certains services bancaires) resterait compatible avec ce schéma.
5. **`push_device_id` n'est écrit par personne.** La colonne existe, nullable, et *aucun* code ne la renseigne à ce jour — ni le client, ni le backend. Elle reste posée pour le badge « notifications activées », qui demanderait de faire remonter le `deviceId` push au moment du login. En attendant, l'écran « Appareils connectés » doit se comporter comme si elle était toujours vide (aucun badge, pas d'erreur), ce qui est le cas aujourd'hui. Rappel : un utilisateur qui refuse les notifications n'aura de toute façon jamais de correspondance dans `user_push_devices`.
6. **L'identifiant matériel n'est pas un secret.** `ANDROID_ID` est diffusé à tout le compte dans `auth:device_revoked`, et `identifierForVendor` est lisible par toute application du même vendeur. Cela reste acceptable : connaître le `device_id` d'un appareil ne permet ni de s'y substituer ni de le révoquer — ces deux gestes demandent un jeton valide du compte. Mais c'est une raison de plus pour que l'API n'expose jamais ce champ dans la liste des appareils, ce qu'elle ne fait pas (pas plus que l'adresse IP).
7. **La push « code scanné » informe, elle ne rejoue pas.** Le dialogue d'ajout en retour n'est déclenché que par l'événement `socket qr:contact_scanned`, reçu app ouverte. La notification `qr_contact_scanned` (émise seulement quand aucun socket n'est au premier plan) n'est pas mise en attente côté client : un propriétaire qui ouvre l'app *après* l'avoir balayée ne revoit pas l'invitation — il lui reste la pastille et la fiche du nouveau contact pour l'ajouter à la main. Limite assumée ; un jeu demanderait de router ce type de push jusqu'au dialogue.

8. **Le verrou business est un TODO nommé.** `_qrPermanentAutorise` refuse tout le monde tant que `account_type` (volet 1) n'existe pas (section [49.1](#)). Toute l'infrastructure du permanent — colonne, *mint-if-absent*, régénération, page publique, extensions client dormantes — attend derrière ce seul prédicat.

Ce qui reste à construire

Les six volets s'appuient sur des briques d'infrastructure qui **n'existent pas aujourd'hui** dans les trois dépôts. Les rassembler ici évite qu'elles apparaissent comme des surprises au moment du développement.

Briques d'infrastructure

Brique	Volet	Pourquoi elle est nécessaire
File de jobs	3, puis 4	Fan-out des notifications de diffusion, puis transcodage des vidéos de fiche. Table MySQL avec <code>SKIP LOCKED</code> — aucun service supplémentaire à héberger.
Index spatial	4	Recherche par proximité. Aucune donnée géographique n'existe ; la colonne doit être <code>NOT NULL</code> avec un <code>SRID</code> explicite dès la création.
Coffre à documents	5	Chiffrement au repos, service authentifié, purge à 90 jours, journal d'accès. La brique la plus sensible du lot.
Journal de vues	4	Tableau de bord du commerçant. Anti-rebond, agrégation et purge à concevoir <i>avec</i> la table, pas après.
Stockage objet	4	Les médias de fiche sont publics et servis en boucle. Le disque local tiendra la mise en service, pas au-delà.
Révocation de session	6	Transforme le refresh token JWT actuel — jamais révocable — en jeton réellement révocable. Condition pour qu'un bouton « Déconnecter » ait un effet immédiat.

Points relevés dans l'existant

Constatés en lisant le code, sans rapport direct avec les fonctionnalités demandées, mais susceptibles de gêner leur implémentation.

Constat	Conséquence
Aucun exécuteur de migrations, ni table de suivi	Rien ne garantit que l'état réel de la base corresponde au contenu de migrations/. À vérifier avant d'appliquer 023.
migrations/013_view_once.sql contient du SQL invalide	La colonne viewedAt n'a jamais pu s'appliquer par ce fichier, alors qu'elle figure dans le schéma consolidé. Trancher lequel décrit la production.
typeCompte codé en dur à 0 en trois endroits du mobile	Un badge dérivé y afficherait faux. À corriger dans le même lot que le volet 1.
uploads/ est servi en statique par server.js	Une pièce d'identité déposée par le chemin habituel serait publiquement téléchargeable. Stockage distinct obligatoire.
auth:conflict défini et écouté côté mobile (lib/talky_models.dart, lib/api/socket_api.dart) mais jamais émis par le backend	Résolu par le volet 6 : l'événement est désormais émis à chaque connexion du compte (authCustomController.js, qrAuthController.js) et remonté à l'UI.
Les migrations 023 à 025 réellement présentes (023_message_delivered_at.sql, 024_groups_roles_mentions.sql, 025_email_change_otp.sql) ne correspondent plus aux numéros réservés par les volets 1 à 3, jamais appliqués	Le premier numéro réellement libre aujourd'hui est 026 — le volet 6 le prend. Les volets 1 à 3 devront être renumérotés au moment de leur implémentation réelle.

Ordre de réalisation recommandé



Le socle conditionne les volets 3, 4 et 5. Les listes de contacts n'en dépendent que pour la numérotation des migrations et du schéma local : elles peuvent être livrées en parallèle, et constituent le cycle le plus court. L'identité par QR code (volet 6) n'a aucune arête entrante : elle porte un axe *session/authentification*, orthogonal à l'axe *qui est ce compte* que porte le socle — elle peut se construire dès maintenant, en parallèle des cinq autres.

Fin du dossier de conception.

Les six volets existent aussi en documents autonomes, dans le même dossier.